

I/O from Python to the operating system — PB158 Lecture 2

Contents

1 The claim

2 1. What does one byte cost?

2.1 Why your laptop will disagree

3 2. Where does the difference come from?

3.1 Why a crossing is expensive

3.2 Reading a trace

3.3 Counting, and the honest limit on the count

3.4 What a buffer does, and when it stops paying

3.5 Trusting the instrument

3.6 Windows counts too, and differently

3.7 Counting on Windows: ask the kernel

4 3. It is not one buffer — it is a stack

4.1 One rule instead of four facts

4.2 One program, one buffer — C and Python take different paths

4.3 The bottom level, and the limit on the thesis

4.4 The demonstration, and what it does not prove

4.5 Which boundaries a tool can actually watch

5 4. The same write, at three levels

5.1 The demo program

5.2 `fwrite` takes a size and a count

6 5. Two operating systems, side by side

6.1 Why the boundaries sit where they do

6.2 What the evidence does not cover

7 6. What we have not looked at

7.1 Buffering moves your errors

7.2 Networking and IPC

7.3 Honest limits

8 What follows

8.1 Assignment 1

9 References

10 Appendix — sources from the author's working notes

10.1 V1 — The cost structure of a system call

10.2 V2 — System calls as a cache- and TLB-shredding CPU mechanism, not a slower function call

10.3 V3 – System-call count as the first-order I/O cost driver

10.4 V4 – The Write()-to-Disk Durability Gap

10.5 V5 – The Non-Guarantee Behind a Successful write() or WriteFile() Call

10.6 V6 – Destination-dependent stdio buffering in C and Python

10.7 V7 – Stdio buffer sizing on regular files

10.8 V8 – Where Windows and Linux place the stable syscall boundary

10.9 V9 – The Layer at Which Windows and Linux Guarantee Syscall Stability

10.10 V10 – The System Call Boundary

10.11 V11 – Syscall batching via io_uring and SQPOLL

10.12 V12 – The gap between an OS call succeeding and its work being done

10.13 V13 – The trustworthiness gap between strace's call counts and its timings

10.14 V14 – The Gap Between a Successful Write and a Durable One

10.15 V15 – Leaky Operating-System Abstractions

10.16 V16 – Observation as the Method for Diagnosing Leaks in Operating-System Abstractions

10.17 V17 – Tracing Mechanisms for Observing a Running Process

10.18 V18 – The four-level memory allocation hierarchy

10.19 V19 – Virtual memory and the page cache

10.20 V20 – File I/O and durability

10.21 V21 – NtTrace, a debug-interface Windows API tracer

10.22 V22 – NtTrace, a user-mode Windows native-API tracer

10.23 V23 – NtTrace, a user-mode Windows native API tracer

10.24 V24 – WiredTiger's fsync() retry-to-panic fix

10.25 V25 – The production-unsuitability of syscall-boundary tracing (ptrace / strace)

10.26 V26 – The System Call Boundary: Mechanism versus Contract

10.27 V27 – The System Call as a Distinct Hardware Mechanism, Not a Costly Function Call

10.28 V28 – The kernel-object model behind sockets, and where each platform pins its contract

10.29 V29 – The kernel-object model behind sockets on Linux and Windows

10.30 V30 – The buffering stack between a program's write and durable storage

10.31 V31 – Mechanisms for observing a running system's interaction with the OS

10.32 V32 – Virtual memory and the page cache as a fault-driven mechanism

10.33 V33 – File I/O durability across the OS buffering stack

10.34 V34 – Virtual memory and the page cache

10.35 V35 – NtTrace, a user-mode Windows Native API tracer

10.36 V36 – FreeBSD's ABI Compatibility Stance

10.37 V37 – The four-layer buffering-and-durability boundary model for file I/O

10.38 V38 – Where Windows and Linux each place their stable syscall boundary

10.39 V39 – The Trade-off Between Observation Cost and Blind Spots in Tracing Mechanisms

10.40 V40 – The 2018 fsync() Error-Reporting Failure ("Fsyncgate")

10.41 V41 – Linux's one-shot fsync() error report and its 2018 database-engine fallout

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Reading this document: *[S<n>] cites the numbered source list at the end. [inference] marks a claim that follows from those sources but is not stated by them. [speculative] marks the author's judgement, offered as judgement.*

This document stands on its own. You do not need the slides to read it, and it goes further than the lecture did in several places. [V<n>] entries are notes from the author's research vault. The appendix at the end summarises each one and lists every external source it draws on.

1 The claim

The call you write and the call the operating system runs are not the same call [inference]. `fflush()` flushes only the C library's buffers, and the kernel buffers need a second flush [S3]. Three of the four boundaries below your code have a tool that watches them:

- the application calling into the library [V10]
- the library calling into the kernel [S1]
- the kernel writing to the device [V30]

The fourth is where data is lost [V20], and no tool in this lecture sees it [V5]. I argue it in three steps. First I measure. Then I count. Then I look.

Four claims should survive a week.

1. Buffering is fast when writes are adjacent [inference]. It buys little when they are not. Lampson gave the batching half in 1983 [S12].
2. A library call is not a system call [S3]. Several levels sit between the call you type and the bytes on a medium.
3. The boundary is observable [S1]. A tracer shows which system call your source line became, and how many times it ran.
4. Linux and Windows nominate different levels as the stable boundary [V8].

2 1. What does one byte cost?

Two Python programs write the same mebibyte — 1,048,576 bytes — to a file. The first opens the file with `buffering=0`. The second takes the default. Python documents the argument: "Pass 0 to switch buffering off (only allowed in binary mode)" [S6].

The two programs hold everything else fixed:

- the loop
- the interpreter
- the iteration count
- the bytes
- the filesystem

That design matters. It eliminates "Python is slow" by construction rather than by argument, because the interpreter does the same work in both runs [inference].

The author prints no measured ratio, in the lecture or in this document. A benchmark the reader did not watch being produced is exactly the kind of claim this course teaches students to distrust [speculative]. The figure also differs between the lecture machine and a student laptop, for reasons covered below. Run it. Write down both wall-clock times. Record the Python version and `io.DEFAULT_BUFFER_SIZE` beside them [inference].

One caveat belongs on the measurement. A one-mebibyte file does not reach a disk in any detectable way. It lands in the operating system's own memory and stays there. The honest rule is to "state which cache layer your number includes, because you cannot eliminate all of them" [V19]. So this measures the cost of asking the operating system to do something, not the cost of a disk [inference].

2.1 Why your laptop will disagree

The size of Python's default buffer is not a constant. It was 8192 bytes for sixteen years. The CPython issue that changed it records that the value "was set to 8192 since 16 years ago" [S7]. The issue gives the reason for that choice. In 2008 "you'd run a 32 bits operating system with less than 2 GB memory available" [S7].

The maintainers then raised it on a measured "3 to 5 times write performance improvement" [S7]. **State the baseline with that multiplier.** It compares larger buffers against the 8192-byte default, on Python 3.11. The benchmark sweeps the buffer from 0 bytes up to 1 MiB [S7]. The workload writes a file streamed over HTTP and times only the write calls. The reporter ran it on local Linux storage, on an NFS home directory and on

Windows [S7]. The figure is the reporter's own measurement, not an independent reproduction [inference]. A core developer confirmed the direction in the same thread and posted no figures [S7]. Python now documents that "On most systems, the buffer will typically be 128 kilobytes long" [S6].

The practical consequence for assignment 1 is direct. A cohort split between Python 3.13 and 3.14 produces crossing counts that differ by a factor of sixteen [inference]. That reason has nothing to do with what the assignment measures [inference]. Report the version and the buffer size with every number.

The general habit is the point. Never quote a remembered constant about a system. Measure the machine in front of you. Say which machine it was [speculative].

3 2. Where does the difference come from?

The difference is the number of times each program crosses into the operating system. You can already define a system call. The author names what he counts instead. Every **crossing** is one system call: "a privilege transition, not a jump" — the processor changes protection domain and your process keeps running [V15].

The arithmetic is simple. The unbuffered program makes one call per byte, so one mebibyte costs 1,048,576 calls. The buffered program makes one call per buffer fill, so one mebibyte divided by the buffer this machine allocated. At 8 KiB that is 128 calls. At 128 KiB it is 8 [inference]. **Print the size before you divide.** Python allocates `max(min(blocksize, 8 MiB), DEFAULT_BUFFER_SIZE)` when the device block size is available [S6]. So the divisor depends on the filesystem as well as on the Python version [inference].

3.1 Why a crossing is expensive

Soares and Stumm measured the processor footprint of individual Linux system calls in FlexSC, at OSDI 2010 [S16]. A single `stat` call "retires 4,972 instructions in 13,585 cycles — an IPC of 0.37 — while evicting 32 L1 instruction-cache lines, 186 L1 data-cache lines, 660 L2 lines, 2,559 L3 lines, and 21 data-TLB entries" [V2]. **Those are 2010 figures, and the counts are not current** [V2]. What carries over is the regime distinction below, not the numbers [inference].

An IPC of 0.37 means the processor finished one instruction every three cycles. Straight-line code on the same processor finishes more than one per cycle, so this is slow work rather than ordinary work [speculative].

The instruction count is not the story. The evicted cache lines belonged to your program, and every one is a miss your own code pays for after the call returns [inference]. The paper separates two regimes, and the distinction matters here. The indirect cost dominates "for medium frequencies of system call invocation (once per 2,000 to 100,000 instructions)", and at higher rates the direct transition cost takes over [S16]. Program A makes one system call for every loop iteration, which plausibly puts it in that high-frequency regime [speculative]. The paper measures no Python program, and nobody here has counted the interpreter's instructions per call, so the placement is the author's [speculative]. Either way the cost is largely fixed per call. Doubling the bytes in one call is nearly free. Doubling the calls is not [inference].

3.2 Reading a trace

A **trace** records the system calls a program actually made, in order, with arguments and return values. The `strace` manual page documents the format and uses this example line [S1]:

```
open("/dev/null", O_RDONLY) = 3
```

Name, then arguments in parentheses, then the return value after the equals sign. Every line looks like this. The `3` here is a file descriptor, defined in section 4 below.

A trace is not a log. A log carries what the program's author chose to print. A trace carries what the program did [inference].

3.3 Counting, and the honest limit on the count

Running the two programs again under `strace -c -e trace=write python3 a.py` counts the calls. The counts match the arithmetic above, and that is the point of the exercise [inference]. You derive a number from a documented buffer size, then confirm it with an independent tool [inference].

The count ratio is not the time ratio, and the author says so explicitly in the lecture.

Total cost "is better modelled as $N_{\text{crossings}} \times \text{fixed_cost} + \text{bytes} \times \text{marginal_cost}$ than as a function of bytes alone. For small payloads the first term dominates" [V1]. Both programs write the same mebibyte through the same million-iteration loop [inference]. The second term and all the interpreter work form a floor under both [inference]. Under that model the count ratio over-predicts the time ratio [inference]. A count ratio of a hundred thousand against a time ratio of forty is not a mistake [inference]. Those two figures are an illustration rather than a measurement from this room [inference].

The source note grades the model confident rather than verified. Per-crossing costs vary by an order of magnitude across machines. Treat the model as a shape rather than a formula [V1].

3.4 What a buffer does, and when it stops paying

A buffer collects **adjacent** bytes and hands them down in one call. The saving sits entirely in the call count. The same bytes are copied either way [inference].

Now the condition. A buffer holds bytes destined for consecutive positions, because a block write carries one offset and one length. Seek somewhere else and those bytes cannot stay in it [inference]. They belong at the old offset, so the library must hand them over before it can move [inference]. With one byte written per seek the buffer never holds more than that byte, and the buffered run converges on the unbuffered one [inference].

So the rule is not "buffering is fast". It is *buffering is fast when writes are adjacent, and buys little when they are not* [inference]. Lampson stated the batching half in 1983: "Use batch processing if possible. Doing things incrementally almost always costs more, even aside from the fact that disks and tapes work much better when accessed sequentially" [S12]. He writes about physical media and sets that point aside from the batching advice. The claim that a seek empties a userspace buffer is this deck's, and it follows from the mechanism above [inference].

Two limits, both load-bearing. Adjacency is necessary and not sufficient: a program already writing a mebibyte in one call has nothing left to collect [inference]. And the seek itself costs a crossing. `pread` and `pwrite` "combine[s] file access and seek operations into a single command... thereby reducing the number of system calls issued from the user environment" [V3]. The call count is the measured part. For identical data volumes the reported result is "a relatively reduced load in terms of overall duration" [V27]. That is the source's own wording. It is weaker than a measured runtime figure, and the author does not upgrade it [inference]. **Why those calls were added is a different question, and the source does not answer it.** The documented rationale is an atomic offset-and-access for threads that share a descriptor [speculative]. An unbuffered random-access run that writes one byte per seek makes two crossings per byte, not one [inference].

3.5 Trusting the instrument

The tracer stops the traced program twice per system call — once before the call runs, once after the return value is set. Both of the tool's properties follow from that single mechanism [inference].

Because it stops every call that reaches the kernel, it counts every one of them. Nothing is sampled. It cannot count what never crosses: Linux serves some calls from a page shared with the process, with no privilege transition, and "strace cannot see them, because there is nothing to intercept" [V26]. Because it stops every call, it also makes the program dramatically slower. The worst case on record is Brendan Gregg's: "dd if=/dev/zero of=/dev/null bs=1 count=500k", a program doing nothing but syscalls — runs 442× slower under strace, and does so even when tracing a syscall the program never makes, because the stop is paid regardless" [V17]. Re-runs of that same dd loop report "roughly 37× to 136×" on different hardware and with different speculative-execution mitigations [V31]. **The later range does not contain Gregg's figure, and that is the point rather than a contradiction.** Overhead is proportional to the syscall rate, so each figure measures one machine running one workload rather than the tool [inference].

The rule for this course: count under the tracer, time without it. That "is why the counts are trustworthy and the absolute times are not" [V13]. A benchmark run under a tracer reports nonsense [inference].

3.6 Windows counts too, and differently

Neither platform needs a tracer to answer the question the benchmark asks. Both kernels already keep a per-process write count, and reading it costs one file read on Linux or one call on Windows [inference]. The counting summary is a feature of strace alone. NtTrace, the tracer this course uses on Windows, is a different design and prints no equivalent [speculative].

On Windows, `GetProcessIoCounters` "Retrieves accounting information for all I/O operations performed by the specified process" and fills an `IO_COUNTERS` structure [S24]. The member that answers the benchmark's question is `WriteOperationCount`. Microsoft documents it as "The number of write operations performed" [S23]. One process may also read another process's count. The handle must carry the `PROCESS_QUERY_INFORMATION` or `PROCESS_QUERY_LIMITED_INFORMATION` access right [S24]. That is why a ready-made process viewer can display the same counts with no code of your own [inference]. Task Manager shows them on its Details tab [speculative].

Linux keeps a similar count in `/proc/pid/io`, a file that "contains I/O statistics for the process and its waited-for children" [S25]. The manual page heads the field "syscw: write syscalls". It defines the field as the number of "file write" system calls — "those from the `write(2)` family, `sendfile(2)`, `copy_file_range(2)`, and `ioctl(2)` `BTRFS_IOC_ENCODED_WRITE[_32]`" [S25]. A program reads its own count through `/proc/self/io`, which is what `demo/iodemo.py` does. On Windows the same program calls `GetProcessIoCounters` through `ctypes`. Both `demo/iodemo.py` and `demo/iodemo.c` print the count when given `--count`. A third program, `demo/iocount_win32.py`, reads the same structure through the `PyWin32` package, which the course does not install.

Three limits, and all three belong beside the count.

First, the two fields are not one measurement. The Linux page states what it counts: "file write" system calls [S25]. Microsoft states "The number of write operations performed" and stops there [S23]. So the Linux count is a count of crossings, and the Windows count cannot be described that way on the evidence the vendor publishes [inference].

Second, these are counts of I/O operations rather than of crossings. A seek is a crossing, and it is not a write. The Linux field counts calls from the write family, so it does not count a seek [inference]. The random-access program seeks once per write. Its count is therefore roughly half its crossings [inference]. The sequential program never seeks, so for that one the two agree [inference]. Microsoft defines a separate member as "The number of I/O operations performed, other than read and write operations" [S23]. The page does not say that a seek increments it, and this document will not say so either.

Third, the count covers the whole process rather than your loop. The interpreter writes before your first line runs [inference]. So take the count for one program, take it for the other, and report the ratio rather than either absolute count [inference]. The Linux file widens the scope once more, to the process "and its waited-for children" [S25]. The same page adds a caveat about reading the value at all. The counters are not atomic. On a machine where a 64-bit read can tear, a concurrent update can yield an intermediate value that was never the count [S25].

All three limits leave the count useful. It answers the benchmark's question — did the buffered program make fewer calls — and it needs no tracer to do it [inference]. The platform comparison with a real Windows trace is still Part V [S10]. Equal footing between the platforms is a course-level promise, not a per-lecture one.

3.7 Counting on Windows: ask the kernel

Section 2 counted on Linux with `strace -c`. This is the Windows path, in a program short enough to read on a slide. It is `demo/iocount_win32.py`, cut to the lines that matter:

```

import win32api
import win32process

def write_operations():
    handle = win32api.GetCurrentProcess()
    return win32process.GetProcessIoCounters(handle)["WriteOperationCount"]

before = write_operations()
with open(path, "wb") as f:
    f.write(b"one\n")
    f.write(b"two\n")
    f.write(b"three\n")
after = write_operations()
print(f"the three writes cost:    {after - before} operation(s)")

```

Why not a tracer. There is no `strace -c` on Windows. NtTrace lists the calls it traces and prints no counting summary [speculative]. So instead of watching the process from outside, the program asks the kernel for the count it keeps anyway [S24].

What it does. `GetProcessIoCounters` fills an `IO_COUNTERS` structure for the process [S24]. The member that answers the benchmark's question is `WriteOperationCount` [S23]. The previous section quotes both pages in full. What is new here is the binding. PyWin32 exposes the call as `win32process.GetProcessIoCounters(hProcess)`. It documents the call as returning "a dictionary representing an `IO_COUNTERS` struct" [S29]. That is why the sample subscripts the result. The PyWin32 page documents the argument as a "Process handle as returned by `OpenProcess`" [S29]. The sample passes `win32api.GetCurrentProcess()` instead. A program that reads its own counters needs no `OpenProcess` call [inference].

What to expect. Python collects the three writes in one buffer [S6]. The kernel therefore records one write operation rather than three, and closing the file may add one more [inference]. That step from a buffered `write()` to a counter increment is an inference rather than a documented mapping. The `IO_COUNTERS` page defines the members. It does not say which call increments which one [S23]. Three `f.write` calls in the source, one or two write operations in the kernel — that gap is what section 2 exists to show [speculative].

The lab does not need the PyWin32 package. `demo/iodemo.py --count` reads the same counter through `ctypes` on Windows, and reads `/proc/self/io` on Linux, taking the field `syscw` [S25]. The count covers the whole process rather than your loop. The interpreter writes before your first line runs [inference]. That is why the sample reads the counter twice and subtracts. The same numbers are visible with no code at all, in Task

Manager or Process Explorer [speculative]. And the limit from the previous section still stands: these are I/O operations, not crossings. A seek is a crossing and it is not a write [inference].

4 3. It is not one buffer — it is a stack

There are four levels between your call and the medium. A different call empties each one.

LEVEL	WHO OWNS IT	EMPTIED BY
1 Application	the language runtime's own buffer [S8]	<code>f.flush()</code> [S17]
2 C library	the <code>stdio</code> buffer on a <code>FILE*</code> [S3]	<code>fflush()</code> [S3]
3 Kernel	the page cache, in operating-system memory [V32]	<code>fsync()</code> [S13], or <code>Flush-FileBuffers</code> on Windows [V37]
4 Device	the drive's own write cache [V20]	a cache-flush command [V33]

To flush means to hand a buffer's contents to the next level down. It does not mean the data reached a disk. The manual page says so in its own words: "Note that `fflush()` flushes only the user-space buffers provided by the C library. To ensure that the data is physically stored on disk the kernel buffers must be flushed too" [S3]. Do not write *save*, *commit* or *persist* for this operation. All three claim a durability that flushing does not provide [inference].

The **page cache** is level 3. It is the operating system's own buffer for file data and it lives in the machine's memory. It is not the disk's cache — the disk has its own, one level further down [V20]. A busy machine shows very little free memory because the page cache has grown into it [V32]. That is correct behaviour rather than a leak [V32].

The stack is not write-only. Reads use the same four levels in the other direction. Level 3 fetches pages ahead of a sequential reader, so "a sequential `read()` just happens to return faster because the data was already fetched" [V34]. Assignment 1 measures writes, so every demonstration in the lecture is a write. The author says the read direction aloud, so that nobody leaves with a one-way picture.

4.1 One rule instead of four facts

Every level takes a large block from the level below and hands out small pieces to the level above. It buys wholesale and sells retail, and that asymmetry is why every level holds a buffer [inference]. The source states the same rule: each level "hands out subdivisions of something it bought wholesale from the level below, and each level's unit of purchase is larger than its unit of sale" [V18]. Applied to level 2: on one measured ext4 file the C library takes a buffer of 4096 bytes, that file's own block size, and sells single bytes to your loop [V7]. The gap between those units is the saving measured in section 1 [inference].

The rule travels to memory allocators, processor caches and network stacks [speculative]. That is why the lecture teaches the rule rather than the four facts. It deliberately offers no physical-world image for the page cache, because a container image reinstalls the intuition the section exists to remove [speculative].

4.2 One program, one buffer — C and Python take different paths

Each program passes through at most one user-space buffer, and the two programs do not use the same one. The C program, `demo/iodemo.c`, calls `fwrite()`. That call fills the C library's `stdio` buffer, which is level 2. One `write()` then empties that buffer into the page cache [S3] [V7]. The Python program, `demo/iodemo.py`, calls `f.write()`. That call fills a `BufferedWriter`, which is level 1. PEP 3116 named that class and specified the layers in 2007 [S8]. One `write()` empties that buffer into the page cache [S6].

So the four-level table invites a wrong reading. A Python write does not fall through level 1 into level 2. "CPython reimplements the same policy above libc rather than inheriting it" [V6]. A Python program uses level 1. A C program uses level 2.

The four levels are the union of the two paths, not the path of any one program.

Every program crosses levels 3 and 4. Each program uses at most one of levels 1 and 2, and never both [inference]. A file opened with `buffering=0` uses neither [S6].

This was a design decision rather than an accident of implementation [inference]. PEP 3116 specified "a defined raw byte-based I/O interface" with separate raw, buffered and text layers in 2007, independently of C `stdio` [S8].

Windows does not add a level between 1 and 2. CPython's `FileIO.write` "Only makes one system call" and calls `_Py_write` [S26]. `_Py_write` calls `write()`, and the `MS_WINDOWS` branch calls the same `write` as the other branch [S27]. On Windows that `write` is the C runtime's low-level entry point, and that is not the buffered `stdio` layer. Microsoft describes the low-level routines as functions that "invoke the operating system directly for lower-level operation than that provided by stream I/O" [S28]. The same

page states that "Low-level input and output calls don't buffer or format data" [S28]. PEP 3116 places Python's own buffer in the layer above. It describes the raw layer below as methods that "are wrappers around the appropriate operating system calls" [S8]. So levels 1 and 2 are the same two levels on both platforms [inference]. This says nothing about levels 3 and 4, which the rest of this section treats separately.

The sizes are the evidence a reader can check. Python documents a typical buffer of 128 kilobytes [S6]. On one measured machine, "On an ext4-backed file with `st_blksize == 4096` the allocated stdio buffer is therefore 4096 bytes, not the 8 KiB commonly quoted" [V7]. That is a measurement of one machine, and the 128 kilobytes above is documentation rather than a measurement. Two adjacent levels, two sizes a factor of thirty-two apart, and neither number appears in the source code [inference].

4.3 The bottom level, and the limit on the thesis

Level 4 acknowledges early too. "most HDDs and many SSDs acknowledge a write once it lands in an onboard DRAM cache, before it reaches NAND or platter; a power cut in that window loses it" [V20]. The specification is permissive about the repair: on `fsync`, POSIX states that "It is explicitly intended that a null implementation is permitted" [S13].

This is where the lecture's thesis stops. No trace separates a device that flushed from one that only reported it — the note's own wording is "No syscall trace can distinguish an honest flush from a fast lie" [V5]. Everything above level 4 is observable with a tool a student can run. Level 4 is not observable from software at all [inference].

4.4 The demonstration, and what it does not prove

```
import os
print("hello")
os._exit(0)
```

Run to a terminal, this prints. Redirected to a file, the file is empty. The rule sits in the C language standard rather than in the operating system: standard output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5]. A terminal is interactive, so the line goes out. A file is not, so the line waits [inference]. The third statement then ends the process without cleanup — the standard permits termination paths that "need not close all files properly" [S5]. A trace of the program holds no `write` call at all [inference].

The demonstration proves level 1 exists and can lose data. It proves nothing about levels 3 and 4. "the honest classroom demo is a power-cut, not a `kill -9`. Killing the process loses only the *runtime's* buffer; the kernel still writes back." [V14] A student who tests with a `kill` watches the file survive. The experiment does not support the conclusion that flushing was enough [inference].

4.5 Which boundaries a tool can actually watch

- **Application to library** — a different tool [V10]. "`printf` is visible to `ltrace` and `write` to `strace`, and a deadlock entirely inside a library is invisible to `strace` because the kernel never hears about it". That pair makes takeaway 2 falsifiable rather than asserted [inference].
- **Library to kernel** — where the system call tracer attaches [S1].
- **Kernel to device** — block-layer tracing [V30], which this course does not cover.
- **Device to medium** — not observable from software [V5].

The justification for the whole approach sits one level up from any of these tools. No level reports which one holds the data, so counting the crossings is the only way to find out: "You cannot deduce which layer is lying; you have to make the boundary crossings visible and count them" [V16].

5 4. The same write, at three levels

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no [inference]

Each level holds a buffer, or it passes the bytes straight down. That is the mechanism section 1 measured, written out as an interface [inference].

A **file descriptor** is the small integer a Linux program uses to name an open file in system calls. It is the `= 3` in the trace line above, and it reappears as the first argument of every later call on that file [S1].

5.1 The demo program

`demo/iodemo.c` is short. It opens a file that does not exist. It then opens the file it means to write. It makes three `fwrite` calls totalling fourteen bytes. It closes the file and checks the return value of the close.

The trace shows **one** `write` call. Three source lines, fourteen bytes, one system call — takeaway 2 as an artefact rather than a claim [inference]. The bytes are adjacent, so the buffer held all of them and handed them down together [inference]. The write does not happen at the third `fwrite`. It happens when `fclose` runs, because `fclose` hands the buffer down to the kernel [inference].

Two deliberate design choices in that program are worth stating.

It prints nothing. A silent program makes a legible trace [speculative]. A Windows console may take a different path from file I/O. No source in this registry establishes that, so the author does not assert it.

The first open is meant to fail. Without a failure both traces are success all the way down, and the most informative platform difference never appears. Both design choices are the author's own stipulations about a program he wrote, so neither carries a citation. Whether silence makes the trace easier to read is his judgement rather than a stipulation, so that sentence carries a marker.

5.2 `fwrite` takes a size and a count

`fwrite("three\n", 1, 6, f)` passes six items of one byte each. The return value counts items, not bytes, and equals a byte count only when the item size is 1 [S21]. Fewer than requested is legal at the system call below: "Note that a successful write() may transfer fewer than count bytes" [S2]. The library call has its own version of that contract, reporting fewer items than requested, and the two are not the same guarantee [inference]. Ignoring either return value is the difference between having written the bytes and believing you did [inference].

6 5. Two operating systems, side by side

The Windows tool is NtTrace rather than Process Monitor. Microsoft describes Process Monitor as showing "real-time file system, Registry and process/thread activity" [S18]. Those are filesystem events rather than system calls [inference]. Comparing that tool with a system call tracer would attribute a tool difference to the platform [inference]. NtTrace watches the native API in `ntdll`, which is the last user-mode layer before the

kernel [S10]. **That is not the same attachment point as `strace`**, which sits at a documented kernel boundary. The gap between the two attachment points is itself this section's subject [inference].

The layer above the native API is Win32, the documented interface a Windows program calls [S19]. The 32 is history rather than a width [inference]. Microsoft renamed the interface the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22].

Its design choice matters for a teaching lab. It produces its trace "via the debug interface so the tool needs no driver and no administrator rights" [V21]. The alternative on Windows is a kernel-mode tracer, which needs a signed driver and a boot-time change to signature enforcement. A teaching lab does not grant either, so that tool cannot be run at all [inference].

The author excerpts three lines from the worked trace published on the tool's own project page [S10]. The published trace carries further lines between them. Three things in it are structural, and none depends on the age of the trace [inference].

A handle is not a file descriptor. The trace shows `FileHandle=4` and `FileHandle=0x14` [S10]. A handle is an opaque value naming an open object, and the object need not be a file. A file descriptor is a small integer indexing a per-process table [S1]. They occupy the same position in a trace and are different mechanisms [inference].

Two error namespaces arrive in one return value. The trace carries `=> 0xc0000034` [2 'The system cannot find the file specified.'] — an `NTSTATUS` together with the Win32 error it translates into [S10]. Linux has one `errno` and nothing to translate, because only one layer names errors there [inference].

The object path shows the layer. "The `\\??\\` prefix is the object-manager namespace, which a Win32 program never types. This is the layer difference, visible in one string." [V22]

6.1 Why the boundaries sit where they do

Microsoft states its own policy: the NT entry points are "internal to the operating system and subject to change from one release of Windows to another" [S9]. Linux treats the system call interface as a public contract. The accurate summary is neither a ranking nor a complaint: "Both are stable at the layer they nominate. The disagreement is about *which layer is nominated*" [V8].

Why the Windows boundary sits where it does is contested, and the author does not settle it [inference]. One reading is design. NT had three operating environments: POSIX, OS/2 and Win32 [S19]. Each served its own clients through its own interface.

Each reached NT through the native API [S19]. An unstable lower layer is then a consequence rather than an accident [inference]. The other reading is vestige: Windows had a POSIX subsystem built "to meet 1980s era DoD Orange Book and FIPS specifications to sell to the defense market", and "The team focus shifted to Win32 so much that the Client-Server Runtime Subsystem (CSRSS) that hosts the Win32 API became mandatory and OS/2 and POSIX subsystems were never really completed" [S15]. That second account is one practitioner's recollection on a personal blog rather than a first-party history [S15]. Teach the mechanism as settled and the intent as contested [speculative]. Linux is the clearer case: it "historically could not control userspace... so it had to make the kernel boundary itself the stable interface" [V38].

And the tempting conclusion is backwards. "Linux is the only mainstream system that treats the raw syscall interface as a public contract." [V9] FreeBSD, OpenBSD, NetBSD, illumos and macOS publish no equivalent promise. **They are not equally strict about it.** FreeBSD "offers strong ABI compatibility *in practice* via compiled-in compatibility layers, while declining to state a formal cross-major-release guarantee" [V36]. The list does not claim five equally unstable systems [V36]. Say *different*, never *worse* [inference].

6.2 What the evidence does not cover

The Linux and Windows traces are not a matched pair. "It is not a matched pair." — the published trace follows the Windows command shell rather than the course's demo program [V23]. It is also "a 32-bit, XP-era capture", so a run on a lab machine today shows 64-bit addresses [V35]. The tool has a single maintainer [inference]. Microsoft documents the layer it watches as subject to change between releases [S9].

The matched pair does not exist yet. The author will record both traces of one program before the first lab and put them in the course repository. The claims above survive without that pair, because each one is structural [inference].

7 6. What we have not looked at

`write()` returns success and the bytes are not on a disk. "A successful return from `write()` does not make any guarantee that data has been committed to disk" [S2].

That is legitimate rather than broken. Saltzer, Reed and Clark gave the general form in 1984: "The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the com-

munication system... (Sometimes an incomplete version of the function provided by the communication system may be useful as a performance enhancement.)" [S11] Every level in the stack is an instance of that parenthesis [inference].

When it is actually safe is the durability lecture's question, and this lecture deliberately leaves it open.

7.1 Buffering moves your errors

`fwrite` returned success because the bytes reached a buffer. Nothing about the device is known yet [inference]. The failure surfaces where the library flushes, which means it arrives at `fclose`, attached to no particular write [inference]. You cannot handle the error where you caused it, because only the call that flushes can report it [inference].

This is why `demo/iodemo.c` checks the return value of `fclose`. Almost no real code does [speculative]. The idiom that hides it is universal: "the most common durability idiom in every mainstream language — open a file, write, let a `with` block or `using` block or destructor close it — contains no durability step anywhere" [V4]. A Python `with` block does at least raise if the close fails [inference]. The two languages differ in what they force a programmer to notice [inference].

Do not retry. "Retrying the `close()` after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4]. The same conclusion was reached one level down in 2018, at `fsync`, for a different reason. "The response was uniform across engines: PostgreSQL, InnoDB, and WiredTiger all replaced retry with **crash/abort** during 2018" [V40]. MongoDB's WiredTiger engine wrote the reason into its source comment: "we don't retry `fsync`-style calls and panic if a flush operation fails" [V24]. PostgreSQL, MySQL's InnoDB and MongoDB's WiredTiger are three database engines with three different architectures, and all three chose a deliberate crash over a retry [V41]. That is how little a failed `fsync` tells you about the state you are in [inference].

This is *error reporting*. *Error recovery* belongs to the durability lecture.

7.2 Networking and IPC

A socket on Linux is a file descriptor, and `read` and `write` work on it. "Linux publishes that handle as a **file descriptor** and makes the file-descriptor contract binding. Windows publishes it as an **opaque SOCKET** and makes the *Winsock API* the contract, even though the NT kernel underneath is also holding a file object" [V28]. That is section 5's pattern arriving a second time, in a place a student does not expect it [inference]. Pipes are file descriptors too, so one program can wait on a socket and a pipe at

the same time: "Because sockets, pipes, terminals, `epoll` instances, `timerfd`s and `eventfd`s are all file descriptors, a single event loop can wait on all of them at once" [V29]. Depth belongs to the next lecture — the point here is only that networking is not a separate subject.

7.3 Honest limits

- **The device level is not observable from software** [V5]. "No syscall trace can distinguish an honest flush from a fast lie".
- **Asynchronous I/O breaks the model** [V11]. "with `IORING_SETUP_SQPOLL` a kernel-side polling thread means steady-state I/O submission needs **no syscalls at all**". The tracer sees the setup and nothing after it [inference].
- **strace is not a production instrument** [V25]. It has a second and cheaper mode, selected with the `--seccomp-bpf` option. On the workload of rebuilding `strace` itself, plain `strace` costs 3.06× the untraced run. That mode costs 1.44× [V39]. A contributor disqualifies it anyway — `strace --seccomp-bpf` "still stops at each syscall of interest and should therefore not be used on production systems!" [V25]. Production Linux uses eBPF and production Windows uses event tracing [speculative]. This registry carries no per-call cost for either, so no figure is given here [inference]. `strace` and NtTrace watch the layer this lecture is about, which is why the course uses them.
- **Memory mapping and crash dumps** are named so that nobody leaves believing I/O is finished [speculative]. Each has its own lecture.

8 What follows

1. **Ask "accepted or done?" of every fast call** [V12]. A fast return reports that the next level accepted the bytes. It does not report that the work is complete. The same structure governs a message broker acknowledging a message [speculative].
2. **Count, do not reason** [V16]. "You cannot deduce which layer is lying; you have to make the boundary crossings visible and count them".
3. **Name the level** [inference]. "Where is my data" has four answers — your own buffer, the C library's buffer, the page cache, and the device cache.
4. **Measure your own machine** [S7]. The default buffer size held at 8192 bytes for sixteen years, then moved by a factor of sixteen [S7].

8.1 Assignment 1

Assignment 1 measures takeaway 1 — buffered against unbuffered, sequential against random, over a 1 MiB file, with the explanation written in Markdown. The two programs from section 1 are the starting point [inference].

Report `io.DEFAULT_BUFFER_SIZE` and the Python version beside every number. Count under the tracer, in one run [V17]. Time the program without the tracer, in a second run [V17]. Say in the write-up that you did both.

The counting tool is Linux-only, so the counts come from the course dev container on either platform. When the count ratio and the time ratio disagree by orders of magnitude, explain the gap rather than assuming a mistake [inference].

9 References

- [S1] *strace(1)*. Linux man-pages project. <https://man7.org/linux/man-pages/man1/strace.1.html>
- [S2] *write(2)*. Linux man-pages project. <https://man7.org/linux/man-pages/man2/write.2.html>
- [S3] *fflush(3)*. Linux man-pages project. <https://man7.org/linux/man-pages/man3/fflush.3.html>
- [S4] *close(2)*. Linux man-pages project. <https://man7.org/linux/man-pages/man2/close.2.html>
- [S5] *C99 draft*, §7.19.3. Publicly hosted working draft. <https://busybox.net/~landley/c99-draft.html>
- [S6] *Built-in Functions* — `open()`. Python documentation. <https://docs.python.org/3/library/functions.html#open>
- [S7] *Increase `io.DEFAULT_BUFFER_SIZE`*. CPython issue 117151. <https://github.com/python/cpython/issues/117151>
- [S8] *PEP 3116 — New I/O*. Python Enhancement Proposals (2007). <https://peps.python.org/pep-3116/>
- [S9] *winternl.h reference*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- [S10] *NtTrace — Native API tracing for Windows*. Roger Orr, project page. <https://roger-orr.github.io/NtTrace/>
- [S11] Saltzer, J. H., Reed, D. P. and Clark, D. D. *End-to-End Arguments in System Design* (1984), p. 2. <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

- [S12] Lampson, B. W. *Hints for Computer System Design* (1983), p. 19. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>
- [S13] `fsync`. The Open Group Base Specifications. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>
- [S14] Taber, K. *Predict-Observe-Explain*. science-education-research.com. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- [S15] Reiter, B. *The sad history of the Microsoft POSIX subsystem* (2010). <https://brian-reiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- [S16] Soares, L. and Stumm, M. *FlexSC: Flexible System Call Scheduling with Exception-Less System Calls*. OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf
- [S17] `io.IOBase.flush()`. Python documentation, `io` module. <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- [S18] *Process Monitor*. Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>
- [S19] Russinovich, M. *Inside Native Applications*. Microsoft Learn, Sysinternals (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- [S20] `os` — *Miscellaneous operating system interfaces*, `os.write()`. Python documentation. <https://docs.python.org/3/library/os.html#os.write>
- [S21] `fread(3)`, the page that documents `fwrite(3)`. Linux man-pages project. <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- [S22] *Windows API index*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- [S23] `IO_COUNTERS` (`winnt.h`). Microsoft Learn, Win32 API reference. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters
- [S24] `GetProcessIoCounters` (`winbase.h`). Microsoft Learn, Win32 API reference. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiocounters>
- [S25] `proc_pid_io(5)`. Linux man-pages project, release 6.19. https://man7.org/linux/man-pages/man5/proc_pid_io.5.html
- [S26] `Modules/_io/fileio.c`. CPython source, main branch. https://github.com/python/cpython/blob/main/Modules/_io/fileio.c
- [S27] `Python/fileutils.c`. CPython source, main branch. <https://github.com/python/cpython/blob/main/Python/fileutils.c>

- **[S28]** *Low-level I/O*. Microsoft Learn, Microsoft C runtime library. <https://learn.microsoft.com/en-us/cpp/c-runtime-library/low-level-i-o>
- **[S29]** `win32process.GetProcessIoCounters`. PyWin32 documentation, Mark Hammond. https://mhammond.github.io/pywin32/win32process__GetProcessIoCounters_meth.html
- **[V1]** *A system call is not a function call* — the two-term cost model.
- **[V2]** *A system call is not a function call* — the measured footprint of one crossing.
- **[V3]** *A system call is not a function call* — `pread` and `pwrite` merge the seek.
- **[V4]** *A Successful write() Is a Handoff, Not a Save* — the idiom with no durability step.
- **[V5]** *A Successful write() Is a Handoff, Not a Save* — the device level is unobservable.
- **[V6]** *Standard output is buffered differently depending on where it points* — CPython above libc.
- **[V7]** *Standard output is buffered differently depending on where it points* — the measured 4096-byte buffer.
- **[V8]** *Windows and Linux place the stable syscall boundary at different layers* — the nomination argument.
- **[V9]** *Windows and Linux place the stable syscall boundary at different layers* — Linux as the outlier.
- **[V10]** *The System Call Boundary* — `ltrace` against `strace`.
- **[V11]** *The System Call Boundary* — `io_uring` with a polling thread.
- **[V12]** *Where the Abstraction Leaks* — accepted, not done.
- **[V13]** *Where the Abstraction Leaks* — trust the counts, not the times.
- **[V14]** *Where the Abstraction Leaks* — a kill is not a power cut.
- **[V15]** *Where the Abstraction Leaks* — a privilege transition, not a jump.
- **[V16]** *Where the Abstraction Leaks* — count, do not deduce.
- **[V17]** *Observing a Running System* — the 442-times worst case.
- **[V18]** *The Memory Hierarchy from Python to Physical Pages* — wholesale down, retail up.
- **[V19]** *Virtual Memory and the Page Cache* — name the cache level in every number.
- **[V20]** *File IO and Durability* — the device acknowledges early.
- **[V21]** *NtTrace — Native API tracing for Windows (Roger Orr)* — a debugger, not a driver.
- **[V22]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the object-manager path prefix.
- **[V23]** *NtTrace — Native API tracing for Windows (Roger Orr)* — what the published trace cannot do.
- **[V24]** *PostgreSQL's fsync() surprise* — never retry after fsync fails.

- **[V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot* — not for production.
- **[V26]** *The System Call Boundary* — the calls a tracer cannot see.
- **[V27]** *A system call is not a function call* — what merging the seek did to overall duration.
- **[V28]** *Sockets and Network I-O* — a Linux socket is a file descriptor, a Windows socket is not.
- **[V29]** *Sockets and Network I-O* — sockets, pipes and terminals are all file descriptors.
- **[V30]** *File IO and Durability* — block-layer tracing watches the kernel-to-device boundary.
- **[V31]** *Observing a Running System* — the published range for the same benchmark, on other machines.
- **[V32]** *Virtual Memory and the Page Cache* — the page cache is the machine's own memory.
- **[V33]** *File IO and Durability* — the kernel issues the device cache-flush command.
- **[V34]** *Virtual Memory and the Page Cache* — readahead moves data up the same four levels.
- **[V35]** *NtTrace — Native API tracing for Windows (Roger Orr)* — the published trace is 32-bit and XP-era.
- **[V36]** *Windows and Linux place the stable syscall boundary at different layers* — the five systems do not decline the promise equally.
- **[V37]** *File IO and Durability* — `FlushFileBuffers` is the Windows counterpart of `fsync`.
- **[V38]** *Windows and Linux place the stable syscall boundary at different layers* — why Linux could not move its own boundary.
- **[V39]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot* — what the cheaper seccomp-bpf mode costs.
- **[V40]** *PostgreSQL's fsync() surprise* — three engines dropped the retry in 2018.
- **[V41]** *PostgreSQL's fsync() surprise* — the three engines are independent of each other.

10 Appendix — sources from the author's working notes

The talk cites material from a private research vault. Each entry below summarises one such note and lists every external source it draws on.

10.1 V1 — The cost structure of a system call

The note argues that total I/O cost is better modelled as $N_{\text{crossings}} \times \text{fixed_cost} + \text{bytes} \times \text{marginal_cost}$ than as a function of bytes alone, and that for small payloads the first term dominates. This follows from reframing what a system call actually is: not a slower function call, but a distinct CPU mechanism (the `SYSCALL` instruction on x86-64, used identically by Linux and Windows) whose dominant cost is paid after the call returns, in evicted processor state. The evidence is Soares and Stumm's FlexSC measurements (USENIX OSDI 2010), which found a single `stat` call evicting 32 L1-instruction, 186 L1-data, 660 L2, and 2,559 L3 cache lines plus 21 TLB entries, with IPC falling to 0.37 — a footprint paid per crossing and largely independent of payload size, which is exactly what grounds the two-term cost model. A direct demonstration holding bytes constant makes the same point empirically: writing 100,000 characters via unbuffered `write()` versus buffered `fputc()` moves identical data, but kernel time falls from roughly 1.510s to 0.002s because the buffered version collapses roughly 100,000 crossings into a few dozen — about a 300x difference driven purely by crossing count. The note also cites `pread / pwrite` replacing `lseek + read` as a second, syscall-choice-level instance of the same effect.

The note is explicit that this model has boundaries and does not claim to be universal. It identifies several regimes where the "count crossings" heuristic breaks down: when there is no crossing at all (Linux's vDSO and Windows' `KUSER_SHARED_DATA` serve some calls entirely in user space); when batching amortizes the fixed cost toward the marginal cost (`io_uring` and Windows I/O Rings), though batching itself is not free at batch size 1, where framework overhead can exceed the syscall it replaces; when the workload is device-bound rather than crossing-bound, so I/O latency rather than crossing count or bytes is the binding constraint; and at extreme call frequency, where the direct cost of the exception and pipeline flush takes over from the indirect cache/TLB-eviction cost the model is built on. The note states outright that the crossover point between these regimes is unquantified and has to be measured per workload rather than derived from the sources it cites. It also flags that the Windows side of the cost story is comparatively unproven: the crossing mechanism is documented to the same depth as Linux's, but no comparable cache/TLB-eviction measurement exists for Windows, so the model's constants there are inferred rather than measured.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#) — Intel Corporation
- [The Definitive Guide to Linux System Calls](#) — packagecloud

- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#) — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010)
- [System calls on Windows x64](#) — n4r1b (2019)
- [Page Table Isolation \(PTI\) — The Linux Kernel documentation](#) — Linux kernel maintainers
- [KPTI/KAISER Meltdown Initial Performance Regressions](#) — Brendan Gregg, Netflix (2018)
- [Kernel page-table isolation](#) — Wikipedia
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#) — Abhinav Upadhyay (2025)
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#) — Microsoft Security Response Center (2022)
- [io_uring for High-Performance DBMSs: When and How to Use It](#) — Jasny et al. (2025)
- [SubmitIoRing function \(ioringapi.h\)](#) — Microsoft Learn
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir, Winsider Seminars & Solutions
- [io](#) — Core tools for working with streams (Python documentation) — Python Software Foundation
- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025)
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, 15-213 Introduction to Computer Systems
- [Meltdown/Spectre Benchmarks](#) — Open Telekom Cloud Image Factory — T-Systems / Open Telekom Cloud
- [On the Costs of Syscalls \(2021\)](#) — Hacker News discussion — Hacker News commenters
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021)
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024)
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium
- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021)

10.2 V2 — System calls as a cache- and TLB-shredding CPU mechanism, not a slower function call

The note argues that treating a system call as "a function call that costs more" produces bad predictions, because the dominant cost is frequently paid after the call returns, in user-mode code whose processor state the crossing destroyed. On x86-64, both Linux and Windows enter the kernel through the `SYSCALL` instruction, which loads `RIP` from a Model-Specific Register, does not preserve the caller's stack pointer, and forces fixed segment-descriptor values rather than following the normal call machinery — a different mechanism, not a slower version of the same one. The central evidence for the cost claim is a processor-structure measurement of individual Linux system calls: a single `stat` retires 4,972 instructions in 13,585 cycles — an IPC of 0.37 — while evicting 32 L1 instruction-cache lines, 186 L1 data-cache lines, 660 L2 lines, 2,559 L3 lines, and 21 data-TLB entries. The note reads the low IPC and the eviction counts as the real cost, since each evicted L3 line becomes a cache miss paid later by unrelated code, and it cites the same source's finding that this indirect cost dominates at moderate call frequency. From this it draws a design conclusion for I/O-heavy programs: minimizing the number of kernel crossings, not the number of bytes moved, is the first-order performance variable, illustrated by a buffered-versus-unbuffered write benchmark showing roughly a 300x difference in system time for identical data volumes.

The note is explicit about where this reframing stops holding. It does not claim every syscall pays this cost: Linux's `vDSO` and Windows' `KUSER_SHARED_DATA` serve certain calls entirely in user space with no kernel crossing at all, and batched interfaces such as `io_uring` or Windows I/O Rings amortize the per-call cost toward the marginal cost, so within those regimes the "just a slower call" model becomes approximately correct again. At very high call frequency it reports the opposite shift, where the direct cost of the trap and pipeline flush overtakes the indirect eviction cost. The quantitative footprint itself is flagged as dated — measured on 2010 hardware — and Linux-only: the note states no comparable measured cache/TLB eviction table exists for Windows, so Windows cost claims here are inferred from shared hardware mechanism rather than measured directly. It also preserves, rather than resolves, disagreement among its sources over the magnitude of related mitigation overhead and over whether an ordinary syscall touches the TLB at all, treating that unresolved spread as evidence that the cost is context-dependent rather than a fixed per-call constant.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#) — Intel Corporation
- [The Definitive Guide to Linux System Calls](#) — packagecloud

- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#) — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010)
- [System calls on Windows x64](#) — n4r1b (2019)
- [Page Table Isolation \(PTI\) — The Linux Kernel documentation](#) — Linux kernel maintainers
- [KPTI/KAISER Meltdown Initial Performance Regressions](#) — Brendan Gregg, Netflix (2018)
- [Kernel page-table isolation](#) — Wikipedia
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#) — Abhinav Upadhyay (2025)
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#) — Microsoft Security Response Center (2022)
- [io_uring for High-Performance DBMSs: When and How to Use It](#) — Jasny et al. (2025)
- [SubmitIoRing function \(ioringapi.h\)](#) — Microsoft Learn
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir, Winsider Seminars & Solutions
- [io](#) — Core tools for working with streams (Python documentation) — Python Software Foundation
- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025)
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, 15-213 Introduction to Computer Systems
- [Meltdown/Spectre Benchmarks](#) — Open Telekom Cloud Image Factory — T-Systems / Open Telekom Cloud
- [On the Costs of Syscalls \(2021\)](#) — Hacker News discussion — Hacker News commenters
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021)
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024)
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium
- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021)

10.3 V3 — System-call count as the first-order I/O cost driver

This note's central claim is that a kernel crossing's dominant cost is paid after the call returns, in the cache and TLB state the crossing evicted, rather than in the work the call itself performs inside the kernel — a distinction it grounds in a peer-reviewed measurement showing a single `stat` call evicting thousands of cache lines and dozens of TLB entries. Because that eviction cost is largely independent of payload size, the note's operational conclusion is that the number of crossings, not the number of bytes moved, is the variable to optimize first. The quoted span backs the sharpest applied instance of that conclusion: replacing separate seek-then-read/write calls with `pread64 / pwrite64` "combine[s] file access and seek operations into a single command... thereby reducing the number of system calls issued from the user environment," and a measured comparison on identical data volumes found this call-merging produced a real reduction in overall duration. The note treats this alongside two other concrete techniques it documents for the same underlying reason — buffering, which absorbed 100,000 unbuffered write calls into a few dozen and cut system time roughly 300-fold for identical bytes, and batched submission via `io_uring` or Windows I/O Rings, which amortizes the fixed per-crossing cost toward its marginal cost.

The note scopes this claim narrowly and says so directly. The call-merging result rests on a single arXiv preprint with documented methodology but unconfirmed peer-review status, so the note grades it as verified against that source rather than independently corroborated. More broadly, the note states that the crossing-count heuristic itself has limits it does not resolve: it stops applying when a call is served entirely from user space (Linux's vDSO, Windows' `KUSER_SHARED_DATA`) and there is no crossing to count; it weakens once batching amortizes the fixed cost, though batching adds its own overhead at very small batch sizes; it is overridden when a workload is bound by device I/O latency rather than CPU-side crossing cost; and at very high call frequencies the note states the direct cost of the exception and pipeline flush takes over from the indirect cache-eviction cost it otherwise emphasizes. The note also flags that no comparable cache/TLB footprint measurement exists for Windows, and that the exact crossover point between these regimes is not derivable from its sources — it says this has to be measured per workload rather than assumed.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#) — Intel Corporation
- [The Definitive Guide to Linux System Calls](#) — packagecloud
- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#) — Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010)

- [System calls on Windows x64](#) — n4r1b (2019)
- [Page Table Isolation \(PTI\)](#) — [The Linux Kernel documentation](#) — Linux kernel maintainers
- [KPTI/KAISER Meltdown Initial Performance Regressions](#) — Brendan Gregg, Netflix (2018)
- [Kernel page-table isolation](#) — Wikipedia
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#) — Abhinav Upadhyay (2025)
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#) — Microsoft Security Response Center (2022)
- [io_uring for High-Performance DBMSs: When and How to Use It](#) — Jasny et al. (2025)
- [SubmitIoRing function \(ioringapi.h\)](#) — Microsoft Learn
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#) — Yarden Shafir, Winsider Seminars & Solutions
- [io](#) — [Core tools for working with streams \(Python documentation\)](#) — Python Software Foundation
- [FileStreamOptions.BufferSize Property \(System.IO\)](#) — Microsoft Learn
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#) — circuitlabs (2025)
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#) — Carnegie Mellon University, 15-213 Introduction to Computer Systems
- [Meltdown/Spectre Benchmarks](#) — [Open Telekom Cloud Image Factory](#) — T-Systems / Open Telekom Cloud
- [On the Costs of Syscalls \(2021\)](#) — [Hacker News discussion](#) — Hacker News commenters
- [On the Costs of Syscalls](#) — Georg Sauthoff (2021)
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#) — Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024)
- [A Deep Dive into Zero-Copy Networking and io_uring](#) — Jatin Umamtora, Medium
- [Comment thread on syscall cost and out-of-order execution](#) — Hacker News — commenter "anarazel" (2021)

10.4 V4 — The Write()-to-Disk Durability Gap

This note argues that a successful `write()` on Linux or `WriteFile()` on Windows only hands responsibility for the data to the kernel — it is not evidence that any byte has reached a storage device. Both operating systems document the same non-guarantee independently: Linux's `write(2)` man page states that a successful return "does not make any guarantee that data has been committed to disk," and Microsoft describes `WriteFile` as writing to an internal buffer the operating system flushes to disk "on a regular basis" through a cache-manager lazy writer. The claim's sharpest consequence, and the span quoted above, is that the most common durability idiom in every mainstream language — open a file, write, let a `with` block or `using` block or destruct-or close it — contains no durability step anywhere. The note supports this by walking each layer of that idiom on both platforms: Linux's `close(2)` man page explicitly denies that closing implies a disk write; `fflush()` only moves bytes from the C library into the kernel; Python's `close()` flushes the language-level stream and goes no further; Java's and .NET's stream contracts describe resource release with no durability language at all. Nearly every source is primary — man pages, the POSIX specification, and Microsoft's own Win32 and driver documentation — and the note adds a real-world case for weight: PostgreSQL held an incorrect model of what a successful `fsync()` guarantees for roughly two decades before a 2018 investigation into kernel writeback behavior forced a fix.

The note is careful about what it does not claim. It is a statement about guarantees, not probabilities: on a healthy machine, dirty pages typically reach the device within the documented writeback window, so most programs that skip `fsync()` survive most of the time — the claim is only that this is not certain, and fails silently when it does not hold. Nor does it treat `fsync()` as a complete fix: POSIX explicitly permits an implementation that does nothing, and on some kernel versions a failed `fsync()` cannot be safely retried because the dirty pages have already been discarded. Two questions it leaves open rather than resolving are whether discarding pages after a failed writeback is a bug or a necessity — a dispute among PostgreSQL and kernel developers presented from both sides without adjudicating — and whether its own power-cut classroom experiment reproduces reliably on modern hypervisors, which it flags as unverified and needing validation before being assigned to students.

Referenced sources

- <https://man7.org/linux/man-pages/man2/write.2.html> — `write(2)` — Linux manual page, Michael Kerrisk / Linux man-pages project
- <https://man7.org/linux/man-pages/man2/fsync.2.html> — `fsync(2)` — Linux manual page, Michael Kerrisk / Linux man-pages project

- <https://man7.org/linux/man-pages/man2/close.2.html> — close(2) — Linux manual page, Michael Kerrisk / Linux man-pages project
- <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html> — fsync — The Open Group Base Specifications Issue 7, The Open Group / IEEE
- <https://docs.kernel.org/admin-guide/ext4.html> — ext4 General Information — The Linux Kernel documentation, Linux kernel documentation project
- <https://learn.microsoft.com/en-us/windows/win32/fileio/file-caching> — File Caching — Win32 apps, Microsoft Learn
- <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers> — FlushFileBuffers function (fileapi.h) — Win32 apps, Microsoft Learn
- <https://man7.org/linux/man-pages/man3/fflush.3.html> — fflush(3) — Linux manual page, Michael Kerrisk / Linux man-pages project
- <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-createfilew> — CreateFileW function (fileapi.h) — Win32 apps, Microsoft Learn
- <https://learn.microsoft.com/en-us/windows-hardware/drivers/storage/querying-for-the-write-cache-property> — Querying for the Write Cache Property — Windows drivers, Microsoft Learn
- <https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/dtrace> — DTrace on Windows — Windows drivers, Microsoft Learn
- <https://github.com/microsoft/DTrace-on-Windows> — microsoft/DTrace-on-Windows, Microsoft
- <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/commit> — `_commit` — C runtime library reference, Microsoft Learn
- <https://docs.python.org/3/library/os.html#os.fsync> — os — Miscellaneous operating system interfaces (Python 3 documentation), Python Software Foundation
- <https://docs.python.org/3/library/io.html> — io — Core tools for working with streams (Python 3 documentation), Python Software Foundation
- <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/io/FileOutputStream.html> — FileOutputStream (Java SE 25 & JDK 25), Oracle
- <https://learn.microsoft.com/en-us/dotnet/api/system.io.filestream.flush> — FileStream.Flush Method — .NET API reference, Microsoft Learn
- <https://www.usenix.org/conference/atc20/presentation/rebello> — Can Applications Recover from fsync Failures?, Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau — USENIX ATC (2020)
- <https://github.com/postgres/postgres/commit/6534d544cd77e14a93f5c779fa8ad-dee8d916a66> — PANIC on fsync() failure — PostgreSQL commit 6534d54, Thomas Munro / PostgreSQL Global Development Group

- <https://lwn.net/Articles/752063/> — PostgreSQL's fsync() surprise, Jonathan Corbet / LWN.net
- <https://kernel-internals.org/io/page-cache-writeback/> — Page Cache Writeback — Linux Kernel Internals, kernel-internals.org
- https://wiki.postgresql.org/wiki/Fsync_Errors — Fsync Errors — PostgreSQL wiki, PostgreSQL Global Development Group
- https://www.thomas-krenn.com/en/wiki/Linux_Page_Cache_Basics — Linux Page Cache Basics, Thomas-Krenn.AG wiki
- <https://winntfs.com/2012/11/29/windows-write-caching-part-2-an-overview-for-application-developers/> — Windows Write Caching — Part 2: An Overview for Application Developers, winntfs.com
- <https://www.postgresql.org/docs/19/wal-reliability.html> — PostgreSQL Documentation: 28.1. Reliability, PostgreSQL Global Development Group
- <https://superuser.com/questions/613561/disk-write-cache-buffer-and-separate-power-supply> — Disk write cache buffer and separate power supply, Super User (Stack Exchange)
- <https://danluu.com/fsyncgate/> — Fsyncgate: errors on fsync are unrecoverable, Dan Luu (archive of the pgsql-hackers thread)

10.5 V5 — The Non-Guarantee Behind a Successful write() or WriteFile() Call

The note's claim is that a successful `write()` on Linux or `WriteFile()` on Windows tells a program only that the kernel accepted its bytes, never that any storage device holds them. It grounds this in primary documentation from both platforms: the Linux `write(2)` man page states directly that a successful return "does not make any guarantee that data has been committed to disk," and Microsoft describes `WriteFile` as writing to "an internal buffer that the operating system writes to a disk ... on a regular basis." The bytes sit in a page cache or cache-manager buffer and are pushed toward the device later, on a schedule set by timers, memory pressure, or an explicit sync request, not by the write call returning. The note follows this gap to the point where the claim is hardest to escape: even after a program calls the correct flush primitive — `fsync()` or `FlushFileBuffers()` — and the kernel reports success, whether the underlying device actually committed the data to persistent media, rather than merely acknowledging the flush request, is firmware behavior invisible to any operating-system API. PostgreSQL's own reliability documentation warns that many consumer drives do not honor cache-flush commands by default. From this the note draws its conclusion: no instru-

ment available to software, including a syscall trace, can tell an honest flush apart from a device that reported completion without having done the work. That is the boundary the quoted sentence names.

The note is explicit that this is a default, not an absolute law. It lists the configurations where the claim stops holding: files opened with synchronous-write flags, a device cache that is battery- or capacitor-backed, write barriers deliberately disabled on hardware known to be protected, and application code sitting on an engine — a database, an embedded store — that already manages its own sync protocol underneath it. It also names unresolved territory rather than presenting a closed case: whether the kernel's practice of discarding dirty pages after a failed write-back is a bug or a necessity is an open, documented disagreement among kernel and PostgreSQL developers, and the note lists several explicit gaps of its own — no current measurement of how often writes are actually non-durable at a randomly chosen instant, no confirmed Windows equivalent to Linux's dirty-page counter, and no verification that the power-cut classroom experiment reproduces reliably on modern hypervisors.

Referenced sources

- [write\(2\) — Linux manual page](#)
- [fsync\(2\) — Linux manual page](#)
- [close\(2\) — Linux manual page](#)
- [fsync — The Open Group Base Specifications Issue 7](#)
- [ext4 General Information — The Linux Kernel documentation](#)
- [File Caching — Win32 apps](#)
- [FlushFileBuffers function \(fileapi.h\) — Win32 apps](#)
- [fflush\(3\) — Linux manual page](#)
- [CreateFileW function \(fileapi.h\) — Win32 apps](#)
- [Querying for the Write Cache Property — Windows drivers](#)
- [DTrace on Windows — Windows drivers](#)
- [microsoft/DTrace-on-Windows](#)
- [_commit — C runtime library reference](#)
- [os — Miscellaneous operating system interfaces \(Python 3 documentation\)](#)
- [io — Core tools for working with streams \(Python 3 documentation\)](#)
- [FileOutputStream \(Java SE 25 & JDK 25\)](#)
- [FileStream.Flush Method — .NET API reference](#)
- [Can Applications Recover from fsync Failures?](#)
- [PANIC on fsync\(\) failure — PostgreSQL commit 6534d54](#)
- [PostgreSQL's fsync\(\) surprise](#)

- [Page Cache Writeback — Linux Kernel Internals](#)
- [Fsync Errors — PostgreSQL wiki](#)
- [Linux Page Cache Basics](#)
- [Windows Write Caching — Part 2: An Overview for Application Developers](#)
- [PostgreSQL Documentation: 28.1. Reliability](#)
- [Disk write cache buffer and separate power supply](#)
- [Fsyncgate: errors on fsync are unrecoverable](#)

10.6 V6 — Destination-dependent stdio buffering in C and Python

The note's central claim is that a program's output timing is not decided by the program itself but by whichever layer sits below it, and it looks at that layer twice — once in C, once in Python. At the C level, glibc's `_IO_file_doallocate` inspects the file descriptor at first use and chooses line buffering only when it is a character device confirmed by `isatty`, otherwise falling back to full (block) buffering; this is why identical source code behaves differently on a terminal versus a pipe or file. Python does not simply inherit this behavior from the C runtime it sits on. As the note states, "CPython reimplements the same policy above libc rather than inheriting it": interactive `stdout` is line-buffered, non-interactive `stdout` is block-buffered, and `stderr` is line-buffered in both cases — a rule documented in CPython's own `sys` module reference and confirmed by the process-level `-u / PYTHONUNBUFFERED` override and the per-call `print(..., flush=True)` mechanism. The note backs this with an executed observation: on a pipe, `sys.stdout.line_buffering` reports `False` while `sys.stderr.line_buffering` reports `True`, which is offered as the reason Python's `logging` module (writing to `stderr`) keeps producing output in a container while a bare `print()` to `stdout` can go silent.

The note is explicit about what this does not establish. It notes that Python's `stderr` is line-buffered rather than fully unbuffered only since version 3.9, and flags as "contested" whether `stderr` should be called "unbuffered" at all — C99 only requires that it not be fully buffered, and Python deliberately implements the weaker guarantee. It also states plainly that nothing about this Python-specific reimplementation transfers by analogy to other managed runtimes: Java, .NET, Node, and Rust are named as needing separate verification, since a runtime that issues `write()` calls itself owns its own flush policy independent of both the C standard and any C-level override such as `stdbuf`. Separately, the note records that `stdbuf` is documented not to work reliably against CPython's own buffering layer, making `-u` the recommended external fix rather than an `LD_PRELOAD`-based one. No claim is made here about Windows UCRT's default `stdout` behavior; that question is left open elsewhere in the note.

Referenced sources

- [setbuf\(3\) — Linux manual page](#) — Michael Kerrisk / man7.org
- [The C99 Draft \(N869, 18 January 1999\), §7.19.3 Files](#) — ISO/IEC JTC1/SC22/WG14 (1999)
- [glibc source: libio/filedoalloc.c \(_IO_file_doallocate \)](#) — GNU C Library, revision glibc-2.39-288 (2024)
- [Output is buffered and lost if the program aborts · Issue #1235 · actions/runner](#) — GitHub / actions-runner issue tracker (2021)
- [sys — System-specific parameters and functions: sys.stdout / sys.stderr](#) — Python Software Foundation
- [1. Command line and environment — the -u option](#) — Python Software Foundation
- [Why pipes sometimes get "stuck": buffering](#) — Julia Evans (2024)
- [How stdbuf works](#) — Harry Marr (2022)
- [stdio buffering](#) — Pádraig Brady
- [setvbuf — C runtime library reference](#) — Microsoft Learn (2020, msvc-170)
- [Issue 13601: sys.stderr should be line-buffered when stderr is not a TTY](#) — Python issue tracker (2011–2019)
- [libio: Always use 8192 bytes as the default buffer size \[BZ #4099\]](#) — Florian Weimer / glibc patchwork (2016)
- [Implementation-Defined ISO/IEC C99 Behavior](#) — Oracle Developer Studio C User's Guide — Oracle Corporation
- [Stdout Buffering](#) — Evan Klitzke
- [Buffering in pipes](#) — Thomas Habets (2008)
- [Why won't journalctl show logs when the program output is piped?](#) — Super User (community Q&A)
- [Kubernetes "logs" does not show any output for a simple python program running in a container in a pod](#) — Stack Overflow (community Q&A, 2021)
- [Rationale for International Standard — Programming Languages — C, revision 5.10](#) — ISO/IEC JTC1/SC22/WG14 (2003)
- [coreutils source: src/stdbuf.c](#) — GNU coreutils
- [SSH output isn't line buffered?](#) — Unix & Linux Stack Exchange (community Q&A)

10.7 V7 — Stdio buffer sizing on regular files

The note establishes that a C program's `stdout` buffering behavior is chosen by the C standard library rather than the operating system, and that the choice depends on what file descriptor 1 refers to: line-buffered when attached to a terminal, fully (block) buffered when attached to a pipe or a regular file. It traces the mechanism to glibc's `_IO_file_doallocate`, which `fstat`s the descriptor and only calls `isatty()` if the result is a character device, then sizes the allocated buffer from `st_blksize` whenever that value is smaller than `BUFSIZ`. The quoted claim — that on an ext4-backed file with `st_blksize == 4096` the allocated stdio buffer is therefore 4096 bytes, not the 8 KiB commonly quoted — is the note's correction of a widely repeated figure. It is backed by reading the pinned glibc 2.39 source for that allocator function and by a local reproduction on Ubuntu 24.04/glibc 2.39: writing 6000 bytes to a redirected file and observing exactly 4096 bytes on disk mid-run, which the note treats as confirming the buffer filled and flushed once at 4096 rather than at the commonly assumed 8192. The note frames this as narrowing a folk figure rather than replacing it outright — 8192 remains real, as glibc's `BUFSIZ` and as the size used whenever `st_blksize` is not the smaller value.

The note is explicit about what this claim does not establish. It does not assert that 4096 is a universal buffer size — `st_blksize` varies by filesystem and device, and the note leaves the behavior for `stdout` attached to a socket as an open question, noting only that the code's character-device test implies full buffering without any source or measurement confirming buffer sizing in that case. It also leaves unresolved why glibc's current source tree still carries the `st_blksize < BUFSIZ` branch despite a 2016 patch proposing an unconditional 8192-byte default, stating plainly that the reconciling commit was not located. The claim is scoped to glibc specifically: the note treats destination-dependent buffering *mode* (line versus block) as a convention shared across libcs, citing Oracle's documentation as a second implementation, but it does not claim the same 4096-versus-8192 *buffer-size* mechanism holds outside glibc, and it states that nothing about libc stdio transfers by analogy to runtimes that bypass it, such as Go, Java, .NET, Node, or Rust.

Referenced sources

- [setbuf\(3\) — Linux manual page](#) — Michael Kerrisk / man7.org
- [The C99 Draft \(N869, 18 January 1999\), §7.19.3 Files](#) — ISO/IEC JTC1/SC22/WG14
- [glibc source: libio/filedoalloc.c \(_IO_file_doallocate \)](#) — GNU C Library
- [Output is buffered and lost if the program aborts · Issue #1235 · actions/runner](#) — GitHub / actions-runner issue tracker
- [sys — System-specific parameters and functions: sys.stdout / sys.stderr](#) — Python Software Foundation

- [1. Command line and environment — the `-u` option](#) — Python Software Foundation
- [Why pipes sometimes get "stuck": buffering](#) — Julia Evans
- [How stdbuf works](#) — Harry Marr
- [stdio buffering](#) — Pádraig Brady
- [setvbuf](#) — C runtime library reference — Microsoft Learn
- [Issue 13601: sys.stderr should be line-buffered when stderr is not a TTY](#) — Python issue tracker
- [libio: Always use 8192 bytes as the default buffer size \[BZ #4099\]](#) — Florian Weimer / glibc patchwork
- [Implementation-Defined ISO/IEC C99 Behavior](#) — Oracle Developer Studio C User's Guide — Oracle Corporation
- [Stdout Buffering](#) — Evan Klitzke
- [Buffering in pipes](#) — Thomas Habets
- [Why won't journalctl show logs when the program output is piped?](#) — Super User (community Q&A)
- [Kubernetes "logs" does not show any output for a simple python program running in a container in a pod](#) — Stack Overflow (community Q&A)
- [Rationale for International Standard — Programming Languages — C, revision 5.10](#) — ISO/IEC JTC1/SC22/WG14
- [coreutils source: `src/stdbuf.c`](#) — GNU coreutils
- [SSH output isn't line buffered?](#) — Unix & Linux Stack Exchange (community Q&A)

10.8 V8 — Where Windows and Linux place the stable syscall boundary

The note argues that Windows and Linux both promise application developers a stable interface to the kernel, but nominate that stability at different layers, and that the disagreement is about which layer is nominated rather than about which system is more careful. Linux fixes the promise at the syscall trap itself: syscall numbers are permanent, per-architecture entries in an in-tree table, and the kernel's own ABI documentation states the interface is "expected to never change and always be available." Windows fixes the promise one layer higher, at Win32; the Native API in `ntdll.dll` beneath it is explicitly disclaimed by Microsoft as "internal to the operating system and subject to change from one release of Windows to another," and the System Service Number beneath that is undocumented and is in fact renumbered between builds, and even between service packs within one release. The note backs this with data recomputed directly from the canonical Windows syscall table: 52% of x64 functions decrease in number at least once across versions, and 63% of functions differ between two con-

currently-shipping SKUs. So Windows guarantees a name where Linux guarantees a number, and everything downstream — where `open()` bottoms out, where a tracer such as `strace` or `NtTrace` attaches, what a stale hardcoded number does — follows from that one placement decision.

The note explicitly does not conclude that Windows is the unstable outlier; it argues the opposite, that Linux is the exception among Unix-family systems, since FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all decline a syscall-number guarantee much as Windows does. Several sub-questions are left unsettled rather than resolved: whether the Win32/`ntdll` split is deliberate design or a vestige of NT's abandoned multi-subsystem architecture; whether `ntdll` counts as a stable interface at all or only a traced convention; whether Microsoft's disclosure of `winternl.h` prototypes was compelled by the 2002 antitrust settlement or an independent decision; and whether Windows's syscall-number churn is purely mechanical, from alphabetical reordering of a service table, or partly deliberate. The note also flags that FreeBSD's classification as a no-guarantee platform is itself contested by a former core-team member, and that "Linux never breaks userspace" is a stronger claim than Linux's own stable-ABI document actually makes, since that document hedges its guarantee to two years and allows exceptions for security fixes.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#)
- [j00ru/windows-syscalls — Windows System Call Tables](#)
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#)
- [NtQuerySystemInformation function \(winternl.h\) — Win32 apps](#)
- [Calling Internal APIs — Win32 apps](#)
- [arch/x86/entry/syscalls/syscall_64.tbl](#)
- [include/uapi/asm-generic/unistd.h](#)
- [Do other Unix-like kernels have stable syscall ABIs?](#)
- [FreeBSD Vendor Information](#)
- [openbsd-tech: system call origin verification](#)
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#)
- [syscall\(2\) — NetBSD Manual Pages](#)
- [port-solaris mailing list, November 2010: syscall number reservation](#)
- [NtTrace — Native API tracing for Windows](#)
- [strace — linux syscall tracer](#)
- [DTrace on Windows — Windows drivers](#)
- [DTrace — Windows Commands](#)

- [Inside the Native API](#)
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#)
- [The Sad History of the Microsoft POSIX Subsystem](#)
- [Syscall ABI compatibility: Linux vs Windows/macOS](#)
- [Never break userspace](#)
- [Updating Syscall Tables — SysWhispers4 documentation](#)
- [CRT-free in 2023: tips and tricks — discussion](#)
- [Architecture of Windows NT](#)
- [U.S. v. Microsoft Corporation — Information on the Settlement](#)
- [Syscalls on Windows 11](#)

10.9 V9 — The Layer at Which Windows and Linux Guarantee Syscall Stability

This note establishes that Windows and Linux each promise application developers a stable interface to the kernel, but they nominate different layers as the boundary of that promise. Linux fixes the promise at the syscall trap itself: syscall numbers are permanent per architecture — `open` is number 2 and has been for the life of the x86-64 port — and the kernel's own ABI documentation states such interfaces are "expected to never change and always be available." Windows fixes the promise one layer higher, at Win32; the Native API exported by `ntdll.dll` beneath it is explicitly disclaimed by Microsoft as "internal to the operating system and subject to change," and the System Service Number beneath that is undocumented and does move — recomputed drift figures show 52% of Windows functions change number across some version pair, and 63% differ between two concurrently shipping SKUs. From this the note draws its central, more surprising point, captured in the claim that Linux is the only mainstream system that treats the raw syscall interface as a public contract: rather than Windows being the odd one out, FreeBSD, OpenBSD, NetBSD, Solaris/illumos, and macOS all decline a syscall-number stability guarantee the way Linux gives one, each in its own documented words, making Linux the outlier and Windows the normal case.

The note is explicit about what it does not claim. It does not claim Linux never breaks anything — it cites LWN's pushback on that stronger retelling, and notes that the kernel's own ABI document hedges compatibility to "at least 2 years" and permits breaking changes for security fixes. Two structural questions are treated as genuinely unsettled rather than resolved: whether the Win32/`ntdll` split is a deliberate design choice or a vestige of NT's abandoned multi-subsystem architecture, and whether Microsoft's disclosure of Native API prototypes in `winternl.h` was compelled by the 2002 antitrust

settlement or an independent decision that merely coincided with it — the note states that no source it consulted draws that causal link. It also reports FreeBSD's ABI status as a substantive, unresolved disagreement between the project's own no-guarantee wiki page and a former core-team contributor's practical dissent, and qualifies "Windows has no syscall tracer" as now only partly true, since DTrace's `syscall` provider exists but remains opt-in.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#)
- [j00ru/windows-syscalls — Windows System Call Tables](#)
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#)
- [NtQuerySystemInformation function \(winternl.h\) — Win32 apps](#)
- [Calling Internal APIs — Win32 apps](#)
- [arch/x86/entry/syscalls/syscall_64.tbl](#)
- [include/uapi/asm-generic/unistd.h](#)
- [Do other Unix-like kernels have stable syscall ABIs?](#)
- [FreeBSD Vendor Information](#)
- [openbsd-tech: system call origin verification](#)
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#)
- [syscall\(2\) — NetBSD Manual Pages](#)
- [port-solaris mailing list, November 2010: syscall number reservation](#)
- [NtTrace — Native API tracing for Windows](#)
- [strace — linux syscall tracer](#)
- [DTrace on Windows — Windows drivers](#)
- [DTrace — Windows Commands](#)
- [Inside the Native API](#)
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#)
- [The Sad History of the Microsoft POSIX Subsystem](#)
- [Syscall ABI compatibility: Linux vs Windows/macOS](#)
- [Never break userspace](#)
- [Updating Syscall Tables — SysWhispers4 documentation](#)
- [CRT-free in 2023: tips and tricks — discussion](#)
- [Architecture of Windows NT](#)
- [U.S. v. Microsoft Corporation — Information on the Settlement](#)
- [Syscalls on Windows 11](#)

10.10 V10 — The System Call Boundary

The note argues that where an operating system places its system call boundary determines what its observability tools can and cannot see, and the strace/ltrace contrast is its sharpest illustration of that claim. strace attaches at the kernel syscall entry and exit points via ptrace, so it reports only crossings that actually reach the kernel — for example, the write syscall a program issues. ltrace attaches one layer higher, hooking calls at the dynamic linker rather than the kernel, so it reports shared-library calls such as printf regardless of whether a kernel crossing ever happens. Because these two tools are bound to different layers of the same call stack, printf is visible to ltrace and write to strace — they are not reporting the same event from different angles, they are reporting genuinely different events. The note extends this to its strongest form: a deadlock that occurs entirely inside a library, before any syscall is issued, produces nothing for strace to observe, because the kernel is never invoked and so has nothing to report. This grounds one of the note's best-practice claims — pick the tracer that matches the layer a bug lives in — and is offered as evidence that observability is bounded structurally by where a tool can attach, not by how thorough the tool is.

The note does not claim strace and ltrace are interchangeable or that either gives a complete picture alone; it treats them as attaching to non-overlapping layers, each blind to what happens at the other's layer. It does not extend the ltrace/strace comparison itself to Windows tooling — the note's separate discussion of NtTrace, Process Monitor, and ETW makes a structurally analogous point (tools can only attach where a stable name exists) but is not presented as the same evidence. The claim rests on a single blog-style source about ltrace's linker-hooking mechanism, which the note itself rates "Medium" credibility as an unverified single-author practitioner account, in contrast to the "High" confidence it assigns claims backed by kernel documentation or vendor statements elsewhere. The note is also careful about scope: it does not assert that all library-internal bugs are invisible to strace in general, only that a bug which never triggers a syscall cannot be visible to a syscall-attached tracer — a narrower, definitionally true point about instrumentation reach, not a broader verdict on strace's usefulness or ltrace's superiority as a debugging tool.

Referenced sources

- <https://lwn.net/Articles/604287> — Anatomy of a system call, part 1 (David Drysdale / LWN.net, 2014)
- <https://lwn.net/Articles/604515/> — Anatomy of a system call, part 2 (David Drysdale / LWN.net, 2014)
- <https://www.felixcloutier.com/x86/syscall> — SYSCALL — Fast System Call (x86 instruction reference) (Felix Cloutier, transcribed from the Intel SDM)

- <https://kernel-internals.org/arch/x86/syscall-entry/> — x86-64 Syscall Entry — Linux Kernel Internals (kernel-internals.org)
- <https://docs.kernel.org/7.1/process/stable-api-nonsense.html> — The Linux Kernel Driver Interface (stable-api-nonsense) (Greg Kroah-Hartman / Linux kernel documentation)
- <https://lwn.net/Articles/173661/> — Add kernel ↔ userspace ABI stability documentation (Greg Kroah-Hartman / LWN.net, 2006)
- https://en.wikibooks.org/wiki/X86_Assembly/Interfacing_with_Linux — x86 Assembly / Interfacing with Linux (Wikibooks contributors)
- <https://unix.stackexchange.com/questions/473137/do-other-unix-like-kernels-have-stable-syscall-abis> — Do other Unix-like kernels have stable syscall ABIs? (Unix & Linux Stack Exchange)
- <https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/libraries-and-headers> — Libraries and Headers — Windows drivers (Microsoft Learn)
- <https://learn.microsoft.com/en-us/windows/win32/api/winternl/> — winternl.h header — Win32 apps (Microsoft Learn)
- <https://www.geoffchappell.com/studies/windows/win32/ntdll/api/index.htm> — NTDLL Functions (Geoff Chappell)
- <https://j00ru.vexillium.org/syscalls/nt/64/> — Windows X86-64 System Call Table (XP through Windows 11 and Server) (Mateusz "j00ru" Jurczyk)
- https://en.wikipedia.org/wiki/Microsoft_Windows_library_files — Microsoft Windows library files — NTDLL.DLL (Wikipedia contributors)
- <https://news.ycombinator.com/item?id=20854857> — Windows X86-64 System Call Table — discussion thread (Hacker News contributors, 2019)
- <https://strace.io/> — strace — linux syscall tracer (The strace project)
- <https://kernel-internals.org/syscalls/ptrace-interception/> — ptrace and Syscall Interception — Linux Kernel Internals (kernel-internals.org)
- <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon> — Process Monitor — Sysinternals (Microsoft Learn / Sysinternals)
- <https://rogerorr.github.io/NtTrace/> — NtTrace — Native API tracing for Windows (Roger Orr)
- <https://blog.codingconfessions.com/p/what-makes-system-calls-expensive> — What Makes System Calls Expensive (Abhinav Upadhyay, 2025)
- <https://kernel-internals.org/syscalls/vdso/> — vDSO and Virtual System Calls — Linux Kernel Internals (kernel-internals.org)

- <https://blog.packagecloud.io/system-calls-are-much-slower-on-ec2/> — Two frequently used system calls are ~77% slower on AWS EC2 (packagecloud engineering)
- <https://developers.redhat.com/articles/2023/04/12/why-you-should-use-iouring-network-io> — Why you should use io_uring for network I/O (Red Hat Developers, 2023)
- <https://github.com/python/cpython/issues/120754> — Speed up open().read() pattern by reducing the number of system calls — cpython#120754 (CPython issue tracker, 2024)
- https://timgolden.me.uk/pywin32-docs/win32file__CreateFile_meth.html — win32file.CreateFile — PyWin32 documentation (Mark Hammond / Tim Golden)
- <https://unixy.io/blog/ltrace-when-strace-is-not-enough/> — ltrace: When strace Is Not Enough (Satesh / unixy.io, 2026)
- <https://yunolay.com/system-calls-user-to-kernel-transition/> — System Calls and the User-to-Kernel Transition (Yunolay, 2026)
- <https://kindatechnical.com/operating-systems/lesson-13-windows-vs-linux-syscall-interfaces.html> — Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL (kindatechnical.com, 2026)
- <https://stackoverflow.com/questions/29440225/in-linux-x86-64-are-syscalls-and-int-0x80-related> — In Linux x86_64, are syscalls and int 0x80 related? (Stack Overflow)
- <https://www.paloaltonetworks.com/blog/security-operations/a-deep-dive-into-malicious-direct-syscall-detection/> — A Deep Dive Into Malicious Direct Syscall Detection (Palo Alto Networks / Unit 42, 2024)
- <https://gpages.juszkiewicz.com.pl/syscalls-table/syscalls.html> — Linux kernel system calls table (Marcin Juszkiewicz)
- https://www.emergentmind.com/topics/io_uring-interface — io_uring Interface Overview (EmergentMind, summarizing Jasny et al.)

10.11 V11 — Syscall batching via io_uring and SQPOLL

The note's central frame splits the "system call boundary" into a mechanism boundary (where the CPU changes privilege level) and a contract boundary (where an OS promises not to change), and one strand of its argument follows the economics of the crossing itself. Having established that a syscall's cost is dominated by microarchitectural effects — pipeline drain, cache and TLB disturbance, and post-Spectre/Meltdown mitigations — rather than by the instruction itself, the note treats `io_uring` as one of the concrete techniques engineered to attack that cost through batching. Submissions

and completions move through shared ring buffers instead of one syscall per operation, and a single `io_uring_enter` call can carry a large batch of operations. The note goes one step further: with the `IORING_SETUP_SQPOLL` flag, a kernel-side thread polls the submission ring on its own, so in steady state a program's I/O submissions require no syscalls at all. It supports this with a cited Red Hat engineering article on using `io_uring` for network I/O, and separately cites a secondary aggregator's summary of a benchmark reporting per-submission cost dropping from roughly 300 cycles unbatched to roughly 40 cycles at a batch of 16 — a figure the note flags explicitly with a `[reported]` tag.

The note does not claim this eliminates the syscall boundary generally — it presents batching as one of three ways its own thesis ("a syscall is a fundamentally different mechanism than a function call") has real limits, alongside the vDSO and the microarchitectural-cost discussion, and it states directly that "where batching applies, per-call cost stops being the right unit of analysis," implying the technique does not apply everywhere. It is explicit about the weakness of its own evidence for the cycle-count figures: the aggregator source is rated low credibility as "a secondary summary of a primary benchmark study rather than the study itself," with the underlying figures unread at source, and the note instructs the claim be treated as reported rather than established. It also declines to rank `io_uring` against the other cost-attacking techniques it names — vDSO, eBPF, kernel-bypass stacks — describing them as complements rather than competitors, and does not assert which is best for a given workload.

Referenced sources

- [Anatomy of a system call, part 1](#) — David Drysdale / LWN.net (2014)
- [Anatomy of a system call, part 2](#) — David Drysdale / LWN.net (2014)
- [SYSCALL — Fast System Call \(x86 instruction reference\)](#) — Felix Cloutier, transcribed from the Intel SDM (n.d.)
- [x86-64 Syscall Entry](#) — Linux Kernel Internals — kernel-internals.org (n.d.)
- [The Linux Kernel Driver Interface \(stable-api-nonsense\)](#) — Greg Kroah-Hartman / Linux kernel documentation (2004–present)
- [Add kernel ↔ userspace ABI stability documentation](#) — Greg Kroah-Hartman / LWN.net (2006)
- [x86 Assembly / Interfacing with Linux](#) — Wikibooks contributors (n.d.)
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange (2018–2023)
- [Libraries and Headers — Windows drivers](#) — Microsoft Learn (last updated 2022-07-12)
- [winternl.h header — Win32 apps](#) — Microsoft Learn (n.d.)

- [NTDLL Functions](#) — Geoff Chappell (ongoing)
- [Windows X86-64 System Call Table \(XP through Windows 11 and Server\)](#) — Mateusz "j00ru" Jurczyk (ongoing)
- [Microsoft Windows library files — NTDLL.DLL](#) — Wikipedia contributors (n.d.)
- [Windows X86-64 System Call Table — discussion thread](#) — Hacker News contributors (2019)
- [strace — linux syscall tracer](#) — The strace project (ongoing)
- [ptrace and Syscall Interception — Linux Kernel Internals](#) — kernel-internals.org (n.d.)
- [Process Monitor — Sysinternals](#) — Microsoft Learn / Sysinternals (ongoing)
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (ongoing)
- [What Makes System Calls Expensive](#) — Abhinav Upadhyay (2025)
- [vDSO and Virtual System Calls — Linux Kernel Internals](#) — kernel-internals.org (n.d.)
- [Two frequently used system calls are ~77% slower on AWS EC2](#) — packagecloud engineering (n.d.)
- [Why you should use io_uring for network I/O](#) — Red Hat Developers (2023)
- [Speed up open\(\).read\(\) pattern by reducing the number of system calls — cpython#120754](#) — CPython issue tracker (2024)
- [win32file.CreateFile — PyWin32 documentation](#) — Mark Hammond / Tim Golden (PyWin32 docs, ongoing)
- [ltrace: When strace Is Not Enough](#) — Satesh / unixy.io (2026)
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026)
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026)
- [In Linux x86_64, are syscalls and int 0x80 related?](#) — Stack Overflow (2015–present)
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Palo Alto Networks / Unit 42 (2024)
- [Linux kernel system calls table](#) — Marcin Juskiewicz (regenerated from kernel source; table dated 2026-08-20 from 7.2.0 sources)
- [io_uring Interface Overview](#) — EmergentMind, summarizing Jasny et al. (updated 2026-01-17)

10.12 V12 — The gap between an OS call succeeding and its work being done

The note's organizing claim is that fast operating-system interfaces report success when the kernel has taken responsibility for a piece of work, not when that work is finished: "Success means 'accepted,' not 'done.'" It names this the single most consequential mismatch among the cases it documents, and treats other failures — a `close()` that appears to finalize a file, a thread count that appears to promise parallelism — as instances of the same pattern. The central evidence is that a Linux `write()` returning its full byte count only means the bytes reached the kernel page cache, with durability requiring an explicit `fsync()` — and, per the `fsync(2)` man page, even that does not make a newly created file findable after a crash without also syncing its containing directory. Windows carries the identical contract under different names: writes are buffered and not flushed automatically, so an application must call `FlushFileBuffers` or open the file unbuffered. The note extends the pattern to `close()`, where errors from an earlier `write()` surface only at close time and are silently lost if uninspected, and it anchors the claim in two incidents — Firefox 3's ext3 `fsync` stall, and Facebook Scuba's `mmap`-backed shared memory, which cut a multi-hour restart to minutes by keeping state resident.

The note does not claim this behavior is permanent or universal. It flags several underlying mechanisms as already withdrawn or in flux — Windows removed User-Mode Scheduling as of Windows 11, CPython's Global Interpreter Lock is being unwound starting with 3.12's per-interpreter GILs and 3.13's free-threaded build, and `io_uring` changed what a "syscall count" means — and states every specific claim carries a version scope requiring re-verification rather than standing as timeless fact. It leaves one adjacent question explicitly unsettled: whether `mmap` is faster than `read()`, where its own cited sources disagree, offering benchmarking one's own access pattern as the resolution rather than a general answer. It also flags one of its own supporting figures — that a mode switch costs on the order of hundreds of cycles — as unsupported by any primary measurement it could locate, recommending readers verify it themselves rather than treating it as established.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024)
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024)
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021)
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023)
- [Durability: Linux File APIs](#) — Evan Jones (2020)

- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022)
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017)
- [fsync, fdatasync, and O_SYNC](#) — kernel-internals.org (2025)
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026)
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024)
- [About Processes and Threads](#) — Microsoft Learn (2023)
- [Fibers](#) — Microsoft Learn (2023)
- [User-Mode Scheduling](#) — Microsoft Learn (2022)
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026)
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022)
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012)
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020)
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025)
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024)
- [strace\(1\) — Linux manual page](#) — strace project / man7.org (2024)
- [DTrace on Windows](#) — Microsoft Learn (2023)
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021)
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026)
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002)
- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026)
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026)
- [How Python's GIL Works](#) — Codefinity (2025)

10.13 V13 — The trustworthiness gap between strace's call counts and its timings

This note's Case 5 concerns a profiler blind spot: a sampling CPU profiler only attributes time to the userspace stack it samples, so time genuinely spent blocked in the kernel gets pinned on whichever call happened to cross into it, not on the true cost, which

is the number of boundary crossings. The proposed corrective is `strace -c`, which tabulates syscalls, errors, and time per call — but the note turns the same scrutiny on this fix. `-c` reports system time, CPU time spent inside the kernel, which is independent of wall-clock time; a syscall that blocks for 200ms shows almost no time under that flag. The flag that answers a latency question is `-w`, which sums wall-clock duration per call, and the note recommends running both on the same program to watch the two tables disagree. It then supplies the reason the disagreement exists at all: `strace` does not observe passively. It stops the traced process at every syscall to inspect it, and that stop is itself an intervention in the timing it is trying to report. This is the point the quoted span closes on — because the tracer pauses execution at each boundary crossing, the number and ordering of those crossings remain reliable, but the absolute durations it reports do not, since the act of measuring has already altered them.

The note does not claim `strace` is useless or that its timing output should be discarded outright — only that its two categories of output carry different evidentiary weight, and that treating a reported duration as ground truth repeats the diagnostic error the tool was invoked to correct in the first place. It does not attempt to quantify how much the tracing overhead distorts a given measurement, and it names no threshold past which a reported time becomes unusable; both are left as open, unresolved caveats rather than measured claims. The note frames this as one instance of a broader limitation it states explicitly elsewhere: observation is never free or neutral, since ETW sessions carry their own buffer and CPU cost and a debugger changes scheduling the same way `strace` does, so no tool's absolute timings should be trusted as much as its counts and orderings. It treats this as a structural property of instrumentation itself, not a defect a better tool would remove.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024)
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024)
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021)
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023)
- [Durability: Linux File APIs](#) — Evan Jones (2020)
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022)
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017)
- [fsync, fdatasync, and O_SYNC](#) — kernel-internals.org (2025)
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026)
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024)

- [About Processes and Threads](#) — Microsoft Learn (2023)
- [Fibers](#) — Microsoft Learn (2023)
- [User-Mode Scheduling](#) — Microsoft Learn (2022)
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026)
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022)
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012)
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020)
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025)
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024)
- [strace\(1\)](#) — Linux manual page — strace project / man7.org (2024)
- [DTrace on Windows](#) — Microsoft Learn (2023)
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021)
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026)
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002)
- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026)
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026)
- [How Python's GIL Works](#) — Codefinity (2025)

10.14 V14 — The Gap Between a Successful Write and a Durable One

This note argues that a successful `write()` call reports that the kernel has *accepted* the data, not that the data is *durable*, and that the two are routinely conflated by programmers reasoning from the language runtime's model rather than the syscall boundary. On Linux, `write(2)` copies the buffer into the kernel page cache and marks the pages dirty; writeback to the device happens later and asynchronously, and only `fsync(2)` forces that transfer, a guarantee documented in the kernel's own manual page. Windows is structurally identical: the OS buffers writes and schedules disk writes on its own timetable, and an application must call `FlushFileBuffers` (or open the file unbuffered) to force the issue. The evidence for the central claim — the quoted span — is a proposed classroom demonstration: killing the process after a `write()` only destroys whatever the *runtime* itself was still holding (e.g. a Python `io.BufferedWriter`'s unflushed contents), because the kernel's copy survives the process and is written back

regardless. Only a power cut, which removes the kernel's chance to complete write-back, actually tests whether the data was durable. The note treats this as the resolution to a common but wrong test methodology: students who validate their understanding with `kill -9` learn the wrong lesson, because that test cannot distinguish "flushed to the kernel" from "flushed to the device."

The note is explicit about what this claim does not cover. It does not claim that calling `fsync()` is sufficient on its own: a newly created file also needs an `fsync()` on the containing directory's descriptor, or the file may not be findable after a crash, and this second gap is presented as a separate, easily missed failure mode. It also does not claim `fsync()` guarantees durability unconditionally — the guarantee is a contract with the device, and the note flags as unsettled, or at least hardware-dependent, whether a given disk actually honors the flush request it receives, citing both older Linux filesystems that cannot flush disk caches and Windows-era drivers that ignored the FUA flag. The note treats this hardware honesty question as outside its own resolution: it names the risk rather than asserting how any particular device behaves.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024)
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024)
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021)
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023)
- [Durability: Linux File APIs](#) — Evan Jones (2020)
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022)
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017)
- [fsync, fdatasync, and O_SYNC](#) — kernel-internals.org (2025)
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026)
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024)
- [About Processes and Threads](#) — Microsoft Learn (2023)
- [Fibers](#) — Microsoft Learn (2023)
- [User-Mode Scheduling](#) — Microsoft Learn (2022)
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026)
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022)
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012)
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020)

- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025)
- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024)
- [strace\(1\)](#) — Linux manual page — strace project / man7.org (2024)
- [DTrace on Windows](#) — Microsoft Learn (2023)
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021)
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026)
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002)
- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026)
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026)
- [How Python's GIL Works](#) — Codefinity (2025)

10.15 V15 — Leaky Operating-System Abstractions

This note argues that abstractions built on top of an operating system leak in a specific, structural way: the call that crosses into the kernel is not an ordinary function call but a privilege transition, so its return can be interrupted, delayed, or reordered relative to anything the caller assumed was sequential. "The boundary is a privilege transition, not a jump" is grounded in `EINTR` on Linux, where the kernel deliberately surfaces an interrupting signal to userspace rather than hiding it — behaviour that only looks like the OS misbehaving if the boundary was believed to be invisible. The note generalizes this into two further properties: that a fast-returning OS call usually reports the kernel having *accepted* work rather than *finished* it (`write()` returning once bytes reach the page cache, not the disk), and that language runtimes stack a second layer of buffering on top of the kernel's own. From these three properties it builds five worked cases — an unsynced write that is not durable, a `close()` that neither flushes nor safely reports an earlier write's error, a "thread" that maps to different underlying objects depending on runtime and platform, an `mmap()` that trades syscalls for page faults, and a profiler that misattributes cost across the syscall boundary — each pinned to a named tool (`strace`, `blktrace`, Process Monitor, ETW) on both Linux and Windows, on the premise that which layer is lying cannot be deduced and must be observed.

The note treats none of this as permanent: it flags that the underlying mechanisms move — Windows withdrew User-Mode Scheduling entirely by Windows 11, `io_uring` changed what a "syscall count" even measures, CPython's GIL is being incrementally

unwound — and asks that every claim carry a version scope rather than be read as timeless. It also names an unresolved disagreement inside its own source set: whether `mmap` is faster than `read()` for sequential access. It concludes the question is malformed rather than settled, since the answer depends on access pattern — repeated random access to resident data versus a single sequential pass — and both benchmarked positions in its bibliography are genuine measurements of different things. One adjacent numeric claim, that a mode switch costs on the order of hundreds of cycles, is explicitly marked in the note as unsupported by any primary measurement and left for the reader to verify independently. The note also concedes that some leaks sit below what the operating system can abstract at all — device write caches that ignore flush requests, NUMA topology, per-CCX cache slicing — so "learn one layer down" has no fixed floor, and it does not claim its five cases are exhaustive.

Referenced sources

- [close\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024)
- [fsync\(2\) — Linux manual page](#) — Michael Kerrisk / man7.org (2024)
- [Flushing System-Buffered I/O Data to Disk](#) — Microsoft Learn (2021)
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn (2023)
- [Durability: Linux File APIs](#) — Evan Jones (2020)
- [The Secret Life of fsync](#) — Andrey Pechkurov / puzpuzpuz.dev (2022)
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft DevBlogs (2017)
- [fsync, fdatasync, and O_SYNC](#) — kernel-internals.org (2025)
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation (2026)
- [Bypassing the GIL for Parallel Processing in Python](#) — Real Python (2024)
- [About Processes and Threads](#) — Microsoft Learn (2023)
- [Fibers](#) — Microsoft Learn (2023)
- [User-Mode Scheduling](#) — Microsoft Learn (2022)
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#) — The Practical Developer (2026)
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn (2022)
- [Which is fastest: read, fread, ifstream or mmap?](#) — Daniel Lemire (2012)
- [Latency implications of virtual memory](#) — Erik Rigtorp (2020)
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#) — linux-vox.com (2025)

- [File Access: Memory-Mapped vs. I/O System Call Performance](#) — Baeldung on Linux (2024)
- [strace\(1\)](#) — Linux manual page — strace project / man7.org (2024)
- [DTrace on Windows](#) — Microsoft Learn (2023)
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#) — Yarden Shafir / Winsider Seminars & Solutions (2021)
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#) — systeminternals.dev (2026)
- [The Law of Leaky Abstractions](#) — Joel Spolsky (2002)
- [There Is a Mountain Lion in the Datacenter](#) — HFT University (2026)
- [Sysinternals Suite 2026.19.08](#) — Neowin (2026)
- [How Python's GIL Works](#) — Codefinity (2025)

10.16 V16 — Observation as the Method for Diagnosing Leaks in Operating-System Abstractions

The note treats higher-level constructs over the operating system — a file handle, a thread, a memory-mapped region — as contracts with an unwritten scope clause: each states what it does but not the conditions under which that holds. Three properties explain why OS-level abstractions leak in ways application-level ones do not: crossing into the kernel is a privilege transition, not a plain call, so control flow can be interrupted or reordered; a fast-returning call usually means the kernel accepted the work, not that the work happened; and language runtimes add their own buffering on top of the kernel's, doubling the places a program's model of the machine can be wrong. Because several layers can each be silently wrong, and nothing in the source distinguishes which one is responsible for a given surprise, the note argues — in the quoted claim — that the only reliable method is to make boundary crossings observable and count them, rather than deduce them. It grounds this in five belief/reality/observation/cost cases: a write that is not durable without an explicit sync; a `close()` that can discard an earlier write's error; a "thread" that names different runtime objects depending on platform; an `mmap` that loses to `read()` on sequential access; and a profiler that blames the wrong function because the true cost is boundary crossings, not bytes moved. Each case pairs a named tool — `strace`, `blktrace`, Process Monitor, ETW — with an artifact that settles the question empirically.

The note is explicit about its limits. It calls itself a synthesis layer for a specific course, not a re-derivation of the underlying mechanisms, and flags every mechanism as version-scoped: Windows' User-Mode Scheduling was withdrawn, `io_uring` changed

what a syscall count means, and CPython's GIL is being incrementally unwound, so its platform comparisons are not offered as permanent fact. It also warns that observation is not neutral — `strace` and ETW both perturb what they measure — so a tool's counts are trusted more than its absolute timings. Some leaks, it notes, sit below the operating system entirely (device write caches that ignore flush requests, NUMA topology, per-CCX cache slicing) and have no OS-level fix. One numeric claim in the profiler case — the cycle cost of a mode switch — is marked in the text as unverified and left as an open gap.

Referenced sources

- [close\(2\) — Linux manual page](#)
- [fsync\(2\) — Linux manual page](#)
- [Flushing System-Buffered I/O Data to Disk](#)
- [FlushFileBuffers function \(fileapi.h\)](#)
- [Durability: Linux File APIs](#)
- [The Secret Life of fsync](#)
- [Revised notes on the reliability of FlushFileBuffers](#)
- [fsync, fdatsync, and O_SYNC](#)
- [os — Miscellaneous operating system interfaces](#)
- [Bypassing the GIL for Parallel Processing in Python](#)
- [About Processes and Threads](#)
- [Fibers](#)
- [User-Mode Scheduling](#)
- [The Libuv Thread Pool Trap: Why Node.js Async APIs Stall Under Load](#)
- [MapViewOfFile function \(memoryapi.h\)](#)
- [Which is fastest: read, fread, ifstream or mmap?](#)
- [Latency implications of virtual memory](#)
- [Why is mmap\(\) Faster Than Sequential IO? Debunking Common Myths](#)
- [File Access: Memory-Mapped vs. I/O System Call Performance](#)
- [strace\(1\) — Linux manual page](#)
- [DTrace on Windows](#)
- [IoRing vs. io_uring: a comparison of Windows and Linux implementations](#)
- [Linux I/O Models — Blocking, epoll, io_uring, AIO, Signal-Driven](#)
- [The Law of Leaky Abstractions](#)
- [There Is a Mountain Lion in the Datacenter](#)
- [Sysinternals Suite 2026.19.08](#)

- [How Python's GIL Works](#)

10.17 V17 — Tracing Mechanisms for Observing a Running Process

The note's central claim is that mechanisms for observing a running program are not interchangeable flavours of the same idea: each attaches at a different point between application code and the kernel, and the attachment point mechanically determines both what the mechanism cannot see and how much it costs. Cost, the note argues, scales with the *rate* of the events being observed rather than with program size, and the sharpest illustration it offers is `ptrace` / `strace` on Linux. Brendan Gregg's worst case — `dd if=/dev/zero of=/dev/null bs=1 count=500k`, a program doing nothing but syscalls — runs 442× slower under `strace`, and does so even when tracing a syscall the program never makes, because the stop is paid regardless. The note treats this as the clearest demonstration that `strace`'s overhead is a function of syscall volume, not of what is being watched: the tracer pays a fixed per-stop cost twice per syscall (once on entry, once on exit), so a syscall-saturated program is the pathological case. It backs the figure directly to Gregg's own published, reproducible benchmark, and separately notes that other measurements of the identical `dd` benchmark on different hardware and mitigation states range from roughly 37× to 136×, and that other sources quote `ptrace` overhead as low as "2–10×" or "15×" for different workloads — all consistent once the workload generating each number is known.

The note is explicit about what this does not establish. It does not claim 442× (or any other figure) as *the* overhead of `strace` in general — it states outright that "a single overhead number for a rate-proportional mechanism is not a fact about the tool — it is a fact about the workload, and quoting one without the workload is the error." It also does not claim this cost is merely a slowdown in wall-clock time: elsewhere it argues, citing separate peer-reviewed work, that tracing overhead is non-uniform and can distort concurrent *behaviour*, not just duration — a claim it keeps distinct from the syscall-rate cost illustrated by the `dd` case. Where the note is speculative rather than sourced, it flags the passage explicitly as `[inference]`; the 442× figure and its workload are not among those flagged, but the note leaves open, and does not attempt to resolve, exactly how `ptrace`'s fixed per-stop cost decomposes on any given piece of hardware.

Referenced sources

- [ptrace and Syscall Interception](#) — Linux Kernel Internals
- [strace Wow Much Syscall](#) — Brendan Gregg (2014)
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026)
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023)

- [NtTrace — Native API tracing for Windows](#) — Roger Orr
- [rogerorr/NtTrace — "How it works"](#) — Roger Orr
- [vdso\(7\) — Linux manual page](#) — Linux man-pages project
- [StraceNT — System Call Tracer for Windows NT](#) — Pankaj Garg
- [Debugging eBPF verifier: Best Practices for High-Performance Kernel Programming](#) — Raven
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — George V. Neville-Neil et al., USENIX ;login: (2017)
- [Tracing and its observer effect on concurrency](#) — Indigo Orton, Alan Mycroft — ACM (2021)
- [We Have It Easy, But Do We Have It Right? / observer effect and measurement bias](#) — Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, Peter Sweeney (2008)
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026)
- [Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers, FOSDEM (2020)
- [System Call Tracing With eBPF: Move Beyond Ptrace](#) — SysTutorials
- [Ptrace & You](#) — Omar S. Navarro Leija
- [vDSO and Virtual System Calls](#) — Linux Kernel Internals
- [System-wide syscalls tracing and monitoring in Linux using bpftrace](#) — Narasimha Prasanna HN
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygódzki
- [ipankaj/stracent — StraceNT](#) — Pankaj Garg
- [Unveiling Dynamic Binary Instrumentation Techniques](#) — arXiv preprint (2025)
- [How to do a million watchpoints: Efficient Debugging using Dynamic Instrumentation](#) — Qin Zhao et al., CC (2008)
- [What is eBPF? An Introduction and Deep Dive into the eBPF Technology](#) — eBPF Foundation
- [Inside the eBPF Verifier: Ensuring Program Safety and Complexity Bounds](#) — How Tech — Systems Programming
- [Loops and Iterators in eBPF](#) — Teodor J. Podobnik
- [CVE-2021-33624 — Red Hat Customer Portal](#) — Red Hat
- [Eliminating eBPF Tracing Overhead on Untraced Processes](#) — Milo Craun et al., eBPF '24
- [DTrace Capabilities](#) — Oracle — Solaris Dynamic Tracing Guide

- [Dynamic Instrumentation of Production Systems](#) — Bryan M. Cantrill, Michael W. Shapiro, Adam H. Leventhal — USENIX ATC (2004)
- [DTrace on Windows](#) — Microsoft Learn
- [Announcing DTrace for Windows Insider](#) — Packt (2019)
- [ETW — Event Tracing for Windows](#) — Mjolnir Security
- [NT Kernel Logger Trace Session](#) — Microsoft Learn
- [Windows Performance Recorder](#) — Microsoft Learn
- [Doubling Down: Detecting In-Memory Threats with Kernel ETW Call Stacks](#) — Elastic Security Labs
- [prash-wghats/DTrace-win32](#) — prash-wghats
- [CSE 422S Studio 5: Tracing the Linux Kernel](#) — Christopher Gill, Washington University in St. Louis
- [microsoft/DTrace-on-Windows](#) — Microsoft
- [bpf-helpers\(7\)](#) — Linux manual page — Linux man-pages project
- [bpf: Change value of MAX_TAIL_CALL_CNT from 32 to 33](#) — Linux kernel mailing list / stable backport series (2022)
- [Windows 11, version 26H1](#) — Microsoft Support (2026)
- [Updating Syscall Tables](#) — SysWhispers4 — SysWhispers4 project documentation
- [strace 7.1 release notes](#) — strace project (2026)

10.18 V18 — The four-level memory allocation hierarchy

The note establishes that a single physical page passes through four distinct allocators on its way from a Python assignment to RAM — a pymalloc pool, a glibc arena or Windows heap, a kernel VMA or Windows region, and a physical page frame — and that all four obey one asymmetry: each level buys a coarse unit from the level below and sells a finer one to its caller, so every layer's bookkeeping necessarily disagrees with the layers around it. It backs this with measurements taken on one host (CPython 3.13.15, glibc 2.39, Linux 6.8 aarch64): a running interpreter's `sys._debugmallocstats()` output showing arenas, pools and blocks whose totals differ from each other by design; `/proc/<pid>/maps` and `pmap -x` traces showing glibc's 128 KiB mmap threshold sliding at runtime after a freed allocation is immediately reused at the same size; and `/proc/self/pagemap` readings showing a read-only page arriving present-but-not-exclusive from the kernel's shared zero page, with only a subsequent write allocating a private frame. Each measurement is offered as one worked instance of the same purchasing asymmetry, read at a different level.

The note is explicit about what it does not cover and what it does not claim. It treats only the allocation path; the deallocation path — why freed memory does not return to the operating system — is deliberately left to a companion note. It does not claim Windows and Linux allocators line up one-to-one: Windows CPython is three levels deep rather than four because `pymalloc` calls `VirtualAlloc` directly, and the note calls out any four-level table that implies otherwise as misleading. It flags its own numeric constants as machine- and build-specific rather than general facts — `pymalloc`'s arena and pool sizes have changed across CPython versions, and `glibc`'s dynamic `mmap` threshold was removed and then reverted within the same year for causing benchmark regressions, so the note treats that default as a contested, unsettled implementation choice rather than a settled fact. It also names a hard observational limit: the physical page frame number at level four is not readable by an unprivileged process on modern Linux, so the sharing behavior there is argued through proxies (`PSS`, `Shared_Dirty`) rather than direct frame identity, and one supporting trace is explicitly marked as quoted from a published source rather than captured on this host.

Referenced sources

- [malloc\(3\) — Linux manual page](#) — Michael Kerrisk / The Linux man-pages project
- [Malloc Tunable Parameters](#) — The GNU C Library manual (2.44) — Free Software Foundation
- [Memory Management](#) — Python/C API Reference Manual — Python Software Foundation
- [CPython — Include/internal/pycore_obmalloc.h](#) — Python core developers
- [\[patch\] Make the mmap/brk threshold in malloc dynamic to improve performance](#) — Arjan van de Ven and Val Henson, `libc-alpha` (2006)
- [\[glibc\] Revert "malloc: Remove dynamic mmap/trim threshold \[BZ #30769\]"](#) — Peter Bergner, `glibc-cvs` (2026-07-17)
- [VMMMap v3.4 — Sysinternals](#) — Mark Russinovich / Microsoft Learn (2023)
- [Windows Process Memory Usage Demystified](#) — DZone
- [proc_pid_maps\(5\) — Linux manual page](#) — Linux man-pages project
- [brk vs mmap — Linux Kernel Internals](#) — kernel-internals.org
- [The /proc Filesystem — The Linux Kernel documentation](#) — The Linux kernel documentation project
- [VMMMap Basics: How to Read a Windows Process's Memory Layout](#) — Pavel Yosifovich / TrainSec
- [pagemap, from the userspace perspective — The Linux Kernel documentation](#) — The Linux kernel documentation project
- [VirtualAlloc function \(memoryapi.h\)](#) — Microsoft Learn

- [Heap Functions — Win32 apps — Microsoft Learn](#)
- [How to analyze a Linux process' memory map with pmap — Red Hat \(2022\)](#)
- [SYSTEM_INFO structure \(sysinfoapi.h\) — Microsoft Learn](#)
- [Malloc Internals and You — Red Hat Developer \(2017\)](#)
- [Python allocators — memray documentation — Bloomberg](#)
- [Increased memory usage with mimalloc \(python/cpython#135153\) — CPython core developers and contributors](#)
- [Legacy NT Heap — windows-heap-exploitation — pwnfuzz \(2025\)](#)
- [mallinfo\(3\) / mallinfo2\(3\) — Linux manual page — Michael Kerrisk / The Linux man-pages project](#)
- [Linux Memory Management — COMS W4118, Columbia University — Columbia University course notes \(2023\)](#)
- [Python Memory Optimization: How CPython's Memory Pool Works — DEV Community](#)
- [Copy-on-Write — Linux Kernel Internals — kernel-internals.org](#)
- [Leaking memory with VirtualAlloc — Software Verify \(2022\)](#)
- [Your memory leak might be standard Python behavior — Ramon Perez, DEV Community \(2026\)](#)

10.19 V19 — Virtual memory and the page cache

"Virtual Memory and the Page Cache" is a lecture-hub note arguing that virtual memory and the page cache are one mechanism — physical memory as a cache, not a container — and it uses that framework to explain why "cold" and "warm" benchmark numbers routinely mislead. Its central claim on this point, closing §3 ("The page cache — why a second read is fast, and when that misleads a benchmark"), is that no available technique clears every cache layer a number could include, so the correct discipline is to name which layer was measured rather than assert a true cold state: "state which cache layer your number includes, because you cannot eliminate all of them." The note backs this with three pieces of evidence: a peer-reviewed database-benchmarking paper (Raasveldt, Holanda, Gubner & Mühleisen, DBTest 2018) that treats cold and hot runs as needing separate reporting and states that dropping caches meaningfully may be impossible inside a cloud VM, because the hypervisor host caches too; a practitioner guide noting that Linux's `drop_caches` does not touch a storage device's own DRAM cache or an HBA/controller cache, so most published "cold-cache" results are wrong; and a blog post by a long-standing QEMU/Linux storage maintainer arguing that

`drop_caches` also distorts the very access pattern being measured, since page-cache prefetch and writeback still intervene, and that `O_DIRECT` is the correct instrument for disk benchmarking instead.

The note does not claim a procedure exists for eliminating every cache layer, nor that its cited sequence — stop the server, drop OS caches, restart, run one query — produces a truly cold measurement; it presents that sequence as the DBTest paper's prescription while immediately flagging it as insufficient on its own. It explicitly leaves open how to fully corroborate a "cold" measurement in a cloud environment, stating only that dropping caches there "may be impossible to do meaningfully." The note does not evaluate whether device- or hypervisor-level caching could ever be disabled outright — it treats that as outside the mechanism it is describing and confines its guidance to disclosure (name the layer) rather than elimination. Other uncertainties flagged elsewhere in the note, such as the unsettled status of Linux's multi-generational LRU reclaim policy, are separate concerns and are not part of this claim.

Referenced sources

- [Concepts overview — The Linux Kernel documentation](#) — The Linux Kernel Organization
- [Windows Kernel Internals: Cache Manager](#) — Microsoft Corporation, University of Tokyo lecture series
- [Page fault — Grokipedia](#) — Grokipedia
- [Working Set - Win32 apps](#) — Microsoft Learn
- [Copy-on-Write and the Page-Fault Lifecycle, Step by Step](#) — PandaStack
- [How to Check Memory Usage per Process in Linux](#) — GoLinuxCloud
- [perf-trace\(1\) — Linux manual page](#) — man7.org / Linux man-pages project
- [The Basics of Page Faults](#) — Microsoft Community Hub, AskPerf team blog
- [Windows Performance Monitor basics](#) — SQLShack
- [Memory Management Tracing Events - Win32 apps](#) — Microsoft Learn
- [Why does Linux, by default, overcommit memory?](#) — Unix & Linux Stack Exchange
- [Virtual vs Physical vs Resident Memory](#) — Linux Kernel Internals
- [The /proc Filesystem](#) — The Linux Kernel documentation — The Linux Kernel Organization
- [RAMMap - Sysinternals](#) — Microsoft Learn / Sysinternals
- [Accurately measuring memory usage](#) — Phusion
- [Readahead - Linux Kernel Internals](#) — Linux Kernel Internals
- [Page Cache Writeback - Linux Kernel Internals](#) — Linux Kernel Internals
- [Windows Memory Management in Windows 10 and Windows 11](#) — CyberRaiden

- [Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing](#) — Raasveldt, Holanda, Gubner & Mühleisen, DBTest (2018)
- [Understanding and tuning the page cache on Debian](#) — Big Iron
- [Excessive Page Faults Generated By Windows Applications](#) — Broadcom Knowledge Base
- [Common disk benchmarking mistakes](#) — Stefan Hajnoczi (2017)
- [Memory-Mapped I/O - Linux Kernel Internals](#) — Linux Kernel Internals
- [File Mapping - Win32 apps](#) — Microsoft Learn
- [Creating Named Shared Memory](#) — MicrosoftDocs/win32 repository
- [Memory-Mapped Files \(mmap\): A Practical Guide to Faster I/O and Shared Memory](#) — martinuke0 (2025)
- [mmap SIGBUS error and initializing the file](#) — Stack Overflow
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn
- [mmap — Memory-mapped file support](#) — Python Software Foundation
- [Are You Sure You Want to Use MMAP in Your Database Management System?](#) — Crotty, Leis & Pavlo, CIDR (2022)
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — Neville-Neil et al., USENIX ;login: (2017)
- [Teaching Operating Systems Concepts with Execution Visualization](#) — Giraldeau, Dagenais & Boucheneb, ASEE
- [Mastering strace, perf, and eBPF — 2026 Cheat Sheet](#) — Tech Bytes
- [CS 110: Principles of Computer Systems — assign3](#) — Stanford University (2021)
- [Page Cache - Linux Kernel Internals](#) — Linux Kernel Internals
- [On pages and folios](#) — Jonathan Corbet, LWN.net
- [Reconsidering the multi-generational LRU](#) — Jonathan Corbet, LWN.net

10.20 V20 — File I/O and durability

This note argues that durability is not a property of a single write but of a sequence of up to four independent flushes, each crossing one boundary between an application and the physical medium, with each boundary able to fail on its own. The quoted claim covers the fourth and last of these boundaries: the storage device's own cache. The note states, citing a source that traces the kernel's flush path down to the device-level FLUSH command, that most HDDs and many SSDs acknowledge a write as soon as it lands in an onboard DRAM cache, before the bytes reach NAND or platter, and that a power cut during that window loses the data even though the operating system was already told

the write had succeeded. The kernel closes this specific gap by issuing a cache-flush command (SYNCHRONIZE CACHE on SCSI/SATA, FLUSH on NVMe) or by marking an individual write FUA to bypass the device cache entirely. The note adds two further points about the reliability of that closure, both attributed to SQLite's own design documentation: a drive's controller can misreport a flush as complete while the data is still sitting in its volatile cache, and most consumer SSDs still lack the capacitors that would protect a cache against power loss.

The note does not claim this failure mode is universal: it says "most" HDDs and "many" SSDs, not all, and elsewhere treats enterprise hardware with power-loss protection as a separate case. It does not claim that issuing the correct flush command is sufficient on its own, since the same section states that a device's honesty about having completed a flush cannot be verified from software — durability at this layer ultimately rests on trusting the hardware. The note also flags, outside this specific claim, that its whole four-boundary picture is a teaching simplification: network filesystems, memory-mapped I/O, and copy-on-write filesystems each reshape or add to the layers described. And it leaves open, marked explicitly as an unresolved gap rather than answered, which Windows storage-tracing tool can observe this device-level flush behavior as cleanly as Linux tooling can.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026)
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project
- [Durability: Linux File APIs](#) — Evan Jones (2020)
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017)
- [io](#) — Core tools for working with streams — Python Software Foundation
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022)
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode
- [Page Cache Writeback](#) — Linux Kernel Internals
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn
- [File Caching](#) — Microsoft Learn

- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al.
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026)
- [Life of a Write](#) — Linux Kernel Internals
- [The errseq_t datatype](#) — [The Linux Kernel documentation](#) — Linux kernel documentation
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018)
- [Fsync Errors](#) — PostgreSQL wiki
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020)
- [CreateFileW function](#) — [FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn
- [File Buffering](#) — Microsoft Learn
- [Windows Write Caching](#) — [Part 2: An Overview for Application Developers](#) — Dilip Naik (2012)
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019)
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019)
- [os](#) — [Miscellaneous operating system interfaces](#) — Python Software Foundation
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow
- [DTrace on Windows](#) — Microsoft Learn
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn
- [Write-Ahead Logging \(SQLite\)](#) — SQLite

10.21 V21 — NtTrace, a debug-interface Windows API tracer

This note documents NtTrace, a Windows tool that reports the calls a process makes into ntdll.dll, the user-mode boundary between application code and the NT kernel. Its central design decision, stated by the tool's own author, is exactly the quoted claim: the tool traces via the Windows debug interface — the same WaitForDebugEvent/ContinueDebugEvent machinery a symbolic debugger uses, writing software breakpoints around the native call stubs inside ntdll — so the tool needs no driver and no administrator rights. The note treats this as the documentation's real subject rather than an implementation detail: a Windows tracer must decide where it can stand without elevated privilege, and NtTrace answers by attaching one layer above the kernel, at the DLL boundary, rather than at the syscall interface Linux tracers use. The claim is corrobor-

ated beyond the author's own account: two independent third-party surveys of Windows tracing tools confirm the same mechanism, and a kernel-mode alternative, Mandiant's STrace, is cited as the contrasting cost — it must be installed as a driver and requires disabling driver signature enforcement at every boot, the overhead NtTrace's user-mode design exists to avoid. The note rates this evidence as strong: first-party, mechanically specific, and independently confirmed by unaffiliated reviewers.

The note is equally explicit about what this design does not reach. Because NtTrace only observes code that actually executes through ntdll, it cannot see kernel calls issued by device drivers on an application's behalf, nor calls that invoke the underlying `int 2e/syscall/sysenter` instructions directly — a gap the note calls non-hypothetical, since bypassing ntdll is the standard technique for evading user-mode monitoring. The debug-interface attachment also serializes calls made simultaneously by different threads, so the ordering visible in a trace can be an artifact of the tracer rather than of the traced program; the note singles this out as a case where tracing can manufacture an appearance of determinism that was never really there. One question the note leaves explicitly unsettled is how strongly ntdll should be described as the interface between application and kernel. Its critical assessment weighs two readings — that NtTrace traces a mere convention, versus that the convention is the interface Windows actually maintains — and concludes the tool's core design choice holds up either way, while cautioning that no claim of complete visibility into a process's kernel-facing behavior survives: a debugger-based, ntdll-only tracer cannot support it.

Referenced sources

- <https://rogerorr.github.io/NtTrace/> — NtTrace — Native API tracing for Windows (Roger Orr)
- <https://github.com/rogerorr/NtTrace> — rogerorr/NtTrace — repository README (Roger Orr)
- <https://github.com/rogerorr/NtTrace/wiki> — Home · rogerorr/NtTrace Wiki (Roger Orr)
- <https://jarekprzygodzki.dev/post/notes-and-thoughts-on-strace-for-windows/> — Notes and Thoughts on Strace for Windows (Jarek Przygodzki)
- <https://downwithup.github.io/blog/post/2026/03/29/post10.html> — AWStrace: Another Windows Strace (downwithup)
- https://blog.pankajgarg.com/system_call_tracer_for_windows/ — StraceNT - System Call Tracer for Windows NT (Pankaj Garg)
- https://drmemory.org/page_drstrace.html — System Call Tracer for Windows (drstrace) (Dr. Memory / DynamoRIO project)
- <https://github.com/mxmssh/drltrace> — mxmssh/drltrace — README (Maksim Shudrak)

- <http://roberto.greyhats.it/2014/03/qtrace-part1.html> — Introducing QTrace, a "zero knowledge" system call tracer (Roberto Paleari)
- <https://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html> — strace Wow Much Syscall (Brendan Gregg)
- <https://pchaigno.github.io/strace/2019/10/02/introducing-strace-seccomp-bpf.html> — Introducing strace --seccomp-bpf (Paul Chaignon)
- <https://unix.stackexchange.com/questions/473137/do-other-unix-like-kernels-have-stable-syscall-abis> — Do other Unix-like kernels have stable syscall ABIs? (Unix & Linux Stack Exchange)
- <https://opensource.com/article/22/12/linux-abi> — A 10-minute guide to the Linux ABI (Alison Chaiken, opensource.com)
- <https://www.paloaltonetworks.com/blog/security-operations/a-deep-dive-into-malicious-direct-syscall-detection/> — A Deep Dive Into Malicious Direct Syscall Detection (Or Chechik and Ofir Ozer, Palo Alto Networks)
- https://accu.org/journals/nonmembers/overload_article_members/ — Overload Content by Article — ACCU
- <https://linuxvox.com/blog/system-calls-in-windows-native-api/> — Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained (linux-vox.com)
- <https://github.com/mandiant/STrace> — mandiant/STrace — A DTrace on Windows Reimplementation (Mandiant)

10.22 V22 — NtTrace, a user-mode Windows native-API tracer

The note documents NtTrace, a tool that traces a Windows process's calls into `ntdll.dll` by planting breakpoints through the Windows debug interface, a design chosen specifically so the tool needs no driver and no administrator rights. Its evidence for the layer this exposes is a single worked capture, published on the tool's own project page and reproduced in the note because the vault held no concrete NtTrace output before, only prose describing the mechanism. In that capture, `NtQueryAttributesFile` is called with `ObjectAttributes="\??\C:\WINDOWS\system32\cmd.exe.Local"`. The note reads that string as the clearest available demonstration that Windows's native API sits one layer beneath the Win32 API an ordinary program calls: the `\??\` prefix names the object-manager namespace, a path form a Win32 program never constructs itself and never sees, because the Win32 layer translates ordinary paths into it before the native call is made. The note treats this as visible, direct evidence of a layer boundary that the rest of its argument otherwise has to establish through documentation quotes and third-party claims about undocumented, version-varying native calls.

The note is explicit about what this single capture does not establish. It is not paired with any equivalent trace of the same program under a different tracer, so it cannot itself support a side-by-side comparison; it only shows what a Windows trace looks like. It is also a 32-bit, Windows XP-era capture — visible in the `ntdll.dll` base address, the stack addresses, and the process base address — so a capture taken on a current system would show 64-bit addresses, and the note cautions against presenting it as representative of what a reader would see today. The note further flags that the capture shows `NtFsControlFile` listing the `UserApcContext` parameter twice, and states this is unresolved: it may be a quirk of NtTrace's configuration file or an error in how the source page presents the line, and the note says this should be re-checked against the live page before further use. Finally, the note records the capture's provenance as a fetch-and-convert retrieval rather than a terminal transcript, and asks that it be spot-checked against the source before being relied on further.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr
- [rogerorr/NtTrace — repository README](#) — Roger Orr
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki
- [AWStrace: Another Windows Strace](#) — downwithup
- [StraceNT – System Call Tracer for Windows NT](#) — Pankaj Garg
- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project
- [mxmssh/drltrace — README](#) — Maksim Shudrak
- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari
- [strace Wow Much Syscall](#) — Brendan Gregg
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks
- [Overload Content by Article — ACCU](#) — ACCU (Association of C and C++ Users)
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com
- [mandiant/STrace — A DTrace on Windows Reimplementation](#) — Mandiant

10.23 V23 — NtTrace, a user-mode Windows native API tracer

The note documents NtTrace, a user-mode Windows tracer that attaches through the Windows debug interface and places breakpoints around the native call stubs in `ntdll.dll`, the boundary between application code and the NT kernel. To make the tool concrete, the note reproduces the project's own published sample capture — a trace of the `cmd` process opening and reading a file — noting that the vault previously held no concrete NtTrace output at all. Assessing what that capture is and is not good for, the note states directly: "It is not a matched pair." The point is that a single trace of `cmd` is not itself evidence for a side-by-side Windows/Linux comparison; it only shows what a Windows-side trace looks like structurally — handles rather than file descriptors, NT object paths prefixed `\\??\\`, an `NTSTATUS` return code translated into a Win32 error message, and small, unbuffered console writes. The note is explicit that establishing this vocabulary is the sample's only function: a genuine comparison would require tracing the same program with both `strace` and NtTrace, produced together, which this single-sided sample does not attempt.

The note is equally explicit about the sample's limits. It is a 32-bit, XP-era capture (`ntdll.dll` at `77F40000`, a 32-bit process base address) and should not be presented as representative of current 64-bit output. One line in the capture repeats an argument name (`UserApcContext` appears twice in the `NtFsControlFile` call) that the note flags as unresolved — possibly a quirk of NtTrace's configuration file, possibly an error on the source page — and recommends re-checking against the live page before it reaches a slide. The capture was also retrieved through a fetch-and-convert step rather than copied from a live terminal, so the note recommends spot-checking it before publishing it anywhere. Beyond the sample itself, the note does not claim the traced mechanism is complete: NtTrace only sees calls that pass through `ntdll`, so it cannot see driver-originated kernel calls or direct syscall stubs that bypass `ntdll` entirely — the standard technique for evading user-mode monitoring. The note leaves unsettled how severely thread serialization distorts a given traced workload's apparent ordering, and whether the duplicated argument name in the sample is a documentation artifact or a live bug in the tool.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr
- [rogerorr/NtTrace — repository README](#) — Roger Orr
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki
- [AWStrace: Another Windows Strace](#) — downwithup
- [StraceNT – System Call Tracer for Windows NT](#) — Pankaj Garg

- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project
- [mxmssh/drltrace](#) — README — Maksim Shudrak
- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari
- [strace Wow Much Syscall](#) — Brendan Gregg
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks
- [Overload Content by Article](#) — ACCU — ACCU (Association of C and C++ Users)
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com
- [mandiant/STrace](#) — A DTrace on Windows Reimplementation — Mandiant

10.24 V24 — WiredTiger's fsync() retry-to-panic fix

The note documents "fsyncgate": Linux reports a writeback error to `fsync()` only once, because the failing page is marked clean and discarded at that point, so a second `fsync()` call finds no dirty pages left and returns success even though the original write was never made durable. The claim this appendix centres on is the note's account of MongoDB's WiredTiger storage engine, which fixed this in WT-4045 and, per the note, "quoted this very LWN article in the source comment as its justification," reproducing the line "Given the shared history of UNIX filesystems, and the difficulty of knowing what specific error will be returned under specific circumstances, we don't retry fsync-style calls and panic if a flush operation fails." The note treats this as more than an isolated fix: it is evidence that the LWN reporting itself became load-bearing engineering justification, cited verbatim inside a shipped commit rather than merely read and acted on independently. The commit replaced `WT_SYSCALL_RETRY / WT_RET_MSG` with `WT_SYSCALL` plus `WT_PANIC_RET` on the `fdatasync`, `fsync`, and macOS `F_FULLFSYNC` paths. The note situates this alongside PostgreSQL's own PANIC-on-failure commit and InnoDB's replacement of a sleep-and-retry loop with a fatal error, reading the three-way convergence — independent teams, independent architectures, same conclusion within months of each other in 2018 — as the strongest evidence in its record that "just retry" cannot be made to work.

The note is explicit about what this specific claim does not establish. It does not claim WiredTiger's citation of the article means the article was the sole or original source of the fix's reasoning — the `pgsql-hackers` thread and PostgreSQL's own commit predate and independently motivate the same conclusion, and WiredTiger's engineers could have reached PANIC without it. It does not claim citing a piece of journalism in a source comment is itself unusual practice worth generalising from; the note calls it "a rare case of a code comment citing its own journalism" without asserting broader significance. And it does not claim the WiredTiger fix, or the pattern of convergence more broadly, closes the underlying problem: elsewhere the note is clear that a 2020 peer-reviewed fault-injection study found no application, PostgreSQL included, handles `fsync` failure perfectly even after these fixes shipped, and that direct I/O — the fix all parties agreed was the real long-term answer — remains, as of PostgreSQL 18, a debug-only parameter documented as unfit for production.

Referenced sources

- [PostgreSQL's `fsync\(\)` surprise](#) — Jonathan Corbet / LWN.net (2018)
- [Fsyncgate: errors on `fsync` are unrecoverable](#) — Dan Luu (archive of the `pgsql-hackers` thread, 2018)
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1 — "PANIC on `fsync\(\)` failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018)
- [PostgreSQL documentation — `data\_sync\_retry`](#) — PostgreSQL Global Development Group
- [MySQL Bug #90296 — "Hard error should be report when `fsync\(\)` return EIO"](#) — Oracle / MySQL bug tracker (2018)
- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a — Bug#27805553](#) — Oracle / MySQL Server (2018)
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96 — WT-4045 "Don't retry `fsync` calls after EIO failure"](#) — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018)
- [Can Applications Recover from `fsync` Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020)
- [fs: introduce new writeback error reporting and convert existing API as a wrapper around it](#) — Jeff Layton, linux-fsdevel (2017)
- [PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs](#) — Jeff Layton, LKML (2018)
- [Linux Filesystems API summary — `filemap\_set\_wb\_err` / `filemap\_check\_wb\_err`](#) — The Linux Kernel documentation

- [PostgreSQL 18 and beyond: From AIO to Direct IO?](#) — Cybertec PostgreSQL International (2025)
- [debug_io_direct](#) — pgPedia — pgPedia, a PostgreSQL Encyclopedia
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts)
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications

10.25 V25 — The production-unsuitability of syscall-boundary tracing

(`ptrace` / `strace`)

The note treats "strace --seccomp-bpf still stops at each syscall of interest and should therefore not be used on production systems!" as the closing verdict on the `ptrace` / `strace` mechanism: even its most optimised variant cannot escape the stop-and-resume structure that makes syscall tracing expensive, so the disqualification is structural, not a tuning shortfall. The note backs this with the mechanism itself — two context switches per syscall, four under record-and-replay instrumentation — and with measurement at several points: Brendan Gregg's 442× worst case on a syscall-saturated benchmark, the strace maintainers' own FOSDEM 2020 figures of roughly 37× and 136× on the same benchmark on different hardware, about 3× on a realistic workload, and roughly 1.44× once `--seccomp-bpf` is enabled. The quote itself comes from the author of that seccomp-bpf optimisation, and Sysdig's independent write-up reaches the same production verdict from the vendor side. The claim sits inside the note's larger thesis that a tracer's attachment point fixes cost and blind spot together: for `strace`, the blind spot is anything that never crosses the syscall boundary, most visibly `io_uring`, which bypasses syscalls entirely and stays invisible no matter how the tracer is tuned.

The note treats this as a ratio-dependent claim, not a blanket one: it names event rate as the variable that can make the ratio stop mattering, citing DTrace guidance that below roughly a thousand traced events per second the overhead becomes unnoticeable — a lightly loaded process is not what the warning covers. It does not settle what "production-safe" means in general: it records, without resolving, a disagreement between Brendan Gregg (architectural non-disruption is enough) and Dan Luu (only continuous fleet-scale cheapness counts) over whether even the cheaper in-kernel mechanisms qualify. It also flags that its overhead figures are not commensurable across benchmarks — a whole-workload multiplier on a syscall-saturated case and a per-event cost elsewhere measure different things — and declines to collapse them into one number, treating only the ordering across mechanisms, not arithmetic between them, as safe to draw.

Referenced sources

- [strace Wow Much Syscall](#) — Brendan Gregg (2014)
- [Ptrace, Utrace, Uprobes: Lightweight, Dynamic Tracing of User Apps](#) — Keniston, Panchamukhi, Hiramatsu — Ottawa Linux Symposium (2007)
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019)
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers — FOSDEM (2020)
- [Sysdig vs DTrace vs Strace: A technical discussion](#) — Sysdig (n.d.)
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (n.d.)
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki (n.d.)
- [Transparent Dynamic Instrumentation](#) — Derek Bruening, Qin Zhao, Saman Amarasinghe — VEE (2012)
- [DynamoRIO Release Notes](#) — DynamoRIO project (n.d.)
- [Dynamic binary instrumentation \(DBI\) with DynamoRio](#) — Cisco Talos Intelligence (2025)
- [Tracing a Subset of Execution](#) — DynamoRIO project (n.d.)
- [DTrace for Linux 2016](#) — Brendan Gregg (2016)
- [Benchmarking Performance Overhead of DTrace on FreeBSD and eBPF on Linux](#) — Mateusz Piotrowski — AsiaBSDCon (2024)
- [ebpf_exporter eBPF overhead benchmark](#) — Cloudflare (n.d.)
- [Pitfalls of relying on eBPF for security monitoring \(and some solutions\)](#) — Artem Dinaburg, Trail of Bits (2023)
- [eBPF Hookpoints Gotchas \(FOSDEM 2026 slides\)](#) — FOSDEM (2026)
- [Instrumenting Your Code with ETW](#) — Microsoft (n.d.)
- [Event Tracing for Windows \(ETW\)](#) — Ryan Myers, Microsoft (archived blog, n.d.)
- [Performance Analysis with ETW: Event Tracing for Windows](#) — Atomic Object (n.d.)
- [Heisenbug](#) — Wikipedia (2026)
- [Engineering Record And Replay For Deployability](#) — O'Callahan, Jones, Froyd, Huey, Noll, Partush — USENIX ATC (2017)
- [rr: lightweight recording & deterministic debugging](#) — rr project (2024)
- [Sampling v. tracing](#) — Dan Luu (n.d.)
- [eBPF monitoring overhead: measure and reduce it](#) — groundcover (n.d.)
- [strace of io_uring events? \(strace-devel mailing list\)](#) — strace-devel (2020)
- [strace\(1\)](#) — Linux manual page — man7.org (n.d.)

- [DTrace pid Provider Overhead](#) — Brendan Gregg (2011)
- [Portable, Low-Overhead Observability for io_uring](#) — arXiv preprint (2026)
- [io_uring War Stories](#) — kernel-internals.org (n.d.)
- [Transparently Integrating Debugging and Dynamic Binary Instrumentation](#) — Rensselaer Polytechnic Institute (n.d.)
- [Event Tracing for Windows \(ETW\) — architecture and providers](#) — Salucci (n.d.)
- [Bypassing ETW For Fun and Profit](#) — White Knight Labs (2021)
- [Support io_uring — rr issue #2613](#) — Robert O'Callahan, rr project (2020)
- [Sampling vs deterministic profilers: which should I use?](#) — Python Developer Tooling Handbook (n.d.)

10.26 V26 — The System Call Boundary: Mechanism versus Contract

This note separates two things usually conflated under "the system call boundary": a mechanism boundary, where the CPU changes privilege level — a fixed hardware fact, identical across operating systems on the same architecture — and a contract boundary, where an operating system promises not to change its interface, a policy choice each vendor places differently (Linux at the numbered syscall ABI itself, Windows one layer higher at Win32). Built on that distinction, the note's central thesis is that a system call is not merely an expensive function call, and its sharpest illustration is the Linux vDSO: a kernel-provided page mapped into every process serves calls such as `clock_gettime`, `gettimeofday`, and `getcpu` with no privilege transition at all. Because there is no crossing for these calls, `strace` cannot see them, because there is nothing to intercept — the note offers this as a classroom exercise, since a student whose `strace` output does show `clock_gettime` has learned something: the vDSO fast path fell back to a real syscall, which happens when the clocksource is HPET rather than TSC, or on virtualized hosts where the guest lacks vDSO support entirely.

The note treats this as one of three named boundaries where its own thesis must be scoped rather than asserted unconditionally — the others being `io_uring` batching, which can remove the per-call trip in steady state, and the true cost of an actual crossing, which it attributes mainly to microarchitectural effects (pipeline drain, cache and TLB disturbance, Spectre/Meltdown mitigations) rather than to the `syscall` instruction itself. For vDSO-served calls the note says its mechanism-boundary framing is "simply false" — they are ordinary function calls — and it does not claim the vDSO covers all syscalls or behaves uniformly across environments; the cloud fallback case is presented as an open, environment-dependent gap rather than a settled exception. It also marks the cycle-count figures it cites for crossing cost as single-machine or single-study numbers, explicitly flagged as reported rather than verified, and says such figures should be

measured on the reader's own hardware rather than quoted from any one source. Beyond this claim, the note leaves an unresolved practitioner disagreement about how unstable Windows' `ntdll` really is, and flags its own register-level detail as specific to x86-64, not yet generalized to AArch64 or RISC-V.

Referenced sources

- [Anatomy of a system call, part 1](#)
- [Anatomy of a system call, part 2](#)
- [SYSCALL — Fast System Call \(x86 instruction reference\)](#)
- [x86-64 Syscall Entry — Linux Kernel Internals](#)
- [The Linux Kernel Driver Interface \(stable-api-nonsense\)](#)
- [Add kernel ↔ userspace ABI stability documentation](#)
- [x86 Assembly / Interfacing with Linux](#)
- [Do other Unix-like kernels have stable syscall ABIs?](#)
- [Libraries and Headers — Windows drivers](#)
- [winternl.h header — Win32 apps](#)
- [NTDLL Functions](#)
- [Windows X86-64 System Call Table \(XP through Windows 11 and Server\)](#)
- [Microsoft Windows library files — NTDLL.DLL](#)
- [Windows X86-64 System Call Table — discussion thread](#)
- [strace — linux syscall tracer](#)
- [ptrace and Syscall Interception — Linux Kernel Internals](#)
- [Process Monitor — Sysinternals](#)
- [NtTrace — Native API tracing for Windows](#)
- [What Makes System Calls Expensive](#)
- [vDSO and Virtual System Calls — Linux Kernel Internals](#)
- [Two frequently used system calls are ~77% slower on AWS EC2](#)
- [Why you should use `io\_uring` for network I/O](#)
- [Speed up `open\(\).read\(\)` pattern by reducing the number of system calls — cpython#120754](#)
- [win32file.CreateFile — PyWin32 documentation](#)
- [ltrace: When strace Is Not Enough](#)
- [System Calls and the User-to-Kernel Transition](#)
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#)
- [In Linux x86_64, are syscalls and `int 0x80` related?](#)

- [A Deep Dive Into Malicious Direct Syscall Detection](#)
- [Linux kernel system calls table](#)
- [io_uring Interface Overview](#)

10.27 V27 — The System Call as a Distinct Hardware Mechanism, Not a Costly Function Call

This note argues that the common mental model of a system call — a function call that merely costs more — produces wrong predictions, because on x86-64 the CPU instruction that performs the crossing, SYSCALL, bypasses ordinary call machinery entirely: it loads RIP from a Model-Specific Register, does not preserve the stack pointer, and loads fixed CS/SS descriptor values rather than consulting the GDT or LDT. Both Linux and Windows register their kernel entry points through this same instruction and MSR, differing only in what happens above it (a libc wrapper and syscall table on Linux versus an ntdll.dll stub and Service Descriptor Table on Windows). The note's central evidential claim, drawn from Soares and Stumm's peer-reviewed FlexSC measurements, is that a single `stat` call evicts 32 L1 instruction-cache lines, 186 L1 data-cache lines, 660 L2 lines, 2,559 L3 lines, and 21 data-TLB entries — and that at moderate call frequencies this indirect, post-return cost of lost processor state dominates the direct cost of the transition itself. That reframing makes the number of crossings, not bytes moved, the first-order performance variable.

The note is explicit about where this stops holding. It does not claim syscalls are always costly: Linux's vDSO and Windows' KUSER_SHARED_DATA serve some calls entirely in user space with no crossing at all, batched submission via io_uring or Windows I/O Rings amortizes the fixed cost toward the marginal cost, device-bound workloads are limited by I/O latency rather than crossing count, and at extremely high call frequency the direct transition cost overtakes the indirect cache-eviction cost. The argument is also flagged as x86-64-specific, since no consulted source measures ARM64's cost decomposition, and whether an ordinary syscall invalidates the TLB at all (independent of KPTI) is marked as genuinely contested between sources. The note further notes that no equal-footing Windows measurement of cache/TLB eviction exists — the Windows mechanism is well documented, but Windows cost is inferred from shared hardware behavior rather than measured — and that the FlexSC eviction figures come from 2010-era hardware, so the qualitative claim is treated as more durable than the specific numbers it rests on.

Referenced sources

- [SYSCALL — Fast System Call \(Intel 64 and IA-32 Architectures Software Developer's Manual\)](#), Intel Corporation

- [The Definitive Guide to Linux System Calls](#), packagecloud
- [FlexSC: Flexible System Call Scheduling with Exception-Less System Calls](#), Livio Soares and Michael Stumm, University of Toronto — USENIX OSDI (2010)
- [System calls on Windows x64](#), n4r1b (2019)
- [Page Table Isolation \(PTI\) — The Linux Kernel documentation](#), Linux kernel maintainers
- [KPTI/KAISER Meltdown Initial Performance Regressions](#), Brendan Gregg, Netflix (2018)
- [Kernel page-table isolation](#), Wikipedia
- [What Makes System Calls Expensive: A Linux Internals Deep Dive](#), Abhinav Upadhyay (2025)
- [Two frequently used system calls are ~77% slower on AWS EC2](#), packagecloud
- [Randomizing the KUSER_SHARED_DATA Structure on Windows](#), Microsoft Security Response Center (2022)
- [io_uring for High-Performance DBMSs: When and How to Use It](#), Jasny et al. (2025)
- [SubmitIoRing function \(ioringapi.h\)](#), Microsoft Learn
- [One I/O Ring to Rule Them All: A Full Read/Write Exploit Primitive on Windows 11](#), Yarden Shafir, Winsider Seminars & Solutions
- [io — Core tools for working with streams \(Python documentation\)](#), Python Software Foundation
- [FileStreamOptions.BufferSize Property \(System.IO\)](#), Microsoft Learn
- [Buffered I/O \(stdio library\) vs. Unbuffered I/O \(Sys. Calls\)](#), circuitlabs (2025)
- [15-213 Lecture 16 Activity Solutions: Unix I/O and Standard I/O](#), Carnegie Mellon University, 15-213 Introduction to Computer Systems
- [Meltdown/Spectre Benchmarks — Open Telekom Cloud Image Factory](#), T-Systems / Open Telekom Cloud
- [On the Costs of Syscalls \(2021\) — Hacker News discussion](#), Hacker News commenters
- [On the Costs of Syscalls](#), Georg Sauthoff (2021)
- [Inspection of I/O Operations from System Call Traces using Directly-Follows-Graph](#), Aravind Sankaran, Ilya Zhukov, Wolfgang Frings, Paolo Bientinesi (2024)
- [A Deep Dive into Zero-Copy Networking and io_uring](#), Jatin Umamtora, Medium
- [Comment thread on syscall cost and out-of-order execution](#), Hacker News — commenter "anarazel" (2021)

10.28 V28 — The kernel-object model behind sockets, and where each platform pins its contract

This note argues that a socket is not a protocol, a port, or a connection, but an object the kernel creates and owns, reached by a program through a handle — and that the real question in network I/O is what kind of object the kernel produced and what the program is licensed to do with it. Linux and Windows answer that question at different layers, not with different underlying mechanisms. On Linux, `socket()` returns a file descriptor that the kernel binds to a `struct file` on a pseudo-filesystem called `sockfs`, with reads, writes, closes, `select` and `fstat` dispatched through the same `file_operations` table used for ordinary files — a claim the note verifies both from the kernel's own `net/socket.c` source and from a C program and Python script the author ran on Linux 6.8.0, whose `/proc/self/fd` output and `fstat` results matched the kernel's own inode-naming code. On Windows, the NT kernel underneath Winsock also represents a socket as a file object, a handle to a device called `\Device\Afd`, which is precisely why `ReadFile`, `WriteFile` and I/O completion ports work on it at all; what Windows does differently is make the Winsock API itself — not the file-descriptor type — the binding contract, so a `SOCKET` must be closed with `closesocket` rather than `CloseHandle`, and reports failures through `WSAGetLastError` rather than `errno`.

The note is explicit that the common claim "Windows doesn't model sockets as files" is wrong, and treats correcting it as central to its argument rather than incidental. It marks exactly one step of its own reasoning, rather than any source, as inference: the reconciliation of two Microsoft Learn pages that read, taken naively, as contradicting each other on whether a socket handle can double as a file handle. It also treats the "Linux is readiness-based, Windows is completion-based" contrast as an aging shorthand rather than a settled fact, since Linux's `io_uring` networking surface is narrowing that gap, and it treats Windows' `AF_UNIX` support as only a partial convergence with the Linux socket model — matching in address family and naming, but still omitting descriptor-passing and other behavior that depends on the everything-is-a-file model. It does not attempt to judge which platform's design is better, only to describe where each one draws the line between mechanism and published contract.

Referenced sources

- [socket\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project
- [socket\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project
- [linux/net/socket.c](#) — Linux kernel source tree
- [Socket Data Type — Win32 apps](#) — Microsoft
- [Socket Handles — Win32 apps](#) — Microsoft

- [Error Codes - errno, h_errno and WSAGetLastError](#) — Win32 apps — Microsoft
- [WSAStartup function \(winsock.h\)](#) — Microsoft
- [closesocket function \(winsock.h\)](#) — Microsoft
- [CloseHandle function \(handleapi.h\)](#) — Microsoft
- [Select, fd_set, and FD_XXX Macros](#) — Win32 apps — Microsoft
- [socket](#) — Low-level networking interface (Python documentation) — Python Software Foundation
- [select](#) — Waiting for I/O completion (Python documentation) — Python Software Foundation
- [AF_UNIX comes to Windows](#) — Microsoft, Windows Command Line blog (2017)
- [Improving AFD Socket Visibility for Windows Forensics & Troubleshooting](#) — diversenok (2025)
- [12 Days of HaXmas: Does it Blend Like a Duck? \(Failing Portably\)](#) — Rapid7 (2014)
- [An Introductory 4.4BSD Interprocess Communication Tutorial](#) — Computer Systems Research Group, UC Berkeley
- [An Advanced 4.3BSD Interprocess Communication Tutorial](#) — Samuel J. Leffler et al., UC Berkeley
- [4.4BSD Networking Implementation Notes](#) — Computer Systems Research Group, UC Berkeley
- [rogerorr/NtTrace](#) — Roger Orr
- [Porting Socket Applications to Winsock](#) — Win32 apps — Microsoft
- [Under the Hood of AFD.sys Part 2: TCP handshake](#) — Mateusz Lewczak (2025)
- [cpython/Modules/socketmodule.h](#) — Python Software Foundation
- [Closing non-blocking socket causes leak](#) — Microsoft
- [vfs: mark pipes and sockets as stream-like file descriptors](#) — Linus Torvalds, Linux kernel (2019)
- [io_uring zero copy Rx](#) — The Linux Kernel documentation — The Linux Kernel Organization
- [Winsock Programmer's FAQ: General Winsock Information](#) — Warren Young

10.29 V29 — The kernel-object model behind sockets on Linux and Windows

This note establishes that a socket is not a protocol or a connection but an object the kernel creates and owns, and that Linux and Windows expose that object through different contracts. On Linux, `socket()` returns a literal file descriptor: the kernel

allocates a `struct socket`, binds it to a `struct file` on the `sockfs` pseudo-filesystem, and routes `read`, `write`, `close`, `select`, `fstat` and `fcntl` through the same dispatch table ordinary files use, evidenced by the kernel's `socket_file_ops` table and by a reproduced `/proc/self/fd` trace showing `fstat` reporting `S_ISSOCK` and the inode matching the `socket:[...]` symlink. On Windows, `socket()` returns an opaque `SOCKET`, and Microsoft's own documentation states that "a socket descriptor is not a file descriptor and must be used with the Winsock functions," with its own initialization (`WSAStartup`), teardown (`closesocket`), and error namespace (`WSAGetLastError`). The note argues the tempting shorthand — Unix treats sockets as files, Windows does not — is wrong: the NT kernel also holds a socket as a file object under `\Device\Afd`, which is what lets `ReadFile`, `WriteFile`, and I/O completion ports work on it. The real divergence is which layer each platform makes binding: Linux nails the promise to the file-descriptor contract, Windows nails it to the Winsock API, even though both kernels use a file object underneath.

The note does not claim this contract difference produces crashes or compiler errors — it stresses the opposite, that the failure mode is silent, surfacing later as a resource leak or a `WSAENOTSOCK` error. It treats the reconciliation of two seemingly contradictory Microsoft Learn pages (one says a socket descriptor is not a file descriptor, the other says a socket handle can optionally be a file handle) as its own inference rather than a claim either source makes directly, and leaves that tension visible rather than smoothing it over. Some claims about the Windows AFD IOCTL mechanism are marked "reported" rather than independently verified by the author, since `strace` was unavailable in the note's environment and the equivalent Windows trace was not reproduced firsthand. The note also flags as unsettled, going forward, whether Linux's `io_uring` networking surface fully closes the readiness-versus-completion gap with Windows I/O completion ports, calling the old "Linux is readiness-based, Windows is completion-based" shorthand one that is "aging" rather than resolved.

Referenced sources

- [socket\(2\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project
- [socket\(7\) — Linux manual page](#) — Michael Kerrisk / Linux man-pages project
- [linux/net/socket.c](#) — Linux kernel source tree
- [Socket Data Type — Win32 apps](#) — Microsoft
- [Socket Handles — Win32 apps](#) — Microsoft
- [Error Codes - errno, h_errno and WSAGetLastError — Win32 apps](#) — Microsoft
- [WSAStartup function \(winsock.h\)](#) — Microsoft
- [closesocket function \(winsock.h\)](#) — Microsoft
- [CloseHandle function \(handleapi.h\)](#) — Microsoft

- [Select, fd_set, and FD_XXX Macros — Win32 apps — Microsoft](#)
- [socket — Low-level networking interface \(Python documentation\) — Python Software Foundation](#)
- [select — Waiting for I/O completion \(Python documentation\) — Python Software Foundation](#)
- [AF_UNIX comes to Windows — Microsoft, Windows Command Line blog \(2017\)](#)
- [Improving AFD Socket Visibility for Windows Forensics & Troubleshooting — diversenok \(2025\)](#)
- [12 Days of HaXmas: Does it Blend Like a Duck? \(Failing Portably\) — Rapid7 \(2014\)](#)
- [An Introductory 4.4BSD Interprocess Communication Tutorial — Computer Systems Research Group, UC Berkeley](#)
- [An Advanced 4.3BSD Interprocess Communication Tutorial — Samuel J. Leffler et al., UC Berkeley](#)
- [4.4BSD Networking Implementation Notes — Computer Systems Research Group, UC Berkeley](#)
- [rogerorr/NtTrace — Roger Orr](#)
- [Porting Socket Applications to Winsock — Win32 apps — Microsoft](#)
- [Under the Hood of AFD.sys Part 2: TCP handshake — Mateusz Lewczak \(2025\)](#)
- [cpython/Modules/socketmodule.h — Python Software Foundation](#)
- [Closing non-blocking socket causes leak — Microsoft](#)
- [vfs: mark pipes and sockets as stream-like file descriptors — Linus Torvalds, Linux kernel \(2019\)](#)
- [io_uring zero copy Rx — The Linux Kernel documentation — The Linux Kernel Organization](#)
- [Winsock Programmer's FAQ: General Winsock Information — Warren Young](#)

10.30 V30 — The buffering stack between a program's write and durable storage

This note argues that durability is not a property of a single write call but of a sequence of independent flushes, each crossing one of four boundaries: a language-runtime buffer, a kernel `write()` syscall, the kernel's page cache, and the storage device's own cache. Each boundary has its own trigger and its own cost, and a successful `write()` only reports that the first kernel-side buffer accepted the bytes — a statement about custody, not persistence. The evidence for this is drawn largely from primary and vendor-normative sources: the Linux `fsync(2)` man page, POSIX text, and Microsoft's Win32 API docu-

mentation for `FlushFileBuffers` and related flags. The note's central case study is the 2018 "fsyncgate" incident, in which PostgreSQL, InnoDB, and WiredTiger all discovered that a failed writeback on Linux causes the kernel to mark the failed pages clean and report the error only once — so a retrying application sees a false success. It frames this as a genuine contract disagreement between two internally coherent positions (the kernel's and PostgreSQL's), not a bug on either side, and traces how all three database engines converged independently on panicking and recovering from a write-ahead log rather than retrying. It then does the same analysis for Windows at equal depth, concluding that Windows arranges the same four layers under different names but — critically — has no documented contract at all for post-failure behavior, which it argues is a worse position than Linux's merely hostile one, because no correct application response can be designed from undocumented behavior.

The note is explicit about where its own model runs out. It flags the four-layer picture itself as a teaching simplification that breaks down for network filesystems, memory-mapped I/O, and copy-on-write filesystems. Several specific claims are marked as inference or speculation rather than verified fact: the mapping from Python's `os.fsync` to the Windows `_commit()` function is called an inferred link rather than a documented one, and the claim that no Windows tracing tool exposes device flush commands as cleanly as Linux's `blktrace` is marked speculative, with an open request for confirming evidence. Most centrally, the note states outright that nobody has publicly characterized whether Windows shares Linux's non-retryable failure class — the question was asked on Hacker News and Stack Overflow and never received an authoritative answer — so it deliberately stops short of claiming Windows either shares or avoids the bug. It also cautions that the PostgreSQL PANIC-on-error response was calibrated for 2018-era local disks, and that on cloud block storage an I/O error is often transient, changing the availability cost of that same design choice without changing its correctness logic.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026)
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project
- [Durability: Linux File APIs](#) — Evan Jones (2020)
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017)
- [io](#) — Core tools for working with streams — Python Software Foundation
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022)

- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode
- [Page Cache Writeback](#) — Linux Kernel Internals
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn
- [File Caching](#) — Microsoft Learn
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al.
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026)
- [Life of a Write](#) — Linux Kernel Internals
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018)
- [Fsync Errors](#) — PostgreSQL wiki
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020)
- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn
- [File Buffering](#) — Microsoft Learn
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012)
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019)
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019)
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow
- [DTrace on Windows](#) — Microsoft Learn
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn
- [Write-Ahead Logging \(SQLite\)](#) — SQLite

10.31 V31 — Mechanisms for observing a running system's interaction with the OS

This note argues that a running program is opaque, and that answering "what is this process asking the operating system to do, right now, and in what order?" requires attaching a second piece of machinery to it — and every such attachment is a design decision with consequences. It surveys seven mechanisms across Linux and Windows (`ptrace` / `strace`, NtTrace, IAT patching/StraceNT, dynamic binary instrumentation, eBPF/`bpfttrace`, DTrace, and ETW) and organizes them around a single claim: observation is attachment, and attachment has a location. Where a mechanism attaches — kernel syscall entry, a user-mode debug interface, the import table, a rewritten instruction stream, an in-kernel programmable probe, or an OS-published event pipeline — mechanically determines its visibility, its blind spot, and its cost. Linux and Windows illustrate this because they place their stable boundary differently: Linux guarantees the raw syscall ABI, so `ptrace` intercepts there directly, while Windows guarantees stability at the Win32 layer instead, which is why NtTrace works by placing breakpoints in `ntdll` rather than intercepting the kernel boundary. The note further argues the observer effect is not just quantifiable overhead: tracing overhead is non-uniform, incurred only where events fire, and can distort concurrent behaviour qualitatively — citing a controlled study showing tracing can amplify or suppress concurrency up to deadlock via thread starvation, and separately a case where under 1.8% additional instrumentation swung measured performance across a $\pm 15\%$ range.

The note does not claim any one mechanism is superior; it explicitly frames the choice as question-dependent — which blind spot is affordable and which perturbation would destroy the phenomenon under study. It treats published overhead figures as unsettled when unaccompanied by their workload, noting that `strace` overhead alone has been reported anywhere from roughly $2\times$ to $442\times$ depending on syscall rate, and states this spread is not a contradiction to resolve but a reason to distrust any single number. Several claims are explicitly marked as the note's own inference rather than sourced fact, and it does not evaluate which platform's design choice is preferable — only what each costs.

Referenced sources

- [ptrace and Syscall Interception](#) — Linux Kernel Internals
- [strace Wow Much Syscall](#) — Brendan Gregg (2014)
- [Windows vs. Linux Syscall Interfaces: Win32 API, NT Native API, and WSL](#) — kinda-technical.com (2026)
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023)
- [NtTrace — Native API tracing for Windows](#) — Roger Orr

- [rogerorr/NtTrace — "How it works"](#) — Roger Orr
- [vdso\(7\)](#) — Linux manual page — Linux man-pages project
- [StraceNT — System Call Tracer for Windows NT](#) — Pankaj Garg
- [Debugging eBPF verifier: Best Practices for High-Performance Kernel Programming](#) — Raven
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — George V. Neville-Neil et al., USENIX ;login: (2017)
- [Tracing and its observer effect on concurrency](#) — Indigo Orton, Alan Mycroft — ACM (2021)
- [We Have It Easy, But Do We Have It Right? / observer effect and measurement bias](#) — Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, Peter Sweeney (2008)
- [System Calls and the User-to-Kernel Transition](#) — Yunolay (2026)
- [Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers, FOSDEM (2020)
- [System Call Tracing With eBPF: Move Beyond Ptrace](#) — SysTutorials
- [Ptrace & You](#) — Omar S. Navarro Leija
- [vDSO and Virtual System Calls](#) — Linux Kernel Internals
- [System-wide syscalls tracing and monitoring in Linux using bpftrace](#) — Narasimha Prasanna HN
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygódzki
- [ipankajg/stracent](#) — StraceNT — Pankaj Garg
- [Unveiling Dynamic Binary Instrumentation Techniques](#) — arXiv preprint (2025)
- [How to do a million watchpoints: Efficient Debugging using Dynamic Instrumentation](#) — Qin Zhao et al., CC (2008)
- [What is eBPF? An Introduction and Deep Dive into the eBPF Technology](#) — eBPF Foundation
- [Inside the eBPF Verifier: Ensuring Program Safety and Complexity Bounds](#) — How Tech — Systems Programming
- [Loops and Iterators in eBPF](#) — Teodor J. Podobnik
- [CVE-2021-33624](#) — Red Hat Customer Portal — Red Hat
- [Eliminating eBPF Tracing Overhead on Untraced Processes](#) — Milo Craun et al., eBPF '24
- [DTrace Capabilities](#) — Oracle — Solaris Dynamic Tracing Guide
- [Dynamic Instrumentation of Production Systems](#) — Bryan M. Cantrill, Michael W. Shapiro, Adam H. Leventhal — USENIX ATC (2004)

- [DTrace on Windows](#) — Microsoft Learn
- [Announcing DTrace for Windows Insider](#) — Packt (2019)
- [ETW — Event Tracing for Windows](#) — Mjolnir Security
- [NT Kernel Logger Trace Session](#) — Microsoft Learn
- [Windows Performance Recorder](#) — Microsoft Learn
- [Doubling Down: Detecting In-Memory Threats with Kernel ETW Call Stacks](#) — Elastic Security Labs
- [prash-wghats/DTrace-win32](#) — prash-wghats
- [CSE 422S Studio 5: Tracing the Linux Kernel](#) — Christopher Gill, Washington University in St. Louis
- [microsoft/DTrace-on-Windows](#) — Microsoft
- [bpf-helpers\(7\)](#) — Linux manual page — Linux man-pages project
- [bpf: Change value of MAX_TAIL_CALL_CNT from 32 to 33](#) — Linux kernel mailing list / stable backport series (2022)
- [Windows 11, version 26H1](#) — Microsoft Support (2026)
- [Updating Syscall Tables](#) — SysWhispers4 — SysWhispers4 project documentation
- [strace 7.1 release notes](#) — strace project (2026)

10.32 V32 — Virtual memory and the page cache as a fault-driven mechanism

This note argues that virtual memory is not an optimization layered on top of memory but the mechanism by which memory works at all, and that the page fault is its interface rather than an error path. The claim rests on three properties: address translation is indirect, population is lazy (a mapping can exist before any physical page backs it), and physical memory is a reclaimable cache rather than a fixed container. It supports this with the Linux kernel's own description of virtual memory as abstracting physical memory and enabling demand paging, and with Windows kernel-internals teaching material stating that the memory manager is the actual cache manager. From there the note builds a fault taxonomy — minor/soft, demand-zero, copy-on-write, and major/hard faults — with order-of-magnitude costs on each side (microseconds for RAM-resolved faults, roughly 0.1 ms on SSD and 10 ms on HDD for disk-resolved ones), and works through five implementation areas: fault accounting, why RSS is a misleading number (it double-counts shared pages), the page cache and the hazard it creates for benchmarking, `mmap`'s real cost trade-offs including a CIDR 2022 measurement of `mmap` collapsing under page-cache eviction, and the tools that make each mechanism observable on Linux and Windows.

The note is explicit about what it does not settle. It defers the atomic argument that a page fault is not an error, and the argument that `mmap` relocates rather than removes cost, to separate zettels it has not yet spun out, treating itself only as the hub that holds the topic together. It states plainly that no single memory metric (RSS, PSS, USS, working set) is simply correct — each answers a different question, and PSS in particular can rise for surviving processes when a sibling process exits, which looks like new allocation but isn't. It also flags its own model as dated in one respect: Linux has largely replaced the page as the page-cache's literal data structure with folios since kernel 5.16, so the page-granular teaching model, while still pedagogically sound, no longer matches what a reader of `mm/` source would find. Finally, it treats reclaim policy (MGLRU) as a genuinely open, contested area rather than settled material, citing an active 2026 removal proposal.

Referenced sources

- [Concepts overview — The Linux Kernel documentation](#) — The Linux Kernel Organization
- [Windows Kernel Internals: Cache Manager](#) — Microsoft Corporation, University of Tokyo lecture series
- [Page fault — Grokipedia](#) — Grokipedia
- [Working Set - Win32 apps](#) — Microsoft Learn
- [Copy-on-Write and the Page-Fault Lifecycle, Step by Step](#) — PandaStack
- [How to Check Memory Usage per Process in Linux](#) — GoLinuxCloud
- [perf-trace\(1\) — Linux manual page](#) — man7.org / Linux man-pages project
- [The Basics of Page Faults](#) — Microsoft Community Hub, AskPerf team blog
- [Windows Performance Monitor basics](#) — SQLShack
- [Memory Management Tracing Events - Win32 apps](#) — Microsoft Learn
- [Why does Linux, by default, overcommit memory?](#) — Unix & Linux Stack Exchange
- [Virtual vs Physical vs Resident Memory](#) — Linux Kernel Internals
- [The /proc Filesystem — The Linux Kernel documentation](#) — The Linux Kernel Organization
- [RAMMap - Sysinternals](#) — Microsoft Learn / Sysinternals
- [Accurately measuring memory usage](#) — Phusion
- [Readahead - Linux Kernel Internals](#) — Linux Kernel Internals
- [Page Cache Writeback - Linux Kernel Internals](#) — Linux Kernel Internals
- [Windows Memory Management in Windows 10 and Windows 11](#) — CyberRaiden
- [Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing](#) — Raasveldt, Holanda, Gubner & Mühleisen, DBTest

- [Understanding and tuning the page cache on Debian](#) — Big Iron
- [Excessive Page Faults Generated By Windows Applications](#) — Broadcom Knowledge Base
- [Common disk benchmarking mistakes](#) — Stefan Hajnoczi
- [Memory-Mapped I/O - Linux Kernel Internals](#) — Linux Kernel Internals
- [File Mapping - Win32 apps](#) — Microsoft Learn
- [Creating Named Shared Memory](#) — MicrosoftDocs/win32 repository
- [Memory-Mapped Files \(mmap\): A Practical Guide to Faster I/O and Shared Memory](#) — martinuke0
- [mmap SIGBUS error and initializing the file](#) — Stack Overflow
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn
- [mmap — Memory-mapped file support](#) — Python Software Foundation
- [Are You Sure You Want to Use MMAP in Your Database Management System?](#) — Crotty, Leis & Pavlo, CIDR
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — Neville-Neil et al., USENIX ;login:
- [Teaching Operating Systems Concepts with Execution Visualization](#) — Giraldeau, Dagenais & Boucheneb, ASEE
- [Mastering strace, perf, and eBPF — 2026 Cheat Sheet](#) — Tech Bytes
- [CS 110: Principles of Computer Systems — assign3](#) — Stanford University
- [Page Cache - Linux Kernel Internals](#) — Linux Kernel Internals
- [On pages and folios](#) — Jonathan Corbet, LWN.net
- [Reconsidering the multi-generational LRU](#) — Jonathan Corbet, LWN.net

10.33 V33 — File I/O durability across the OS buffering stack

This note establishes that durability is not a property of a single write call but of a sequence of independent flushes, each crossing one of four boundaries between a program and the physical medium: a language-runtime buffer, a libc/user-space buffer, the kernel's page cache, and the storage device's own cache. A successful `write()` reports only that the first kernel-side buffer accepted the bytes — a statement about custody, not persistence — and neither `close()` nor Python's `with` block closes the remaining gap. The evidence for the sharpest claim is the 2018 PostgreSQL "fsync-gate" incident: Linux's `errseq_t` mechanism reports a writeback error to only the next `fsync()` call and then discards the failed pages, so a database that retried `fsync()` after a checkpoint failure was, unknowingly, retrying against data the kernel had already

thrown away. The note frames this as a genuine contract disagreement between two internally coherent positions — POSIX never specified behavior after a failed `fsync` — rather than a bug on either side, and traces how PostgreSQL, InnoDB, and WiredTiger independently converged on panicking and recovering from the write-ahead log instead of retrying. Windows is covered at the same depth: it has the identical four-layer structure but, per the note's research, no documented contract at all for what happens after a flush failure, which it argues is the worse of the two positions because no correct response can be engineered from silence.

The note is explicit about what it does not claim. It does not assert that Windows shares Linux's exact non-retryable failure class — it states plainly that nobody has publicly characterized this and Microsoft has not documented it, offering only an inference about the likely shape of the problem. It does not treat the four-layer model as complete, flagging network filesystems, memory-mapped I/O, and copy-on-write filesystems as cases the simplification does not fit. It also does not present PANIC-on-error as a universal rule, noting that this response was calibrated to 2018-era local disks and may cost availability unnecessarily on cloud block storage, where I/O errors are often transient. Two atomic claims — the exact meaning of a successful `write()`, and the non-retryability of a failed `fsync()` — are deliberately left unexpanded here, tracked instead as separate, not-yet-written notes.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026)
- [fsync\(2\) — Linux manual page](#) — Linux man-pages project
- [Durability: Linux File APIs](#) — Evan Jones (2020)
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017)
- [io — Core tools for working with streams](#) — Python Software Foundation
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022)
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode
- [Page Cache Writeback](#) — Linux Kernel Internals
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn

- [File Caching](#) — Microsoft Learn
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al.
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026)
- [Life of a Write](#) — Linux Kernel Internals
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018)
- [Fsync Errors](#) — PostgreSQL wiki
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020)
- [CreateFileW function](#) — [FILE_FLAG_WRITE_THROUGH](#) and [FILE_FLAG_NO_BUFFERING](#) semantics — Microsoft Learn
- [File Buffering](#) — Microsoft Learn
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012)
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019)
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019)
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow
- [DTrace on Windows](#) — Microsoft Learn
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn
- [Write-Ahead Logging \(SQLite\)](#) — SQLite

10.34 V34 — Virtual memory and the page cache

This note establishes that a second read of a file is fast because the first read already populated the page cache, and that readahead sharpens this further: the kernel detects sequential access, maintains a per-file-description readahead window, and issues I/O for pages before the process asks for them, which is why "a sequential `read()` just happens to return faster because the data was already fetched." The evidence is the Linux kernel's own documentation on the page cache's purpose, plus a kernel-internals reference that quotes `mm/readahead.c` and `struct file_ra_state` directly. The note is explicit that this behavior is conditional, not universal: readahead applies only to

buffered I/O, and `O_DIRECT` bypasses the page cache and therefore readahead entirely. It also traces the identical result on Windows through a different mechanism — the Cache Manager maps cached streams into kernel address space and tracks reusable, already-cached content on a standby list, which is what makes a Windows soft fault cheap. From there the note turns the same fact into a warning: because a warm cache is invisible from user space, a "fast" run and a genuinely cold run are easy to conflate, which is exactly the trap the database-benchmarking literature it cites was written to name.

The note does not claim that caching effects can be eliminated or fully controlled by the experimenter. It explicitly rejects the idea that dropping the OS cache produces a truthful cold measurement: `drop_caches` does not touch the storage device's own DRAM cache or a controller/HBA cache, and it also disturbs the very prefetch and writeback pattern being measured, so most published "cold-cache" benchmarks are wrong by its own citation. It further notes that in a cloud VM, dropping caches may be impossible to do meaningfully at all, since caching also happens at the virtualization host. The note treats this as an open, layered problem rather than a solved one — its stated best practice is not "clear the cache" but "state which cache layer your number includes, because you cannot eliminate all of them" — and it does not attempt to quantify how much of a given speedup is attributable to the page cache versus readahead specifically, leaving that decomposition unmeasured.

Referenced sources

- [Concepts overview — The Linux Kernel documentation](#) — The Linux Kernel Organization
- [Windows Kernel Internals: Cache Manager](#) — Microsoft Corporation, University of Tokyo lecture series
- [Page fault — Grokipedia](#) — Grokipedia
- [Working Set - Win32 apps](#) — Microsoft Learn
- [Copy-on-Write and the Page-Fault Lifecycle, Step by Step](#) — PandaStack
- [How to Check Memory Usage per Process in Linux](#) — GoLinuxCloud
- [perf-trace\(1\) — Linux manual page](#) — man7.org / Linux man-pages project
- [The Basics of Page Faults](#) — Microsoft Community Hub, AskPerf team blog
- [Windows Performance Monitor basics](#) — SQLShack
- [Memory Management Tracing Events - Win32 apps](#) — Microsoft Learn
- [Why does Linux, by default, overcommit memory?](#) — Unix & Linux Stack Exchange
- [Virtual vs Physical vs Resident Memory](#) — Linux Kernel Internals
- [The /proc Filesystem — The Linux Kernel documentation](#) — The Linux Kernel Organization

- [RAMMap - Sysinternals](#) — Microsoft Learn / Sysinternals
- [Accurately measuring memory usage](#) — Phusion
- [Readahead - Linux Kernel Internals](#) — Linux Kernel Internals
- [Page Cache Writeback - Linux Kernel Internals](#) — Linux Kernel Internals
- [Windows Memory Management in Windows 10 and Windows 11](#) — CyberRaiden
- [Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing](#) — Raasveldt, Holanda, Gubner & Mühleisen, DBTest
- [Understanding and tuning the page cache on Debian](#) — Big Iron
- [Excessive Page Faults Generated By Windows Applications](#) — Broadcom Knowledge Base
- [Common disk benchmarking mistakes](#) — Stefan Hajnoczi
- [Memory-Mapped I/O - Linux Kernel Internals](#) — Linux Kernel Internals
- [File Mapping - Win32 apps](#) — Microsoft Learn
- [Creating Named Shared Memory](#) — MicrosoftDocs/win32 repository
- [Memory-Mapped Files \(mmap\): A Practical Guide to Faster I/O and Shared Memory](#) — martinuke0
- [mmap SIGBUS error and initializing the file](#) — Stack Overflow
- [MapViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn
- [mmap — Memory-mapped file support](#) — Python Software Foundation
- [Are You Sure You Want to Use MMAP in Your Database Management System?](#) — Crotty, Leis & Pavlo, CIDR
- [Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation](#) — Neville-Neil et al., USENIX ;login:
- [Teaching Operating Systems Concepts with Execution Visualization](#) — Giraldeau, Dagenais & Boucheneb, ASEE
- [Mastering strace, perf, and eBPF — 2026 Cheat Sheet](#) — Tech Bytes
- [CS 110: Principles of Computer Systems — assign3](#) — Stanford University
- [Page Cache - Linux Kernel Internals](#) — Linux Kernel Internals
- [On pages and folios](#) — Jonathan Corbet, LWN.net
- [Reconsidering the multi-generational LRU](#) — Jonathan Corbet, LWN.net

10.35 V35 — NtTrace, a user-mode Windows Native API tracer

This note documents NtTrace, a user-mode Windows tool that traces calls into `ntdll.dll` — the Native API boundary between application code and the NT kernel — by attaching as a debugger and placing breakpoints around the native call stubs, then reporting on return from each call. The note reproduces the one worked example of output that the tool's own project page publishes: a capture of NtTrace running against `cmd`, showing calls such as `NtOpenFile`, `NtQueryVolumeInformationFile`, and `NtWriteFile` with decoded arguments and NTSTATUS/Win32 return codes. The note states plainly that the sample **is a 32-bit, XP-era capture** — `ntdll.dll` at `77F40000`, stack addresses like `0x0012FD8C`, and a process base at `4AD0B814` all point to an old Windows build. It treats the capture as authentic and structurally correct: the mechanism it illustrates — handles rather than file descriptors, NT object paths under `\\??\\`, dual NTSTATUS/Win32 error reporting, unbuffered console writes — still holds. The evidence for the capture is the tool's own published documentation, retrieved by the note's author directly from the project home page.

The note is explicit about what this single capture cannot support. It is not paired with a same-program `strace` trace, so it cannot serve as a side-by-side Windows/Linux comparison — that would require both traces produced together. Because the addressing is from an XP-era, 32-bit run, the note warns against presenting the capture as representative of what a trace looks like today, since a current capture would show 64-bit addresses. It also flags one detail as unresolved: the sample lists the `NtFsControlFile` argument `UserApcContext` twice, and the note cannot say whether that is a quirk of NtTrace's configuration file or an error on the published page, recommending a re-check against the live source before any public use. Finally, the note records that its own transcription was produced through a fetch-and-convert step rather than copied from a terminal, and asks for a spot-check against the original page before the capture is used anywhere else.

Referenced sources

- [NtTrace — Native API tracing for Windows](#) — Roger Orr
- [rogerorr/NtTrace — repository README](#) — Roger Orr
- [Home · rogerorr/NtTrace Wiki](#) — Roger Orr
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki
- [AWStrace: Another Windows Strace](#) — downwithup
- [StraceNT – System Call Tracer for Windows NT](#) — Pankaj Garg
- [System Call Tracer for Windows \(drstrace\)](#) — Dr. Memory / DynamoRIO project
- [mxmssh/drltrace — README](#) — Maksim Shudrak

- [Introducing QTrace, a "zero knowledge" system call tracer](#) — Roberto Paleari
- [strace Wow Much Syscall](#) — Brendan Gregg
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange
- [A 10-minute guide to the Linux ABI](#) — Alison Chaiken, opensource.com
- [A Deep Dive Into Malicious Direct Syscall Detection](#) — Or Chechik and Ofir Ozer, Palo Alto Networks
- [Overload Content by Article — ACCU](#) — ACCU (Association of C and C++ Users)
- [Windows System Calls & Native API: A Guide to Assembly Calling Conventions Explained](#) — linuxvox.com
- [mandiant/STrace — A DTrace on Windows Reimplementation](#) — Mandiant

10.36 V36 — FreeBSD's ABI Compatibility Stance

This note's headline argument is that "Windows is the anomaly" gets the syscall-boundary comparison backwards: among Unix-family systems, it is Linux that treats the raw syscall number as a public contract, while FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all decline that guarantee. FreeBSD is the one case in that list the note treats as contested rather than settled. The project's own vendor wiki states plainly that "No compatibility for API and ABI is guaranteed from one to the next major release," which is what places FreeBSD in the no-guarantee column of the survey the note relies on. But a former FreeBSD core-team contributor pushes back on the practical reading of that statement, describing a syscall ABI that is in fact stable in operation: superseded system calls from prior releases remain available through versioned syscall definitions and compiled-in `COMPAT_FREEBSD*` kernel options, which ship by default in the generic build reaching back to FreeBSD 4 — twenty-three years of continuity. The note treats both accounts as describing the same underlying mechanism and differing only on whether a guarantee contingent on a kernel build option still counts as a guarantee, rather than treating one source as simply wrong.

The note is explicit about what this nuance does not change: it does not disturb the note's headline correction, since FreeBSD still publishes no formal Linux-style cross-release promise, and Linux remains the outlier for making the number itself part of its contract. What the note does draw from this case is a narrower caution — that the five non-Linux systems it lists should not be read as equally unstable, because FreeBSD's practical compatibility is markedly stronger than a bare "no guarantee" reading implies. The note grades this specific disagreement as genuinely contested, distinguishing it from two other contested items elsewhere in the same note where sources agree on

the underlying mechanism and differ only in characterization; here the two sources disagree about a substantive fact of practice, surfaced by comparing results across two different search engines. The note does not attempt to resolve which framing is correct, and it does not extend this in-practice-stronger reading to OpenBSD, NetBSD, Solaris/illumos, or macOS, which it leaves under the stricter no-guarantee framing supplied by its survey source.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — The Linux Kernel Organization
- [j00ru/windows-syscalls — Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — Mateusz "j00ru" Jurczyk
- [NtQuerySystemInformation function \(winternl.h\)](#) — Win32 apps — Microsoft
- [Calling Internal APIs — Win32 apps](#) — Microsoft
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — Linux kernel source tree
- [include/uapi/asm-generic/unistd.h](#) — Linux kernel source tree
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange
- [FreeBSD Vendor Information](#) — The FreeBSD Project
- [openbsd-tech: system call origin verification](#) — OpenBSD project mailing list
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — The Go Project
- [syscall\(2\) — NetBSD Manual Pages](#) — The NetBSD Project
- [port-solaris mailing list, November 2010: syscall number reservation](#) — port-solaris list archive (Carnegie Mellon mirror)
- [NtTrace — Native API tracing for Windows](#) — Roger Orr
- [strace — linux syscall tracer](#) — The strace project
- [DTrace on Windows — Windows drivers](#) — Microsoft
- [DTrace — Windows Commands](#) — Microsoft
- [Inside the Native API](#) — Mark Russinovich, Sysinternals
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — Mark H. Lucovsky, Microsoft
- [The Sad History of the Microsoft POSIX Subsystem](#) — Brian Reiter (2010)
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023)
- [Never break userspace](#) — LWN.net

- [Updating Syscall Tables — SysWhispers4 documentation](#) — SysWhispers4 project documentation
- [CRT-free in 2023: tips and tricks — discussion](#) — Lobsters discussion thread
- [Architecture of Windows NT](#) — Wikipedia
- [U.S. v. Microsoft Corporation — Information on the Settlement](#) — U.S. Department of Justice, Antitrust Division
- [Syscalls on Windows 11](#) — hammertux

10.37 V37 — The four-layer buffering-and-durability boundary model for file I/O

This note argues that durability is not a property of a single write call but of a sequence of flushes, each crossing one of four independent boundaries between a program and persistent media: a language-runtime buffer, a kernel-side write buffer, the kernel's page cache, and the storage device's own cache — each with its own flush trigger and its own cost to cross. It establishes, from Linux and Windows API documentation and kernel sources, that a successful `write()` call only confirms the kernel accepted the bytes into its own buffer — a statement about custody, not persistence — and that closing a file or exiting a `with` block does not force a flush to the device. The note's central case study is the 2018 "fsyncgate" incident, in which Linux's page cache discarded dirty pages after a failed writeback and reported the error only once, causing PostgreSQL's checkpoint to believe a retried `fsync()` had succeeded when it had not; PostgreSQL, InnoDB and WiredTiger independently converged on crashing and recovering from a write-ahead log rather than retrying. It extends the same four-layer model to Windows, showing that `FlushFileBuffers`, `FILE_FLAG_WRITE_THROUGH`, and `FILE_FLAG_NO_BUFFERING` cross different combinations of the same boundaries, and argues — its most load-bearing claim — that Microsoft has never publicly documented what happens to buffered data after a flush failure, unlike Linux's documented, if hostile, contract.

The note is explicit that this is a teaching simplification: it does not claim the four-layer table applies unmodified to network filesystems, memory-mapped I/O, or copy-on-write filesystems, each of which adds or reshapes layers. On the Windows question at its center, it does not claim "Windows is safe" or "Windows has the same bug" — only that the contract is undocumented — and it flags the CRT `_commit()`-to-`FlushFileBuffers` mapping and the claim that Windows exposes no dirty-page-age knob as inferences rather than sourced facts. It treats PostgreSQL's PANIC-on-`EIO` response as calibrated for 2018-era local disks and explicitly unsettled for cloud block storage, where I/O errors are often transient and self-clearing. It also leaves three questions open in its

own "Grounding gap" section: whether any Windows tracing mechanism exposes device-level `FLUSH_CACHE` commands as cleanly as `blktrace` does on Linux, whether `_commit()` actually maps to `FlushFileBuffers` internally, and whether Windows exposes any user-visible control over dirty-page age comparable to Linux's `dirty_expire_centisecs`. The note does not resolve these; it names them as unresolved rather than asserting an answer.

Referenced sources

- [fsync\(\), Barriers, and the Hardware Durability Contract](#) — Sagar Madala / Medium (2026)
- [fsync\(2\)](#) — Linux manual page — Linux man-pages project
- [Durability: Linux File APIs](#) — Evan Jones (2020)
- [FlushFileBuffers function \(fileapi.h\)](#) — Microsoft Learn
- [Revised notes on the reliability of FlushFileBuffers](#) — Raymond Chen / Microsoft Developer Blogs (2017)
- [io](#) — Core tools for working with streams — Python Software Foundation
- [FileStreamOptions.BufferSize Property](#) — Microsoft Learn
- [CSE333 L07: Buffering, POSIX I/O, Syscalls](#) — University of Washington, CSE333 (2022)
- [Bug 1400144 — glibc: Performance penalty due to block size-based buffering in stdio](#) — Red Hat Bugzilla
- [What Buffer Flush Means in C \(and C++\)](#) — TheLinuxCode
- [Page Cache Writeback](#) — Linux Kernel Internals
- [Performance Tuning for Cache and Memory Manager Subsystems](#) — Microsoft Learn
- [File Caching](#) — Microsoft Learn
- [Atomic Commit In SQLite](#) — SQLite / D. Richard Hipp et al.
- [Power Loss Protection shouldn't be an enterprise exclusive](#) — How-To Geek (2026)
- [Life of a Write](#) — Linux Kernel Internals
- [The errseq_t datatype](#) — The Linux Kernel documentation — Linux kernel documentation
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (curator); pgsql-hackers thread participants (2018)
- [Fsync Errors](#) — PostgreSQL wiki
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau / USENIX ATC (2020)

- [CreateFileW function — FILE_FLAG_WRITE_THROUGH and FILE_FLAG_NO_BUFFERING semantics](#) — Microsoft Learn
- [File Buffering](#) — Microsoft Learn
- [Windows Write Caching — Part 2: An Overview for Application Developers](#) — Dilip Naik (2012)
- [Discussion: does FlushFileBuffers\(\) on Windows also forget failed writes? / Files Are Hard thread](#) — Hacker News (2019)
- [Is it safe to call the Win32 FlushFileBuffers\(\) function again if it previously returned an error?](#) — Stack Overflow (2019)
- [os — Miscellaneous operating system interfaces](#) — Python Software Foundation
- [Monitoring certain system calls done by a process in Windows](#) — Stack Overflow
- [DTrace on Windows](#) — Microsoft Learn
- [FlushViewOfFile function \(memoryapi.h\)](#) — Microsoft Learn
- [Write-Ahead Logging \(SQLite\)](#) — SQLite

10.38 V38 — Where Windows and Linux each place their stable syscall boundary

This note argues that Windows and Linux both promise application developers a stable interface to the kernel, but they draw the line at different layers, and the difference is structural rather than a matter of degree. Linux draws the boundary at the syscall trap itself: a syscall is a numbered entry in a per-architecture table, the kernel's own ABI documentation states that such interfaces are "expected to never change and always be available," and numbers like `open` = 2 have held for the life of the x86-64 port, so a program may legitimately bypass the C library and issue the trap directly. Windows draws the boundary one layer higher, at Win32. Below Win32 sits the Native API exported by `ntdll.dll`, which Microsoft's own SDK documentation explicitly disclaims as "internal to the operating system and subject to change from one release of Windows to another," and below `ntdll` sits an undocumented, renumbered System Service Number. The note backs this with a recomputation of Windows syscall-number drift directly from the primary `nt.csv` artifact: across measured builds a majority of functions (52%) decrease in number at least once between versions, and two concurrently shipping SKUs disagree on numbering for 63% of shared functions. It extends the finding into practice — tracers attach at whichever layer has stable names (`ptrace` on the Linux syscall interface, the Windows debug interface on `ntdll`) — and into a portability rule: write down names, not numbers, since on Windows the number was never yours to write down.

The note is explicit about what it does not claim. It does not conclude that Windows is the unstable outlier; it argues the opposite, that FreeBSD, OpenBSD, NetBSD, Solaris/illumos and macOS all behave like Windows, making Linux the exception among Unix-family systems that needs explaining, not Windows. It leaves unsettled whether the Win32/`ntdll` split reflects deliberate design intent or is a vestige of NT's abandoned multi-subsystem architecture, and similarly leaves open whether `ntdll` counts as a genuine interface or merely a traced convention — both are flagged as framing disagreements rather than factual ones. It also treats as an open question whether Microsoft's disclosure of Native API prototypes was compelled by the 2002 antitrust settlement or an independent decision, and whether Windows syscall-number reshuffling is purely mechanical (an artifact of alphabetically ordered service tables) or partly deliberate — the note asserts only the mechanical explanation and flags the alternative as unresolved. Similarly, it qualifies "Linux never breaks userspace" as a stronger claim than the kernel's own documentation actually makes.

Referenced sources

- [ABI stable symbols — The Linux Kernel documentation](#) — The Linux Kernel Organization
- [j00ru/windows-syscalls — Windows System Call Tables](#) — Mateusz "j00ru" Jurczyk
- [Windows X86-64 System Call Table \(XP/2003/Vista/7/8/10/11 and Server\)](#) — Mateusz "j00ru" Jurczyk
- [NtQuerySystemInformation function \(winternl.h\)](#) — Win32 apps — Microsoft
- [Calling Internal APIs — Win32 apps](#) — Microsoft
- [arch/x86/entry/syscalls/syscall_64.tbl](#) — Linux kernel source tree
- [include/uapi/asm-generic/unistd.h](#) — Linux kernel source tree
- [Do other Unix-like kernels have stable syscall ABIs?](#) — Unix & Linux Stack Exchange
- [FreeBSD Vendor Information](#) — The FreeBSD Project
- [openbsd-tech: system call origin verification](#) — OpenBSD project mailing list
- [all: stop using non-libc system calls on OpenBSD \(golang/go#36435\)](#) — The Go Project
- [syscall\(2\) — NetBSD Manual Pages](#) — The NetBSD Project
- [port-solaris mailing list, November 2010: syscall number reservation](#) — port-solaris list archive (Carnegie Mellon mirror)
- [NtTrace — Native API tracing for Windows](#) — Roger Orr
- [strace — linux syscall tracer](#) — The strace project
- [DTrace on Windows — Windows drivers](#) — Microsoft

- [DTrace — Windows Commands](#) — Microsoft
- [Inside the Native API](#) — Mark Russinovich, Sysinternals
- [NT OS/2 Subsystem Design Rationale \(NT Design Workbook\)](#) — Mark H. Lucovsky, Microsoft
- [The Sad History of the Microsoft POSIX Subsystem](#) — Brian Reiter (2010)
- [Syscall ABI compatibility: Linux vs Windows/macOS](#) — Mark Mossberg (2023)
- [Never break userspace](#) — LWN.net
- [Updating Syscall Tables — SysWhispers4 documentation](#) — SysWhispers4 project documentation
- [CRT-free in 2023: tips and tricks — discussion](#) — Lobsters discussion thread
- [Architecture of Windows NT](#) — Wikipedia
- [U.S. v. Microsoft Corporation — Information on the Settlement](#) — U.S. Department of Justice, Antitrust Division
- [Syscalls on Windows 11](#) — hammertux

10.39 V39 — The Trade-off Between Observation Cost and Blind Spots in Tracing Mechanisms

The note's central claim is that a tracer is an edit to the program it observes, not a window onto it: wherever a mechanism attaches — kernel syscall boundary, a breakpoint in a user-mode DLL, every instruction via a JIT'd code cache, a verified in-kernel probe, or a provider that agreed in advance to emit events — that single choice simultaneously fixes what the observation costs and what it structurally cannot see. The two are read from the same decision, not set independently. The note backs this with a comparison across six mechanisms. `ptrace / strace` stops the traced process twice per syscall and costs one to two orders of magnitude (442× on a syscall-saturated benchmark, ~3× on a realistic build), and cannot see anything that does not cross the syscall boundary, including `io_uring` traffic. NtTrace breakpoints the Windows Native API in user mode, avoiding administrative privileges at the cost of missing driver-originated kernel calls and serialising concurrent threads. DynamoRIO instruments every instruction via a code cache at 2–10× overhead for real clients, and can alter timing enough to manufacture bugs that do not occur natively. eBPF and DTrace attach in-kernel at nanosecond-to-low-microsecond cost and are treated as production-safe, but silently drop events under recursion or overload. ETW is a provider/consumer architecture with kernel-managed buffering, cheap and crash-immune, but shows only what a provider was

written to emit in advance. Record-and-replay (`rr`) and statistical sampling are offered as alternatives that trade the classic cost/blind-spot pair for different limitations — serialised execution and resolution/rarity, respectively.

The note is explicit about where this claim does and does not bind. It states two boundary conditions under which the trade stops mattering: when event rates are low enough that cost becomes irrelevant, and when a bug is both inside a mechanism's field of view and not timing-sensitive, so perturbation cannot change the answer. It flags one point as genuinely contested rather than settled — whether in-kernel tracing (eBPF/ DTrace) is cheap enough to run continuously at fleet scale, where Brendan Gregg and Dan Luu are read as answering different questions rather than being reconciled. It also lists open gaps: no published per-call overhead measurement exists for NtTrace, ETW's headline cycle-cost figures trace to Pentium III-era Microsoft benchmarking of un-stated modern validity, and no source quantifies how often eBPF's silent event drops actually occur under a characterized workload. The note treats this synthesis as evidence-backed (27 of 30 claims verified against primary sources) rather than speculative, but is careful to mark which specific claims rest on vendor self-assessment, a single practitioner blog, or unmeasured structural argument rather than independent benchmarking.

Referenced sources

- [strace Wow Much Syscall](#) — Brendan Gregg (2014)
- [Ptrace, Utrace, Uprobes: Lightweight, Dynamic Tracing of User Apps](#) — Keniston, Panchamukhi, Hiramatsu, Ottawa Linux Symposium (2007)
- [Introducing strace --seccomp-bpf](#) — Paul Chaignon (2019)
- [strace: fight for performance \(FOSDEM 2020 slides\)](#) — strace maintainers, FOSDEM (2020)
- [Sysdig vs DTrace vs Strace: A technical discussion](#) — Sysdig (n.d.)
- [NtTrace — Native API tracing for Windows](#) — Roger Orr (n.d.)
- [Notes and Thoughts on Strace for Windows](#) — Jarek Przygodzki (n.d.)
- [Transparent Dynamic Instrumentation](#) — Derek Bruening, Qin Zhao, Saman Amarasinghe, VEE (2012)
- [DynamoRIO Release Notes](#) — DynamoRIO project (n.d.)
- [Dynamic binary instrumentation \(DBI\) with DynamoRio](#) — Cisco Talos Intelligence (2025)
- [Tracing a Subset of Execution](#) — DynamoRIO project (n.d.)
- [DTrace for Linux 2016](#) — Brendan Gregg (2016)
- [Benchmarking Performance Overhead of DTrace on FreeBSD and eBPF on Linux](#) — Mateusz Piotrowski, AsiaBSDCon (2024)

- [ebpf_exporter eBPF overhead benchmark](#) — Cloudflare (n.d.)
- [Pitfalls of relying on eBPF for security monitoring \(and some solutions\)](#) — Artem Dinaburg, Trail of Bits (2023)
- [eBPF Hookpoints Gotchas \(FOSDEM 2026 slides\)](#) — FOSDEM (2026)
- [Instrumenting Your Code with ETW](#) — Microsoft (n.d.)
- [Event Tracing for Windows \(ETW\)](#) — Ryan Myers, Microsoft (archived blog, n.d.)
- [Performance Analysis with ETW: Event Tracing for Windows](#) — Atomic Object (n.d.)
- [Heisenbug](#) — Wikipedia (2026)
- [Engineering Record And Replay For Deployability](#) — O'Callahan, Jones, Froyd, Huey, Noll, Partush, USENIX ATC (2017)
- [rr: lightweight recording & deterministic debugging](#) — rr project (2024)
- [Sampling v. tracing](#) — Dan Luu (n.d.)
- [eBPF monitoring overhead: measure and reduce it](#) — groundcover (n.d.)
- [strace of io_uring events? \(strace-devel mailing list\)](#) — strace-devel (2020)
- [strace\(1\)](#) — Linux manual page — man7.org (n.d.)
- [DTrace pid Provider Overhead](#) — Brendan Gregg (2011)
- [Portable, Low-Overhead Observability for io_uring](#) — arXiv preprint (2026)
- [io_uring War Stories](#) — kernel-internals.org (n.d.)
- [Transparently Integrating Debugging and Dynamic Binary Instrumentation](#) — Rensselaer Polytechnic Institute (n.d.)
- [Event Tracing for Windows \(ETW\) — architecture and providers](#) — Salucci (n.d.)
- [Bypassing ETW For Fun and Profit](#) — White Knight Labs (2021)
- [Support io_uring](#) — rr issue #2613 — Robert O'Callahan, rr project (2020)
- [Sampling vs deterministic profilers: which should I use?](#) — Python Developer Tooling Handbook (n.d.)

10.40 V40 — The 2018 fsync() Error-Reporting Failure ("Fsyncgate")

This note covers LWN's 2018 report on a Linux kernel behaviour that caused PostgreSQL to silently lose committed data. When a buffered write's later writeback to disk fails, Linux marks the dirty page clean and discards it, recording the failure as a one-shot flag on the file. The first `fsync()` call after the failure reports the error — but the act of reporting clears the flag. A second `fsync()` then finds no dirty pages and no set flag, and truthfully (by its own narrow semantics) returns success, even though the data never reached storage and is no longer recoverable from the kernel. PostgreSQL's

checkpoint, architected to open files rather than hold every descriptor open, retried after the first failure, saw success, and recycled its write-ahead log — destroying the only remaining copy of the lost writes. The note traces the evidence for this mechanism: the original `pgsql-hackers` report, a reproduction under `strace` with a fault-injecting block device, and the convergent response of three independent storage engines (PostgreSQL, InnoDB, WiredTiger), all of which replaced retry with a hard crash-and-replay-from-log response during 2018, reasoning that once an unreported failure may have occurred, the in-memory model of what is on disk cannot be trusted and must be rebuilt from the log instead.

The note is explicit about what remains unresolved. It is contemporaneous reporting on a then-live, unresolved mailing-list thread, so it presents proposals rather than settled outcomes, and some of those proposals (a `syncfs()`-based polling fix) were not how any affected database actually solved the problem. Whether POSIX's text actually settles the dispute in PostgreSQL's favour, and whether the kernel's read-and-clear behaviour is a defect or a coherent but undocumented design choice, are both treated as contested rather than resolved — kernel developers defended the semantics as necessary given bounded memory for dirty pages. The note also flags that a later academic fault-injection study found that no application, PostgreSQL included, handles `fsync` failure perfectly even after the 2018 fixes, and that direct I/O — named by both sides as the real long-term fix — still has not shipped as a production path years later.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018)
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (archive of the `pgsql-hackers` thread, 2018)
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1 — "PANIC on fsync\(\) failure"](#) — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018)
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group
- [MySQL Bug #90296 — "Hard error should be report when fsync\(\) return EIO"](#) — Oracle / MySQL bug tracker (2018)
- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a — Bug#27805553](#) — Oracle / MySQL Server (2018)
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96 — WT-4045 "Don't retry fsync calls after EIO failure"](#) — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018)
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020)

- fs: introduce new writeback error reporting and convert existing API as a wrapper around it — Jeff Layton, linux-fsdevel (2017)
- PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs — Jeff Layton, LKML (2018)
- Linux Filesystems API summary — `filemap_set_wb_err` / `filemap_check_wb_err` — The Linux Kernel documentation
- PostgreSQL 18 and beyond: From AIO to Direct IO? — Cybertec PostgreSQL International (2025)
- `debug_io_direct` — pgPedia — pgPedia, a PostgreSQL Encyclopedia
- All Your GUCs in a Row: `data_sync_retry` — The Build (PostgreSQL Experts)
- `fsync` — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications

10.41 V41 — Linux's one-shot `fsync()` error report and its 2018 database-engine fallout

The note establishes that Linux's `fsync()` reports a writeback error to the caller only once: the failed page is marked clean and its data discarded by the kernel, and the act of reporting the error clears the flag that recorded it. A subsequent `fsync()` call therefore finds nothing wrong and returns success — not because the data was recovered, but because the kernel's own bookkeeping has already forgotten the failure. This is the mechanism behind 2018's "fsyncgate": PostgreSQL's checkpointer retried a failed `fsync()`, saw success, and recycled its write-ahead log, permanently losing data it believed was safely on disk. The note treats this as well-established rather than speculative — the sequence is reproduced under `strace` with a fault-injecting device, independently confirmed by a peer-reviewed fault-injection study across three filesystems, and stated in the commit messages of three separate storage engines. PostgreSQL, MySQL's InnoDB, and MongoDB's WiredTiger each independently concluded, within months of each other in 2018, that retrying a failed `fsync()` cannot be made safe, and replaced retry logic with a hard crash followed by replay from the durable log — treating any unreported write failure as reason to discard the in-memory model of what is on disk entirely rather than repair it.

The note is explicit about what remains unsettled. It does not claim the kernel's behavior is a defect: kernel developers defend the one-shot report as a coherent, if undocumented, tradeoff against unbounded memory held by permanently-dirty pages, and POSIX itself only specifies the success case for `fsync()`, saying nothing about what is guaranteed after a failure — so the note treats "whose fault this is" as a live, unresolved dispute rather than a resolved question. It also does not claim the fix is complete: the

kernel-side `errseq_t` change adopted afterward makes error *reporting* more reliable but does not make lost *data* recoverable, and PostgreSQL's own commit acknowledges a residual window where an error can still be missed. Direct I/O, named by both sides as the real long-term fix, remains an explicitly unsupported debug-only feature years later. The note flags, without resolving, that any software using buffered writes plus `fsync()` shares this exposure and most has likely never been audited for it.

Referenced sources

- [PostgreSQL's fsync\(\) surprise](#) — Jonathan Corbet / LWN.net (2018)
- [Fsyncgate: errors on fsync are unrecoverable](#) — Dan Luu (archive of the postgresql-hackers thread, 2018)
- [PostgreSQL commit 9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1](#) — "PANIC on fsync() failure" — Craig Ringer / Thomas Munro, PostgreSQL Global Development Group (2018)
- [PostgreSQL documentation — data_sync_retry](#) — PostgreSQL Global Development Group
- [MySQL Bug #90296](#) — "Hard error should be report when fsync() return EIO" — Oracle / MySQL bug tracker (2018)
- [mysql-server commit 8590c8e12a3374eecb547359750a9d2a128fa6a](#) — Bug#27805553 — Oracle / MySQL Server (2018)
- [WiredTiger commit ae8bccce3d8a8248afa0e4e0cf67674a43dede96](#) — WT-4045 "Don't retry fsync calls after EIO failure" — Keith Bostic / Alex Gorrod, MongoDB WiredTiger (2018)
- [Can Applications Recover from fsync Failures?](#) — Rebello, Patel, Alagappan, Arpaci-Dusseau & Arpaci-Dusseau, USENIX ATC '20 (2020)
- [fs: introduce new writeback error reporting and convert existing API as a wrapper around it](#) — Jeff Layton, linux-fsdevel (2017)
- [PATCH v2 3/5 vfs: track per-sb writeback errors and report them to syncfs](#) — Jeff Layton, LKML (2018)
- [Linux Filesystems API summary — filemap_set_wb_err / filemap_check_wb_err](#) — The Linux Kernel documentation
- [PostgreSQL 18 and beyond: From AIO to Direct IO?](#) — Cybertec PostgreSQL International (2025)
- [debug_io_direct](#) — pgPedia — pgPedia, a PostgreSQL Encyclopedia
- [All Your GUCs in a Row: data_sync_retry](#) — The Build (PostgreSQL Experts)
- [fsync](#) — POSIX Programmer's Manual — IEEE Std 1003.1 / The Open Group Base Specifications